Subroutines in ARM Assembly

CSE 4095

Special Topics in Computer Engineering III:
Introduction to Embedded Systems
Spring 2023

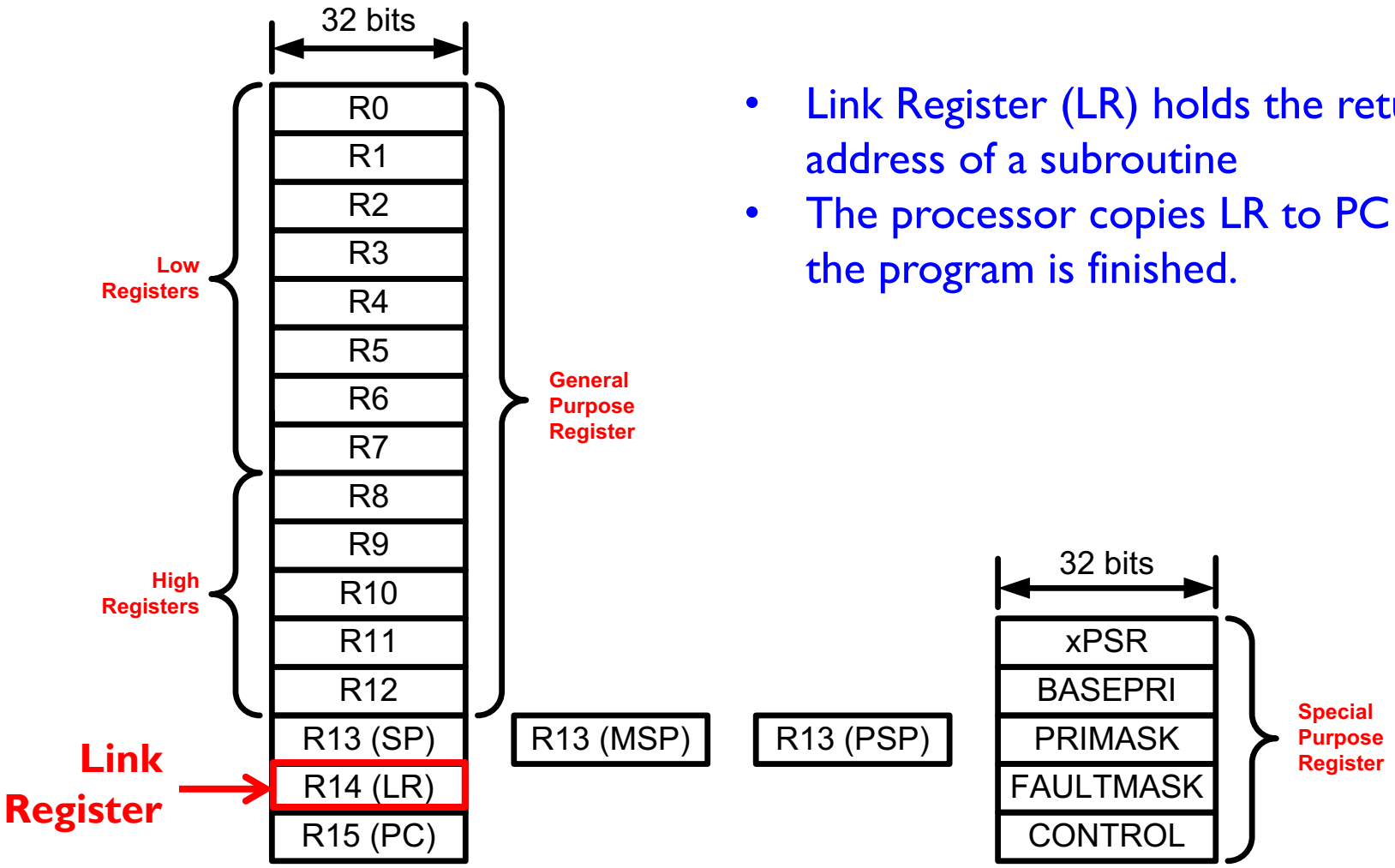
Taken from Dr. Yifeng Zhu's Lecture Notes

Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C – Chapter 8

Overview

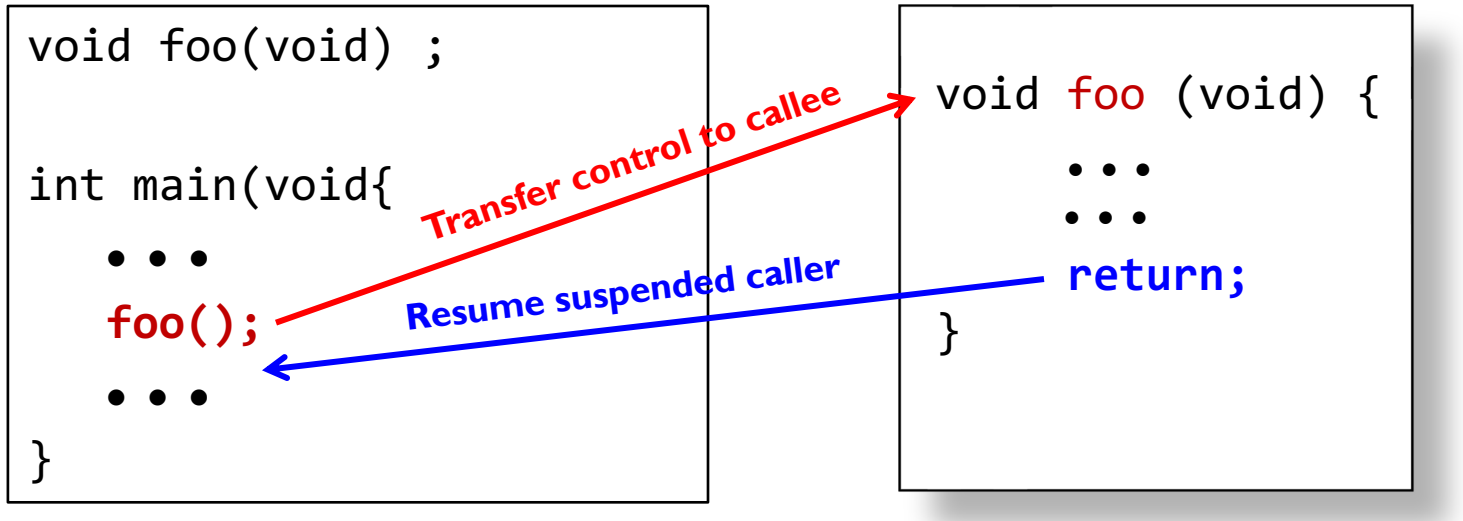
- How to call a subroutine?
- How to return the control back to the caller?
- How to pass arguments into a subroutine?
- How to return a value in a subroutine?
- How to preserve the running environment for the caller?

Link Register (LR)

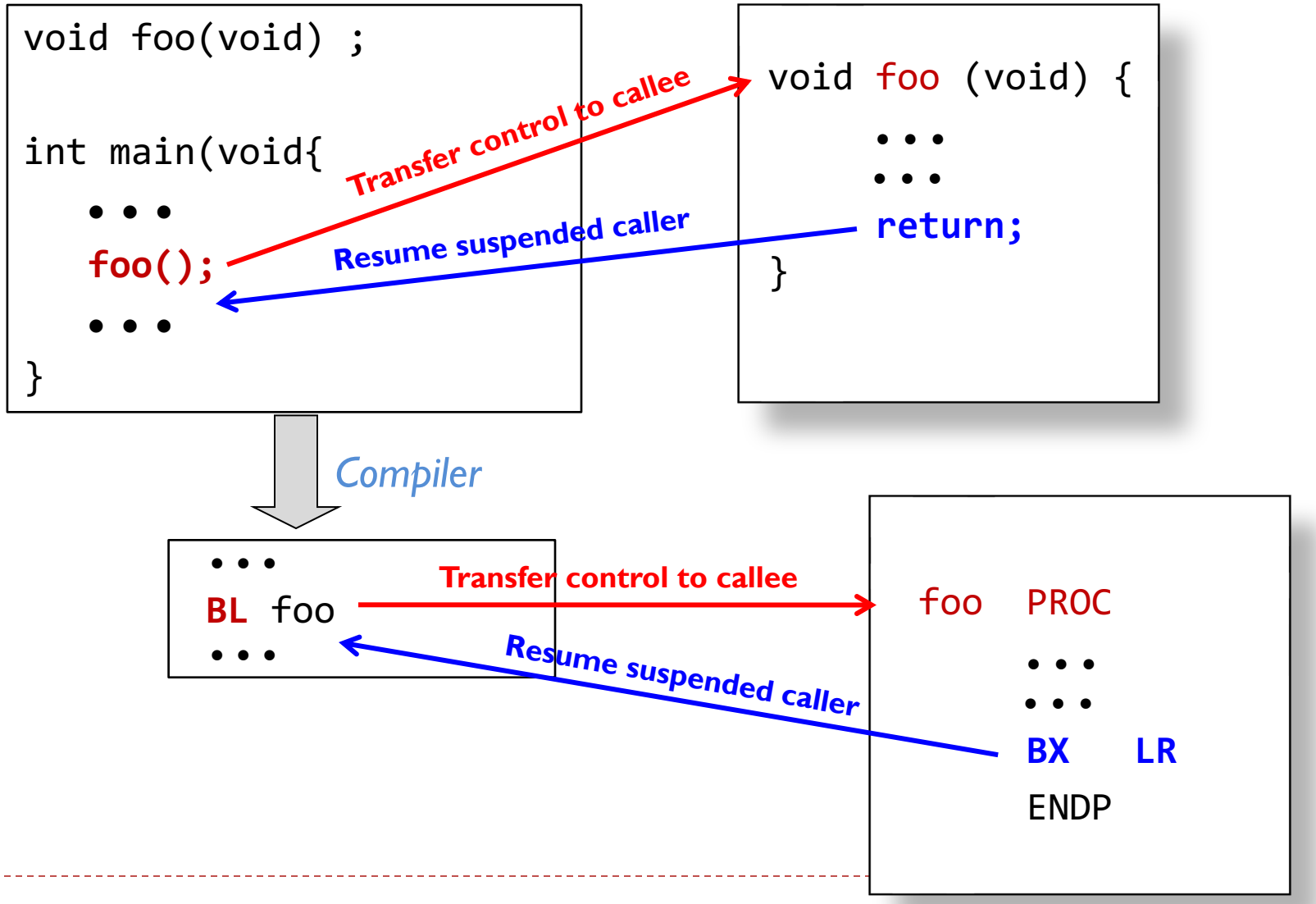


- Link Register (LR) holds the return address of a subroutine
- The processor copies LR to PC after the program is finished.

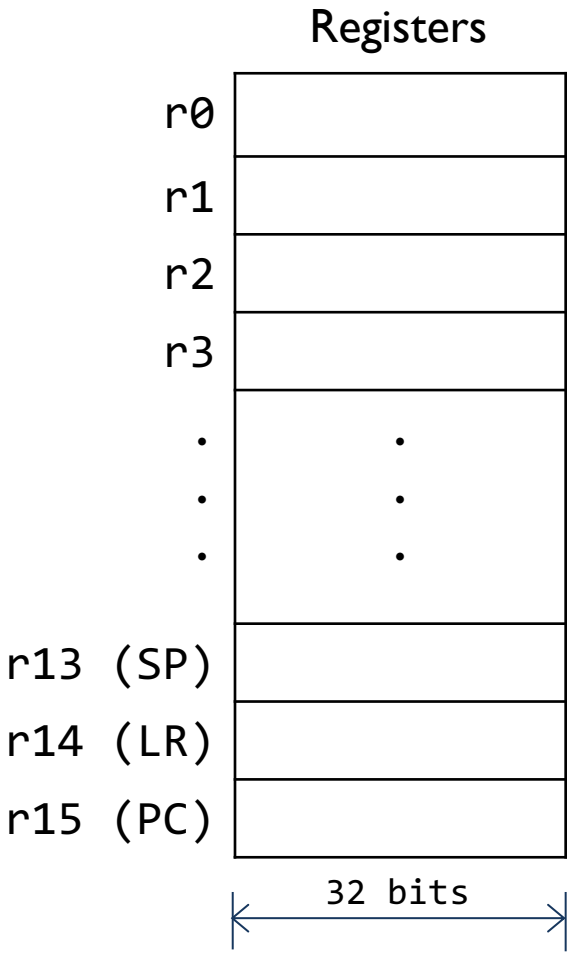
Link Register (LR)



Link Register (LR)



Link Register (LR)




```
void foo(void) ;

int main(void{
   • • •
   foo();
   • • •
}
```

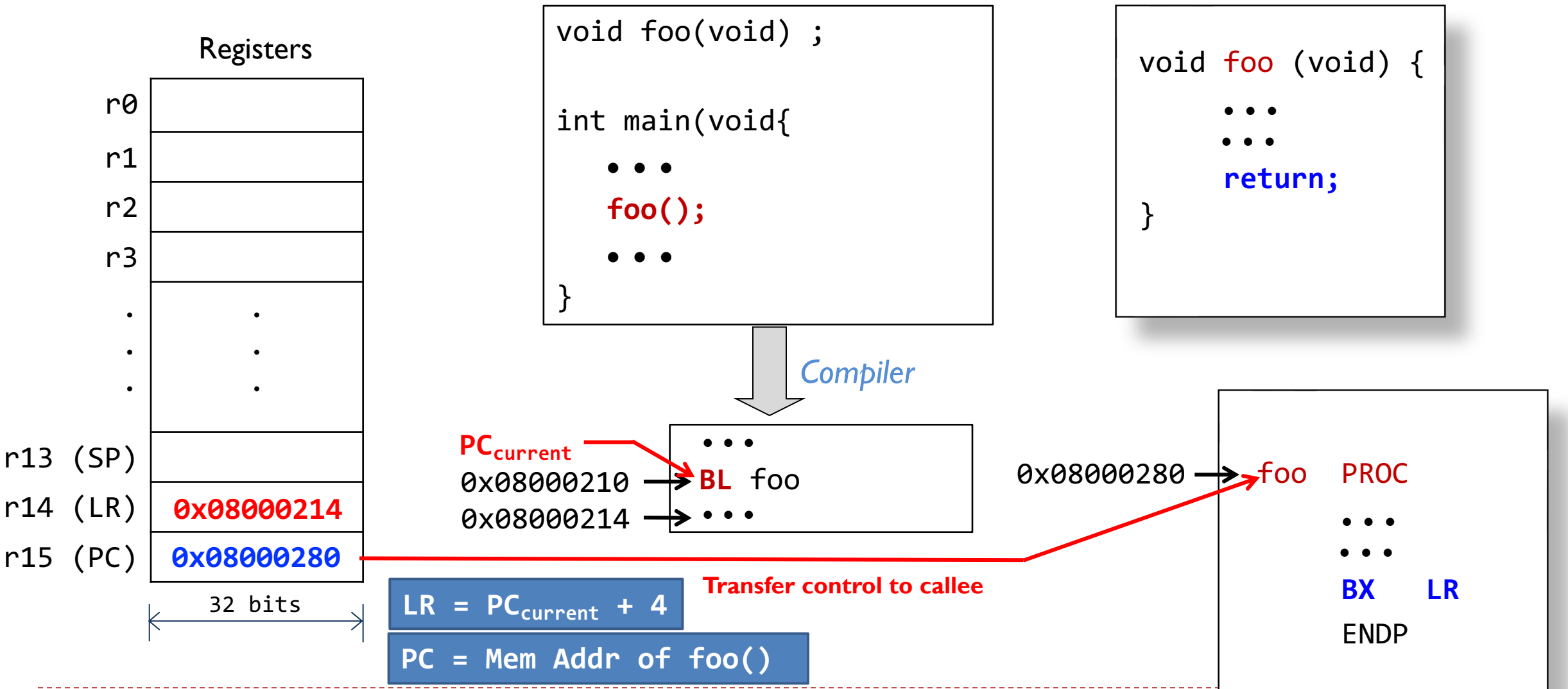
Compiler

```
• • •
BL foo
• • •
```

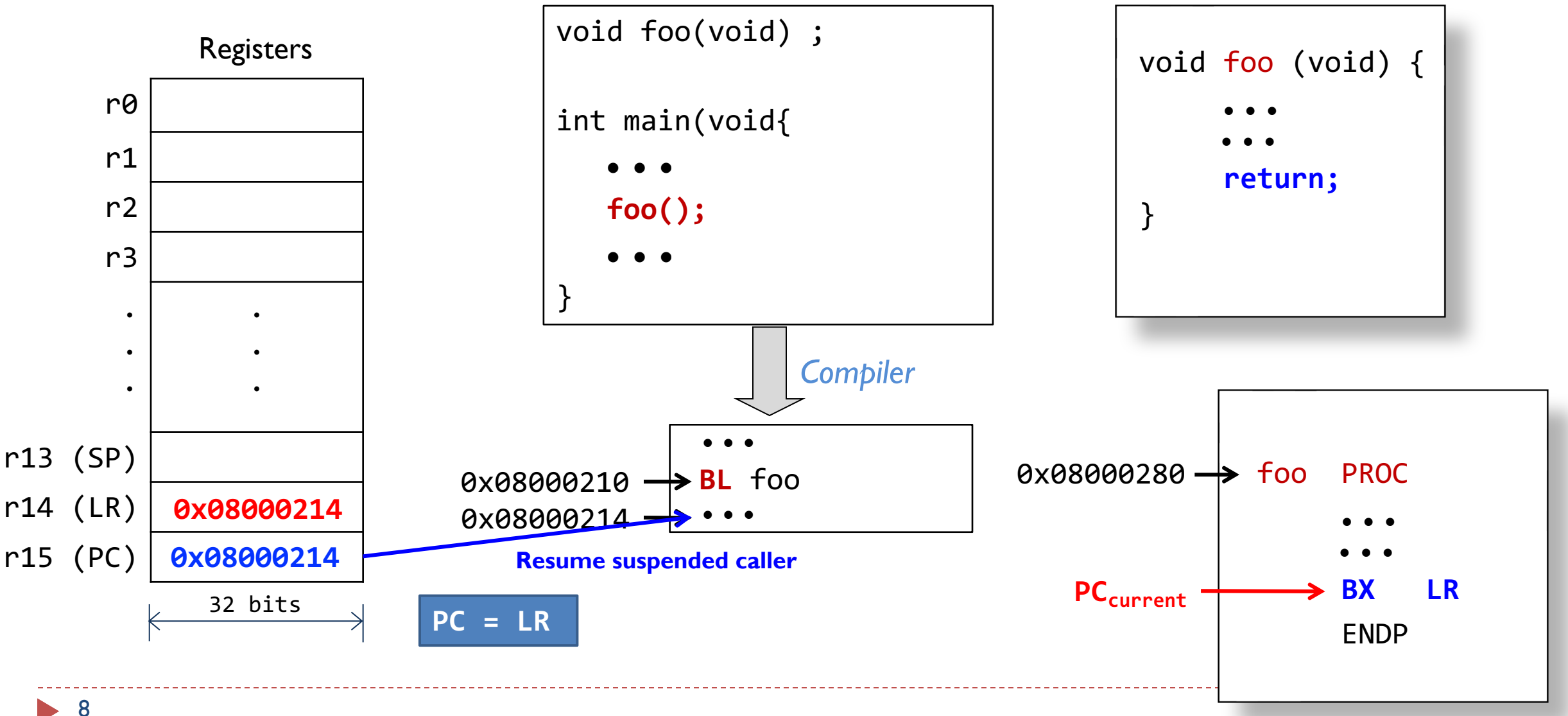
```
void foo (void) {
      • • •
      • • •
      return;
}
```

```
foo   PROC
      • • •
      • • •
      BX    LR
      ENDP
```

Link Register (LR)



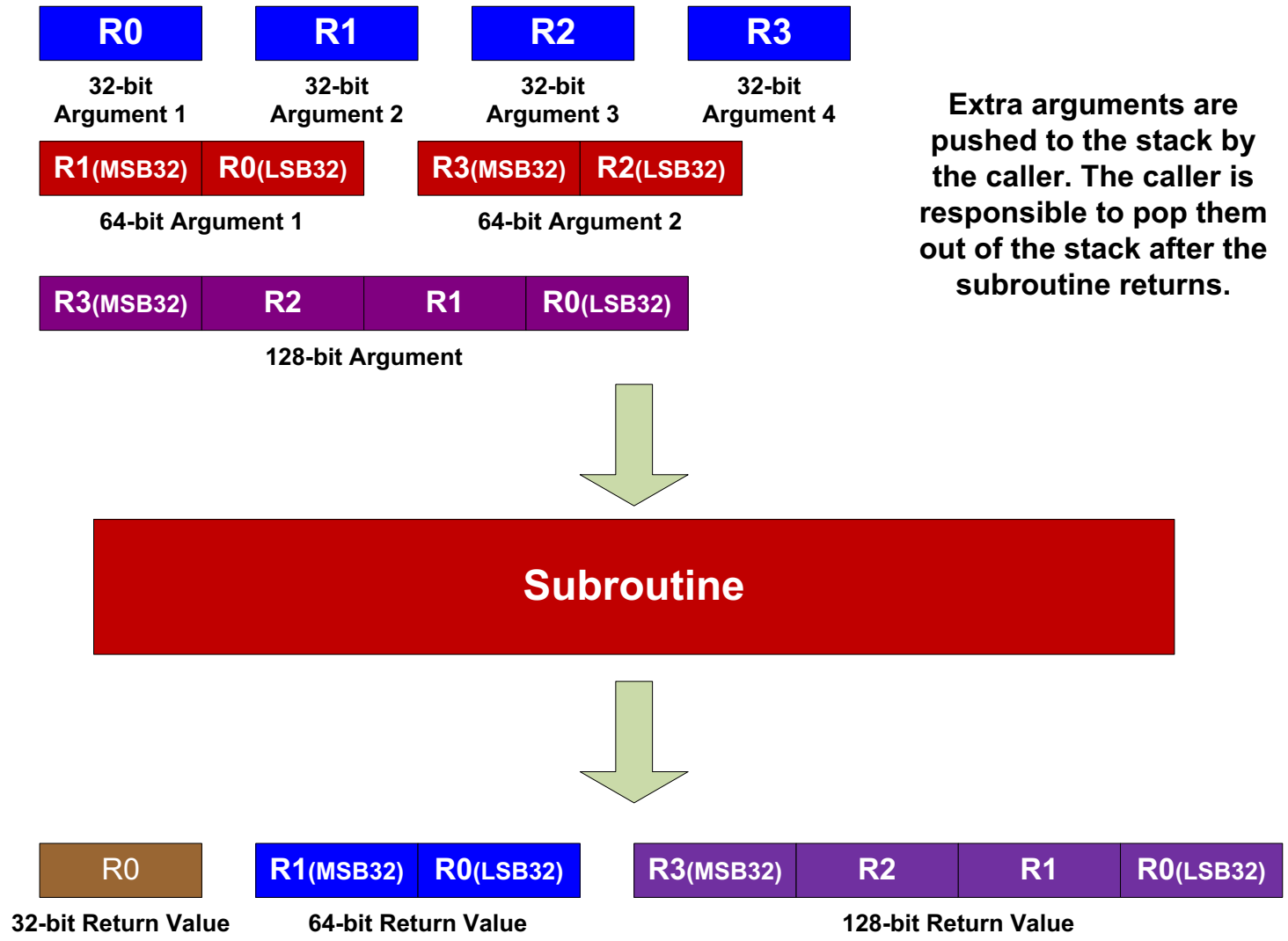
Link Register (LR)



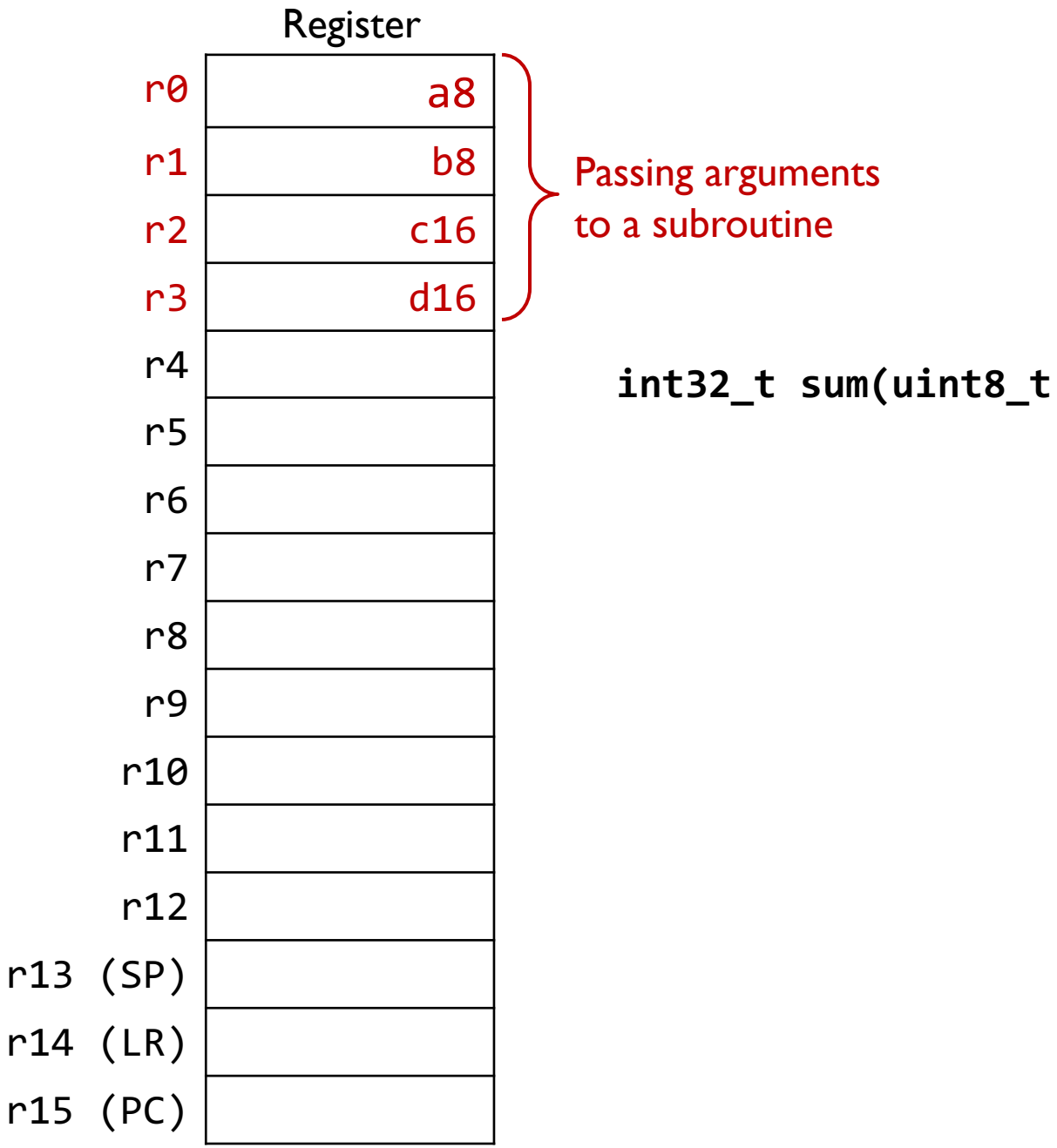
Procedure Call Standard

- What is it?
  - Contract between a calling subroutine (caller) and a called subroutine (callee)
- Why need it?
  - Allows subroutines to be separately written, compiled, assembled but work together
  - Allows C program calls an assembly function, or vice versa
- This talk focuses on
  - How to pass arguments to a subroutine?
  - How to return a result from a subroutine?

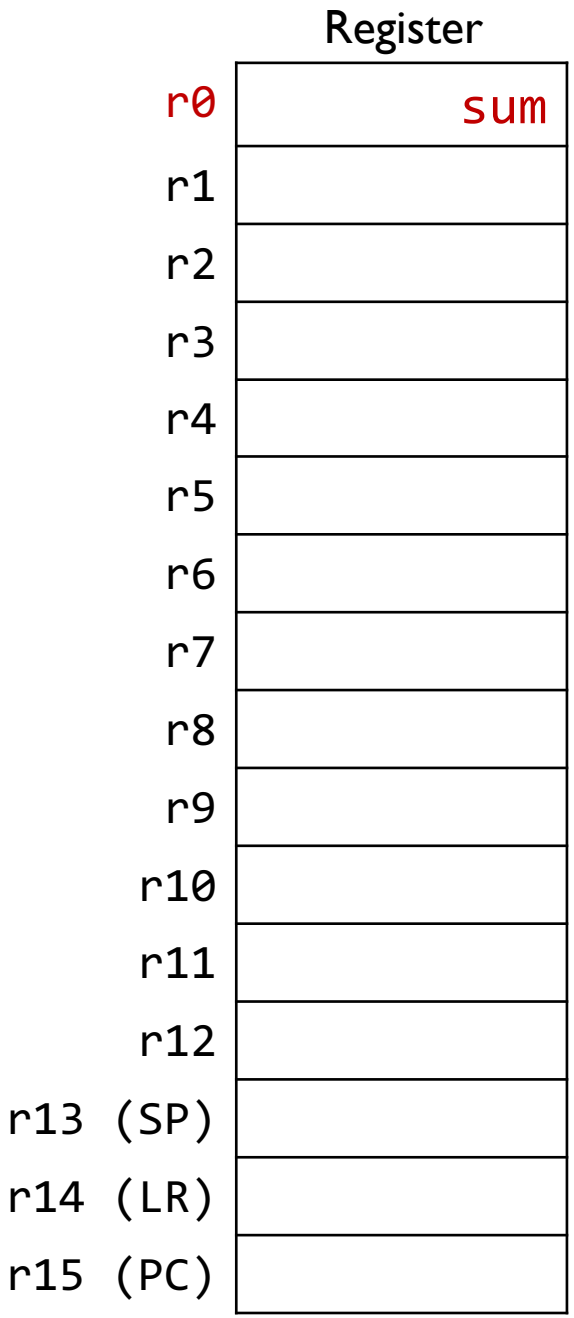
Passing Arguments and Returning Value



Extra arguments are pushed to the stack by the caller. The caller is responsible to pop them out of the stack after the subroutine returns.




```
int32_t sum(uint8_t a8, int8_t b8, uint16_t c16, uint16_t d16);
```



Return the sum in register r0

```
int32_t sum(uint8_t a8, int8_t b8, uint16_t c16, uint16_t d16);
```

Passing Arguments and Returning Value

```
int32_t sum(uint8_t a8, int8_t b8, uint16_t c16, uint16_t d16);

s = sum(1, 2, 3, 4);
```

Caller

```
  MOVS r0, #1   ; a8
  MOVS r1, #2   ; b8
  MOVS r2, #3   ; c16
  MOVS r3, #4   ; d16
  BL   sum
```

Callee

```
sum PROC
  ADD r0, r0, r1   ; a8 + b8
  ADD r0, r0, r2   ; add c16
  ADD r0, r0, r3   ; add d16
  BX  LR
  ENDP
```

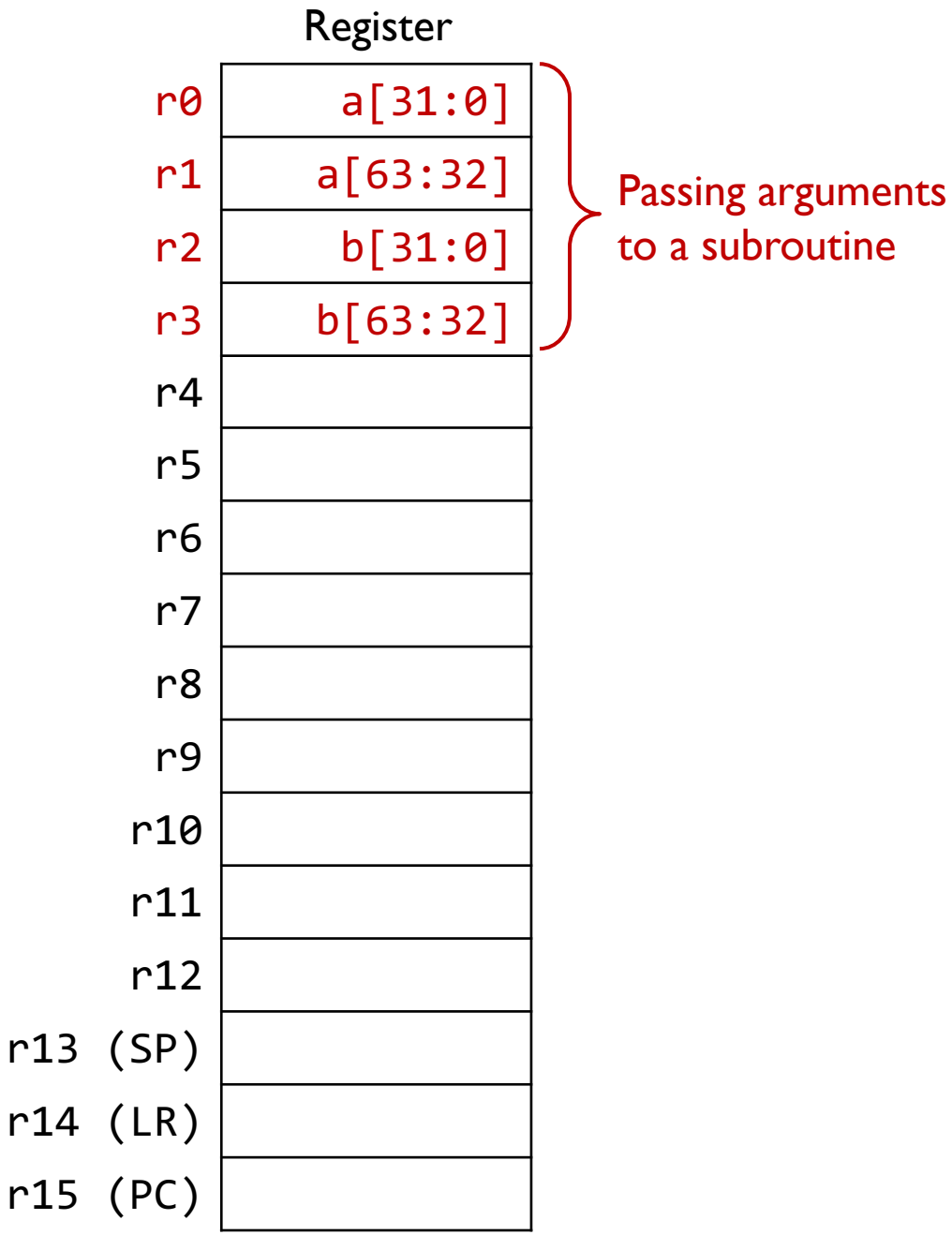
Returning Value

```
int32_t s32;

int32_t sum(uint8_t a8, int8_t b8, uint16_t c16, uint16_t d16);

s32 = sum(1, 2, 3, 4) + 100;
```

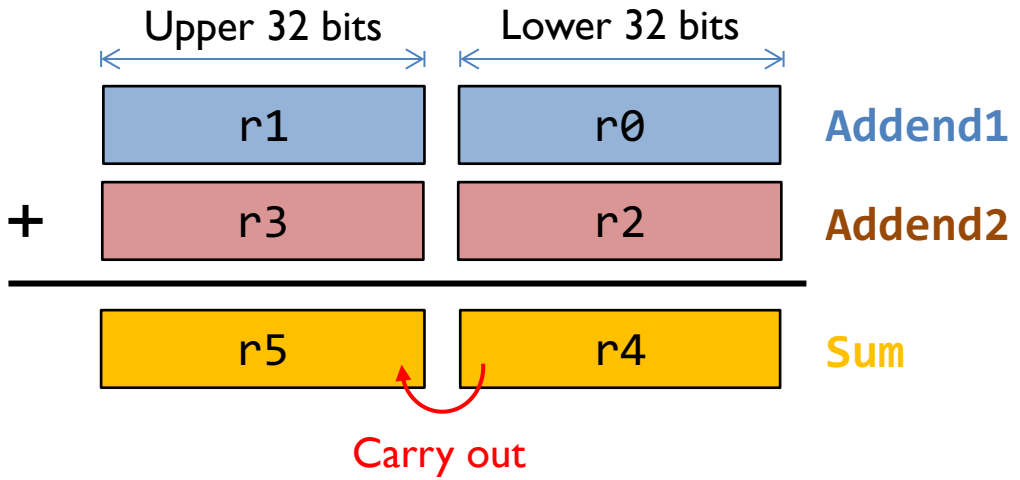
```
MOVS r0, #1   ; 1st argument a8
MOVS r1, #2   ; 2nd argument b8
MOVS r2, #3   ; 3rd argument c16
MOVS r3, #4   ; 4th argument d16
BL   sum      ; result is returned in r0
ADDS r0, r0, #100
```

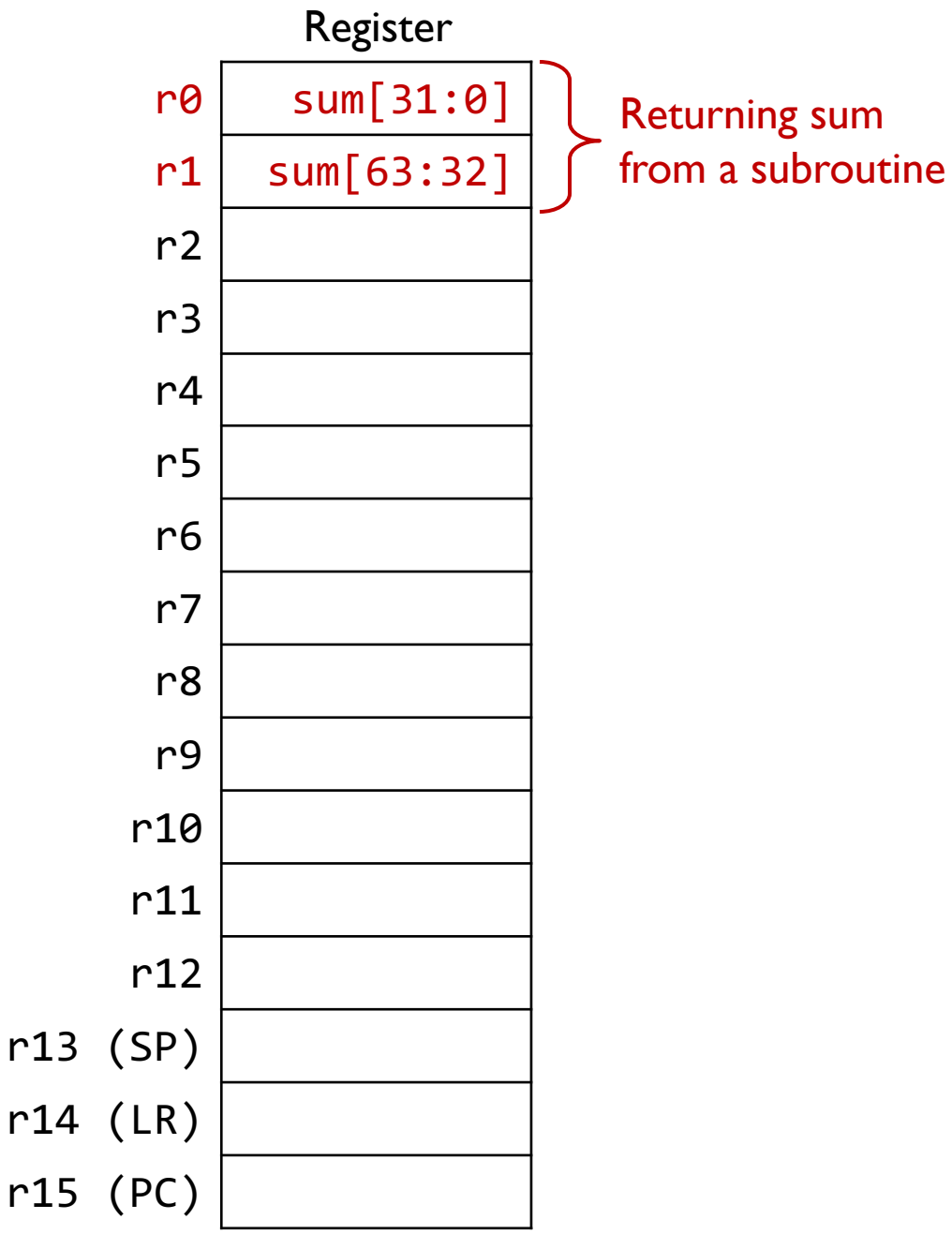



```
int64_t sum(int64_t a, int64_t b);
```

Callee

```
sum PROC
    ADDS r0, r2, r0  ; Adding lower 32 bits
    ADC  r1, r3, r1  ; Adding upper 32 bits
    BX   LR          ; Return in r1:r0
    ENDP
```

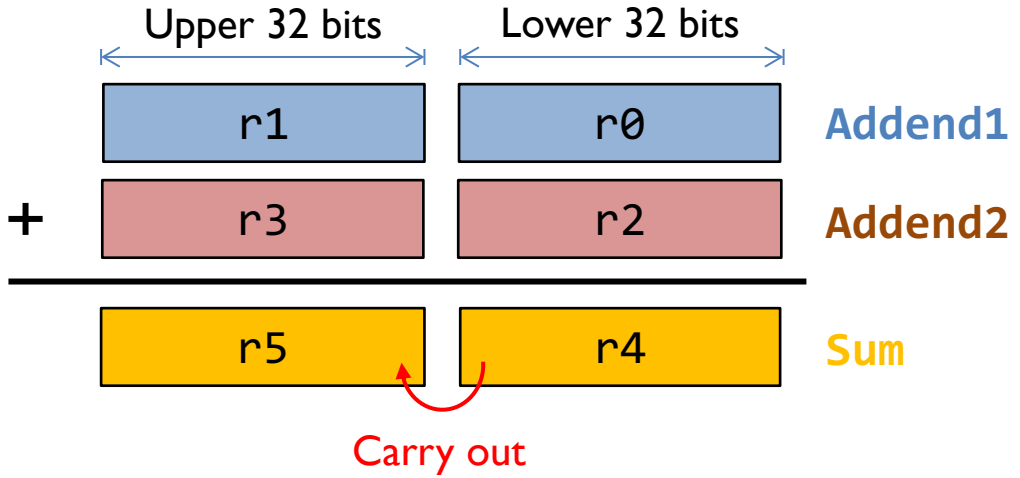


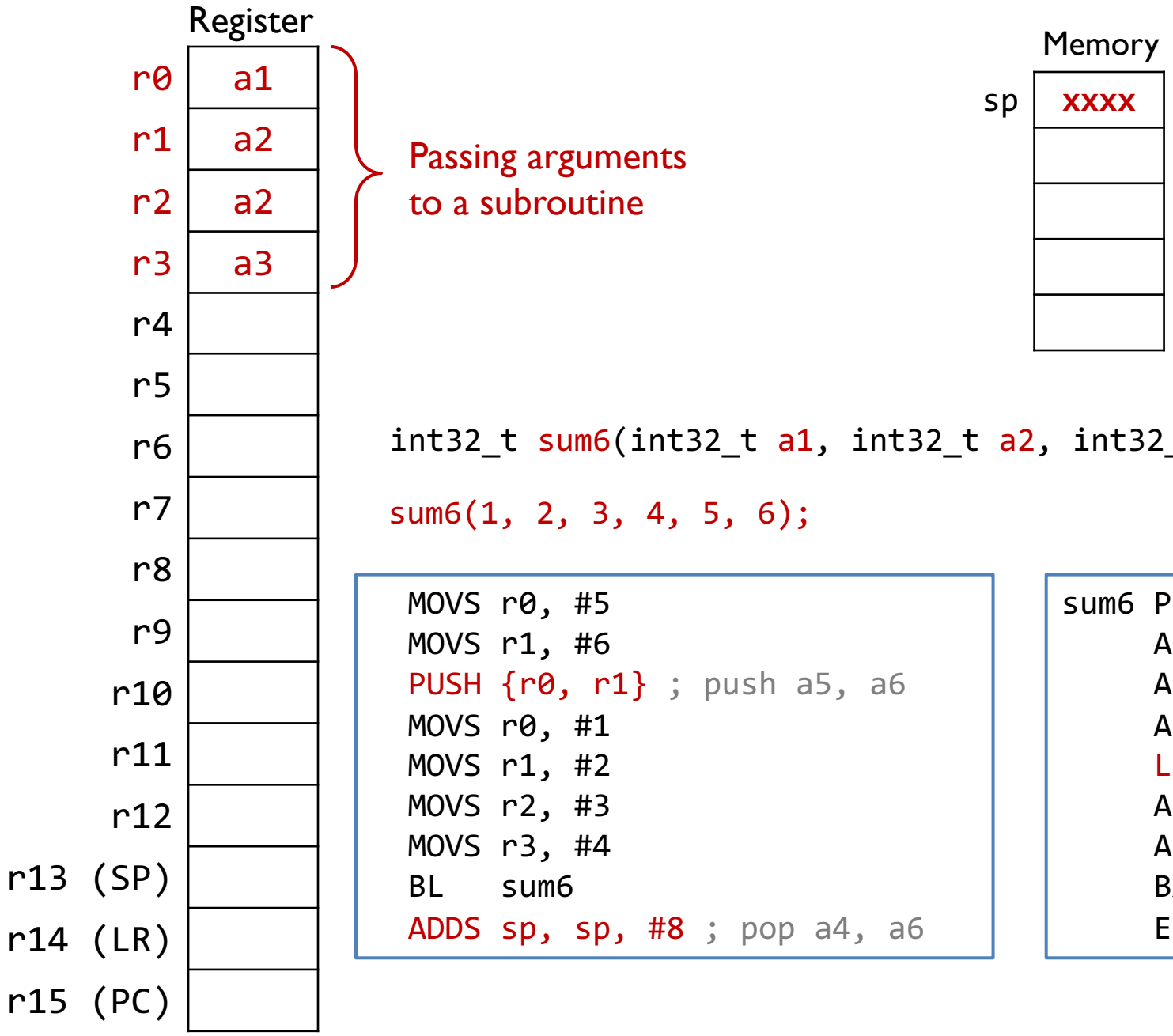



```
int64_t sum(int64_t a, int64_t b);
```

Callee

```
sum PROC
    ADDS r0, r2, r0  ; Adding lower 32 bits
    ADC  r1, r3, r1  ; Adding upper 32 bits
    BX   LR          ; Return in r1:r0
    ENDP
```



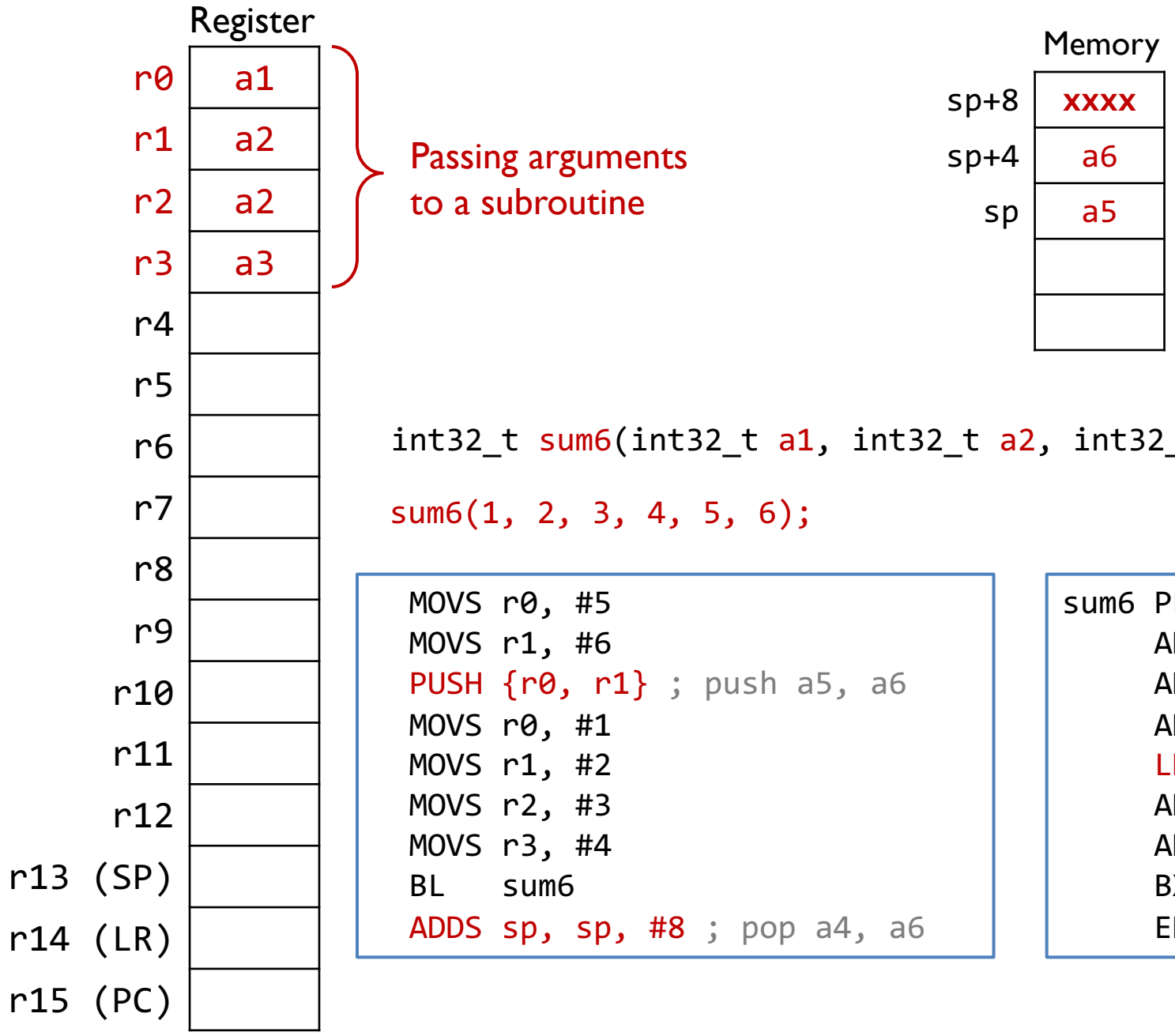



```
int32_t sum6(int32_t a1, int32_t a2, int32_t a3, int32_t a4, int32_t a5, int32_t a6)

sum6(1, 2, 3, 4, 5, 6);
```

```
MOVS r0, #5
MOVS r1, #6
PUSH {r0, r1} ; push a5, a6
MOVS r0, #1
MOVS r1, #2
MOVS r2, #3
MOVS r3, #4
BL   sum6
ADDS sp, sp, #8 ; pop a4, a6
```

```
sum6 PROC
     ADD  r0, r0, r1    ; sum = a1 + a2
     ADD  r0, r0, r2    ; sum += a3
     ADD  r0, r0, r3    ; sum += a4
     LDRD r1, r2, [sp] ; load a5, a6
     ADD  r0, r0, r1    ; sum += a5
     ADD  r0, r0, r2    ; sum += a6
     BX   LR            ; return in r0
     ENDP
```

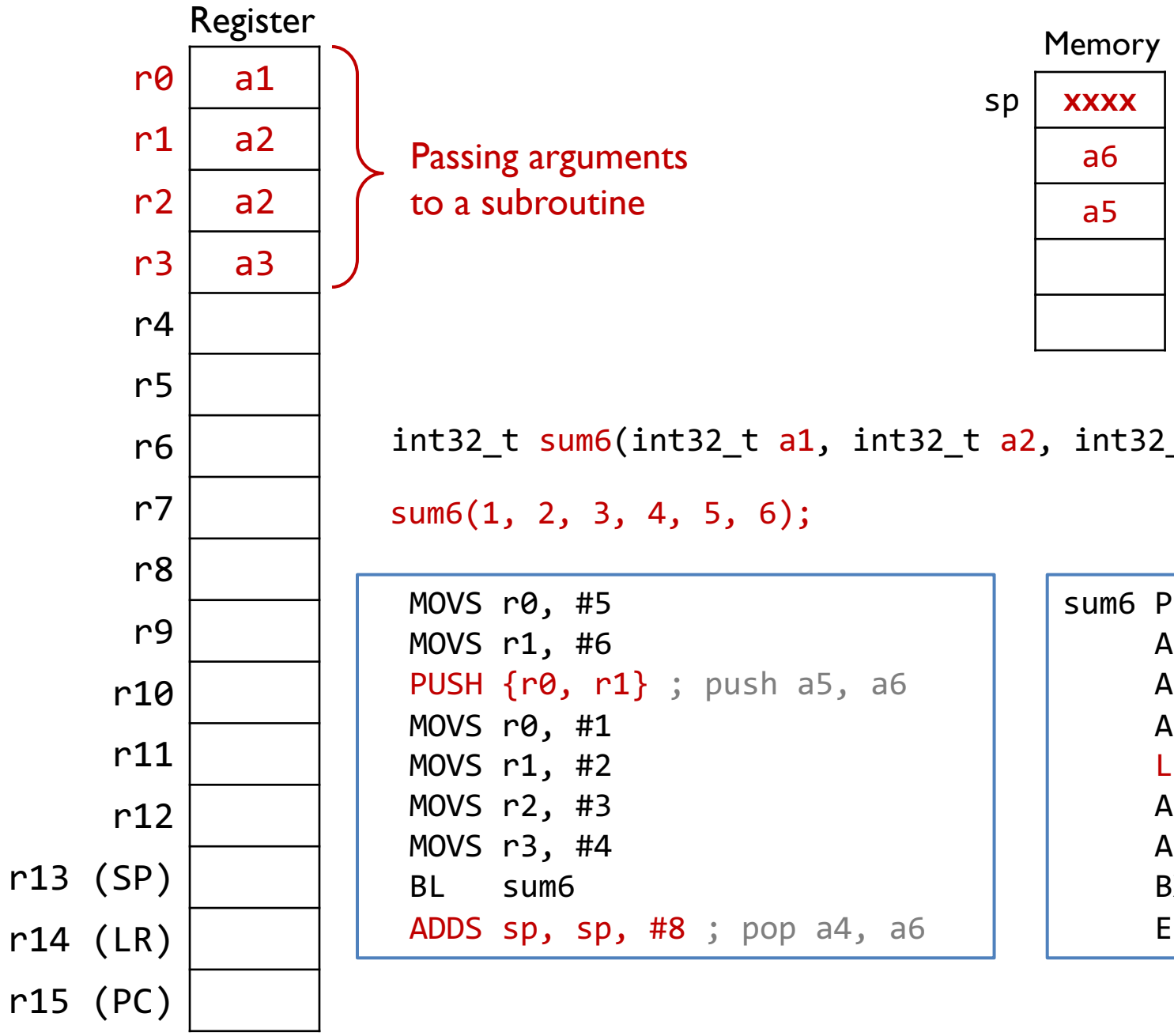



```
int32_t sum6(int32_t a1, int32_t a2, int32_t a3, int32_t a4, int32_t a5, int32_t a6)

sum6(1, 2, 3, 4, 5, 6);
```

```
MOVS r0, #5
MOVS r1, #6
PUSH {r0, r1} ; push a5, a6
MOVS r0, #1
MOVS r1, #2
MOVS r2, #3
MOVS r3, #4
BL   sum6
ADDS sp, sp, #8 ; pop a4, a6
```

```
sum6 PROC
     ADD  r0, r0, r1    ; sum = a1 + a2
     ADD  r0, r0, r2    ; sum += a3
     ADD  r0, r0, r3    ; sum += a4
     LDRD r1, r2, [sp] ; load a5, a6
     ADD  r0, r0, r1    ; sum += a5
     ADD  r0, r0, r2    ; sum += a6
     BX   LR            ; return in r0
     ENDP
```




```
int32_t sum6(int32_t a1, int32_t a2, int32_t a3, int32_t a4, int32_t a5, int32_t a6)

sum6(1, 2, 3, 4, 5, 6);
```

```
MOVS r0, #5
MOVS r1, #6
PUSH {r0, r1} ; push a5, a6
MOVS r0, #1
MOVS r1, #2
MOVS r2, #3
MOVS r3, #4
BL   sum6
ADDS sp, sp, #8 ; pop a4, a6
```

```
sum6 PROC
     ADD  r0, r0, r1    ; sum = a1 + a2
     ADD  r0, r0, r2    ; sum += a3
     ADD  r0, r0, r3    ; sum += a4
     LDRD r1, r2, [sp] ; load a5, a6
     ADD  r0, r0, r1    ; sum += a5
     ADD  r0, r0, r2    ; sum += a6
     BX   LR            ; return in r0
     ENDP
```

Returning Value

```
uint32_t s32;

uint32_t sum(uint8_t a8, uint8_t b8, uint16_t c16, uint16_t d16);

s32 = sum(1, 2, 3, 4) + 100;
```

```
MOVS r0, #1   ; 1st argument a8
MOVS r1, #2   ; 2nd argument b8
MOVS r2, #3   ; 3rd argument c16
MOVS r3, #4   ; 4th argument d16
BL   sum      ; result is returned in r0
ADDS r0, r0, #100
LDR  r4, =s32  ; Get memory address of s32
STR  r0, [r4]  ; Save returned result to s32
```

Calling Assembly Subroutine in C

- If your assembly code follows the procedure call standard, a C code can call an assembly subroutine, and vice versa.

```
extern int32_t sum3(int32_t a1, int32_t a2, int32_t a3);


int main(void){
  int32_t s

  ...
  s = sum3(-1, -2, -3) + sum3(4, 5, 6);
  ...

}
```

```
sum3 PROC
     EXPORT sum3
     ADD  r0, r0, r1    ; sum = a1 + a2
     ADD  r0, r0, r2    ; sum += a3
     BX   LR            ; return in r0
     ENDP
```

Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX LR
        ENDP
        ...
```

Sum of Square: $x^2 + y^2$

R1: second argument

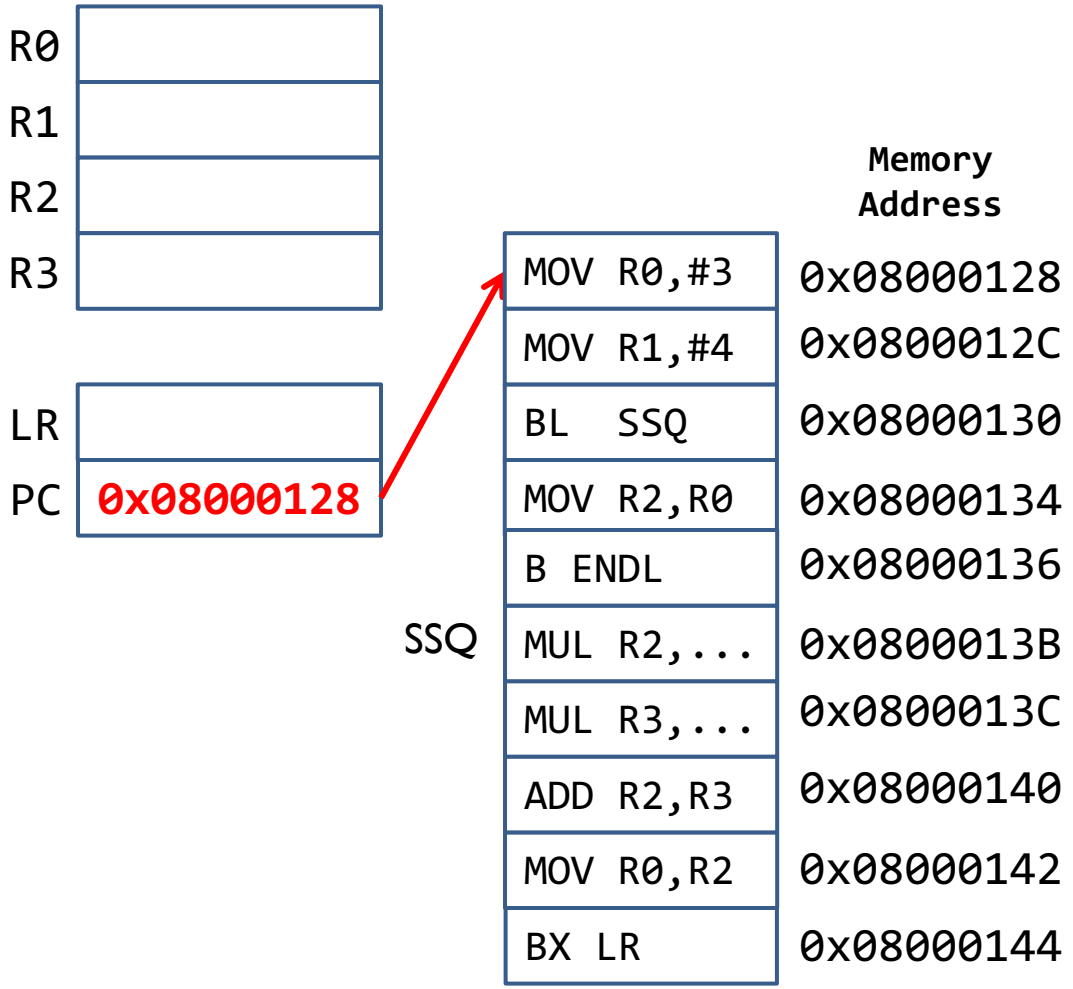
R0: first argument

```
int SSQ(int x, int y){
      int z;
      z = x*x + y*y;
      return z;
}
```

R0: Return Value

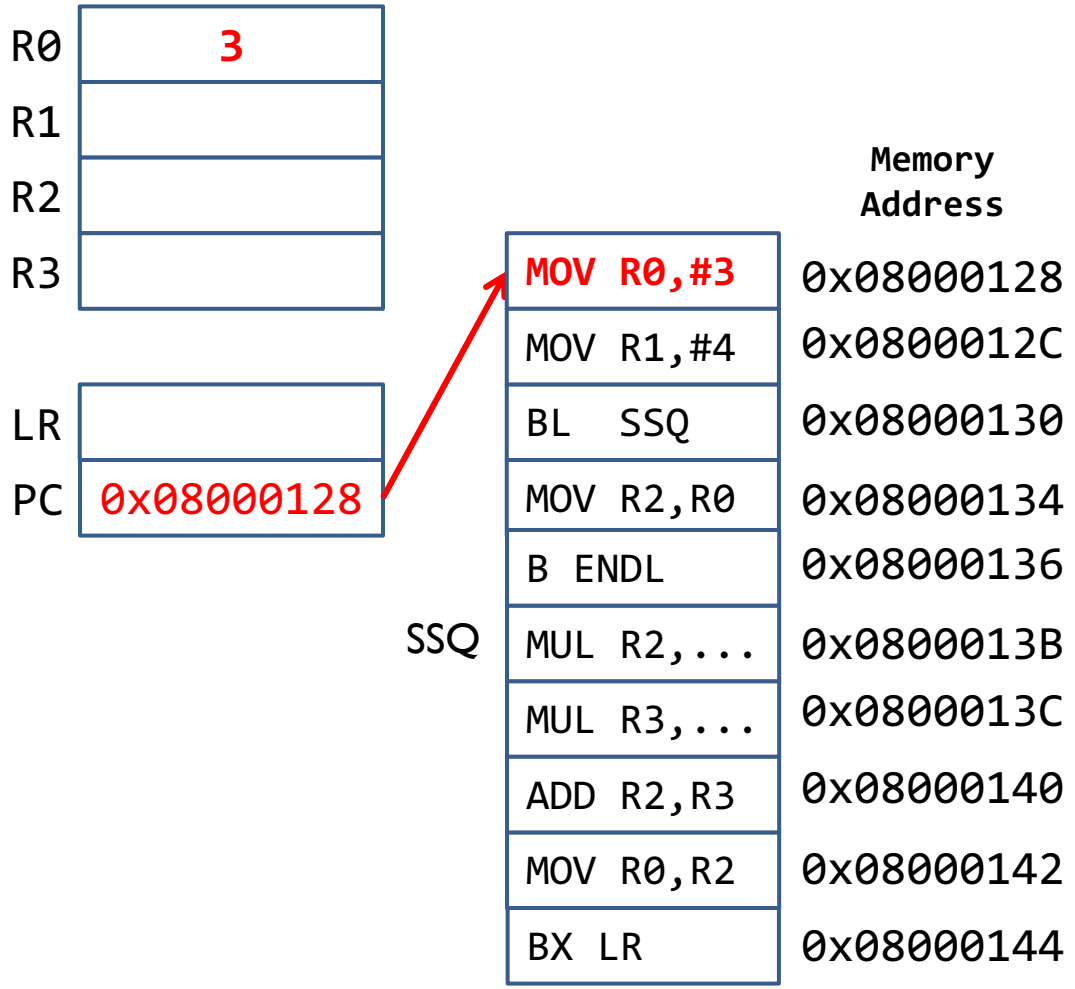
Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



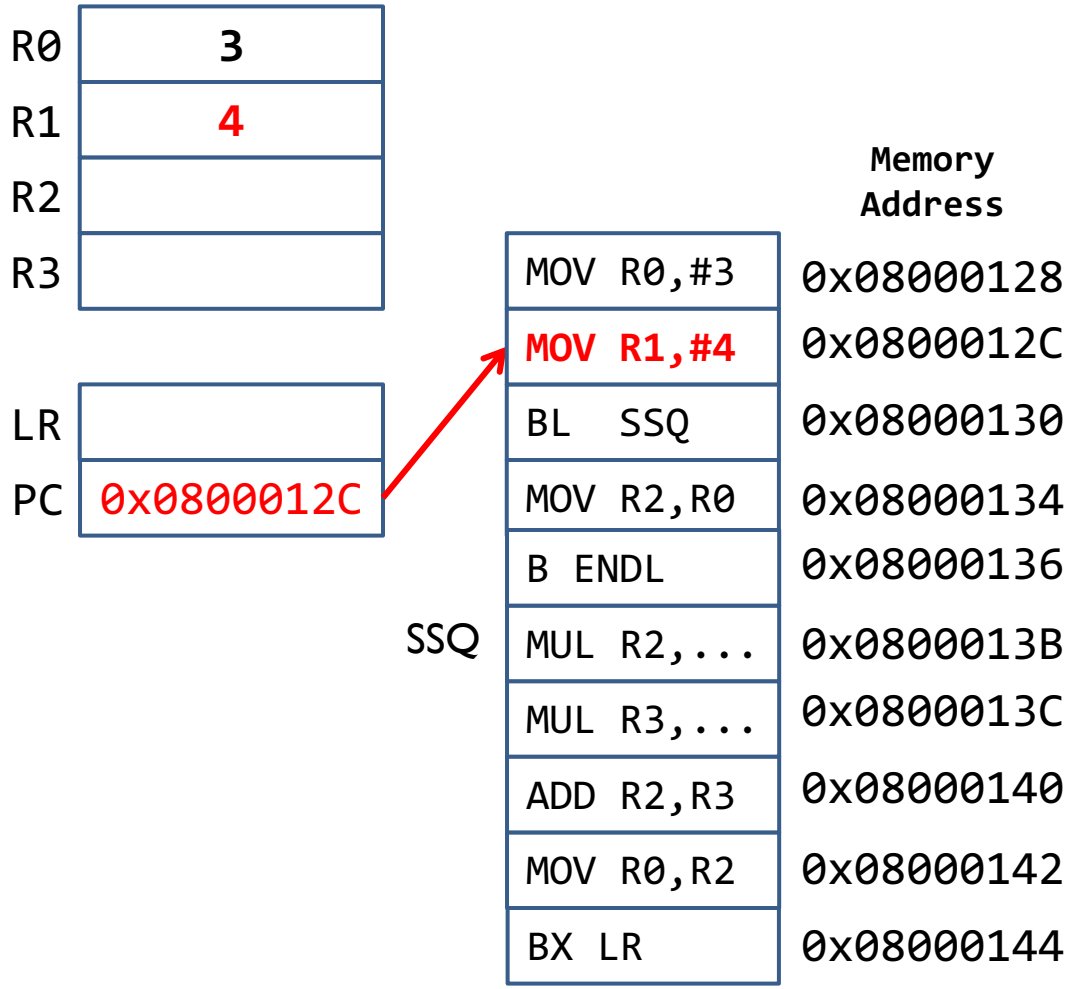
Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



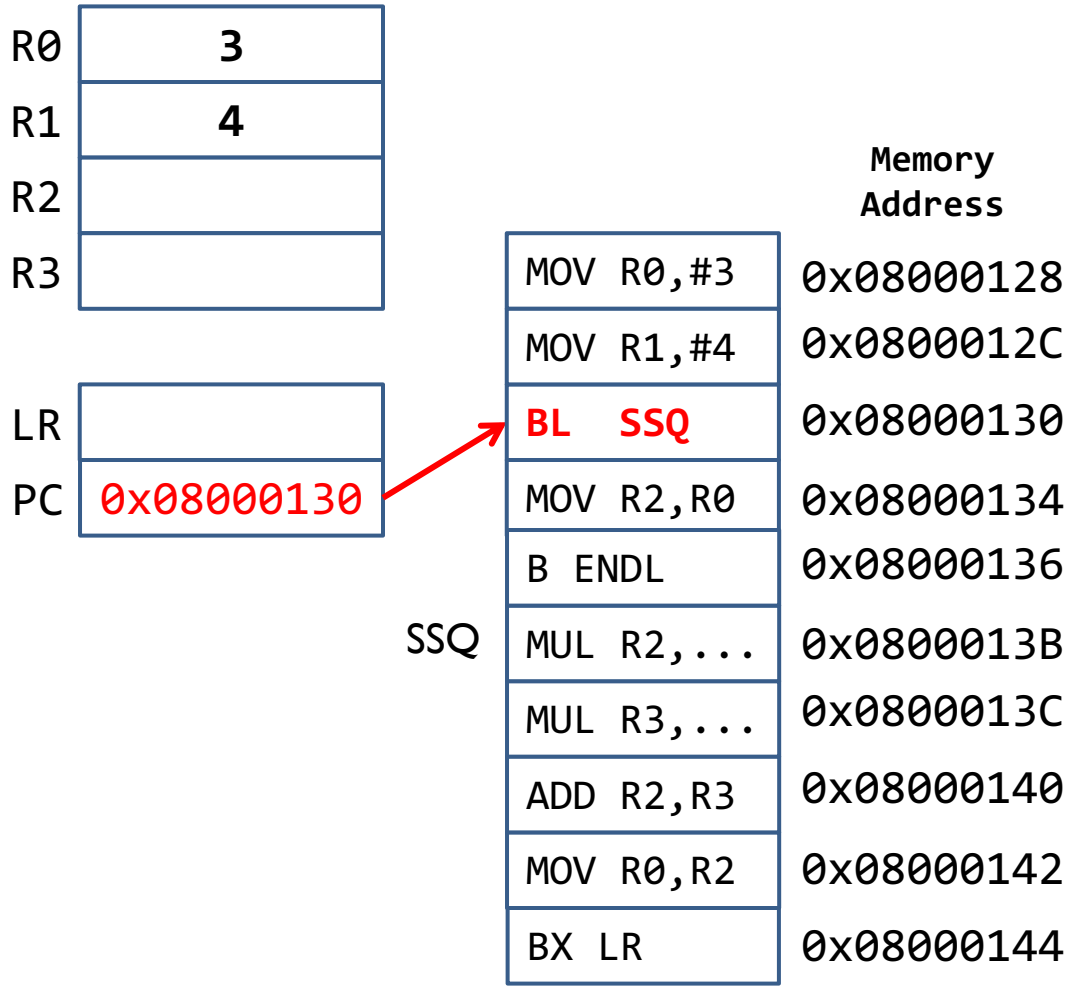
Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



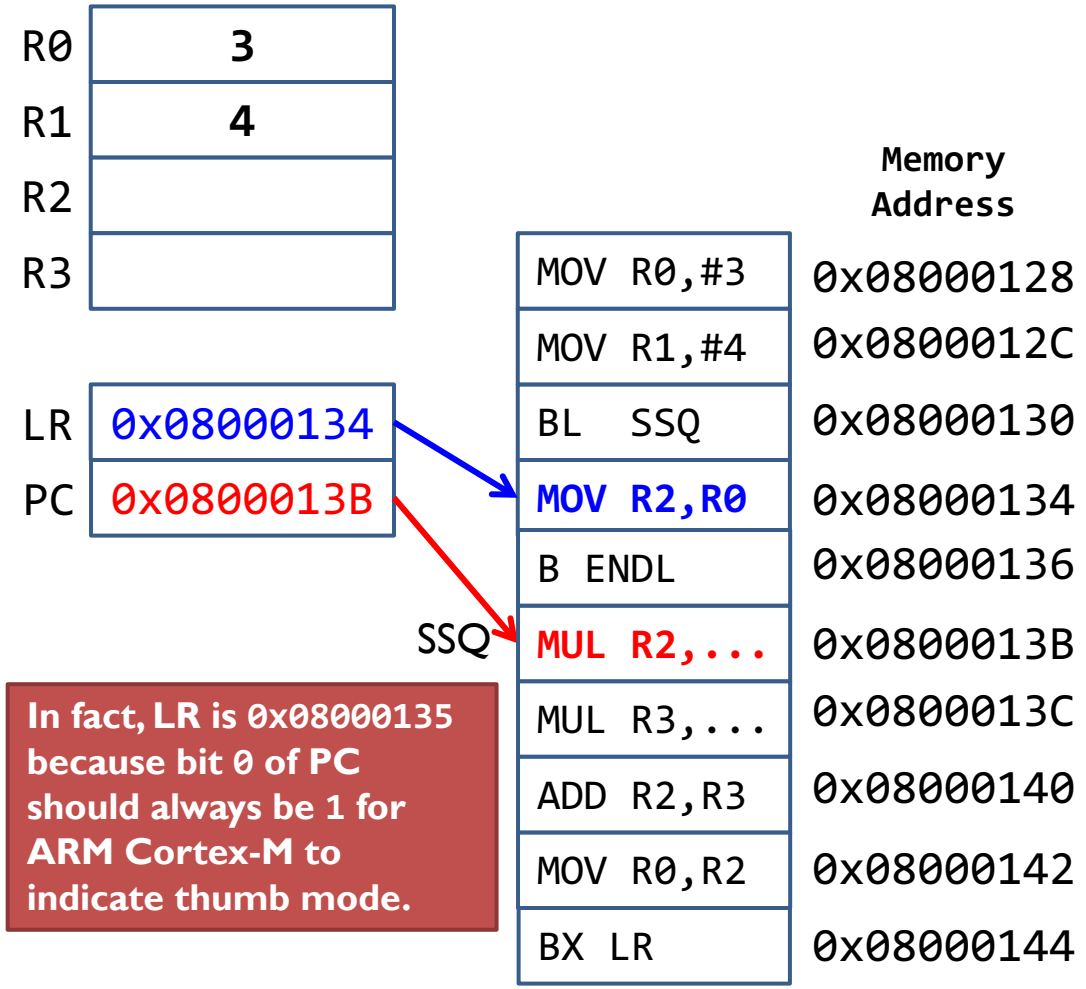
Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```

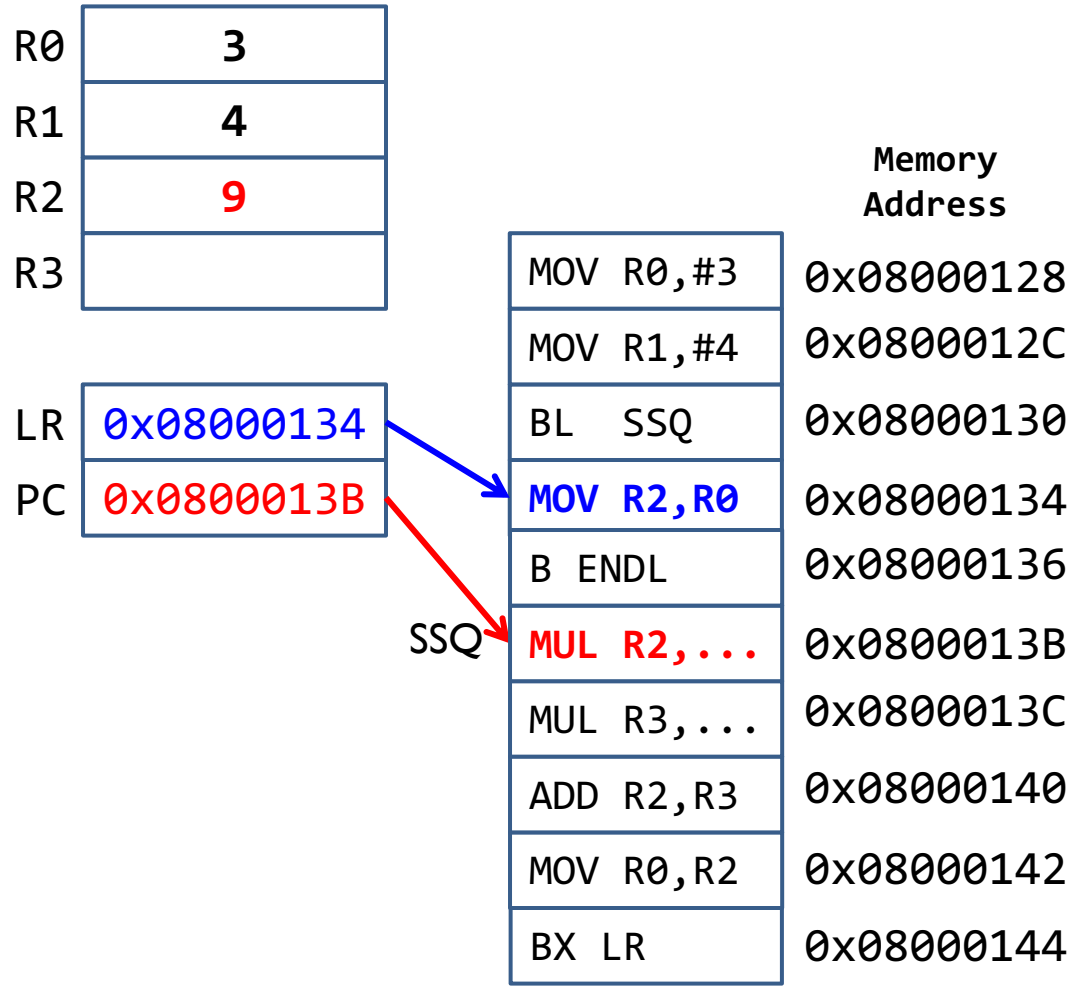


In fact, LR is 0x08000135 because bit 0 of PC should always be 1 for ARM Cortex-M to indicate thumb mode.

Address of the next instruction after the branch is saved into LR.

Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```

R0	3
R1	4
R2	9
R3	16

LR	0x08000134
PC	0x0800013C



	Memory Address
MOV R0,#3	0x08000128
MOV R1,#4	0x0800012C
BL SSQ	0x08000130
MOV R2,R0	0x08000134
B ENDL	0x08000136
MUL R2,...	0x0800013B
MUL R3,...	0x0800013C
ADD R2,R3	0x08000140
MOV R0,R2	0x08000142
BX LR	0x08000144

Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```

R0	3
R1	4
R2	25
R3	16

LR	0x08000134
PC	0x08000140

SSQ

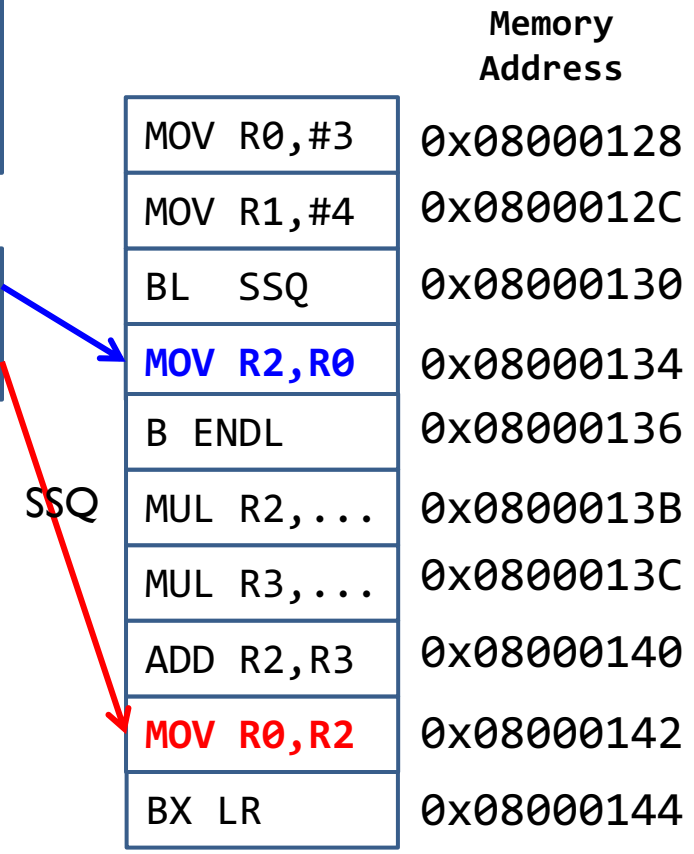
Memory Address	
0x08000128	MOV R0,#3
0x0800012C	MOV R1,#4
0x08000130	BL SSQ
0x08000134	MOV R2,R0
0x08000136	B ENDL
0x0800013B	MUL R2,...
0x0800013C	MUL R3,...
0x08000140	ADD R2,R3
0x08000142	MOV R0,R2
0x08000144	BX LR

Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```

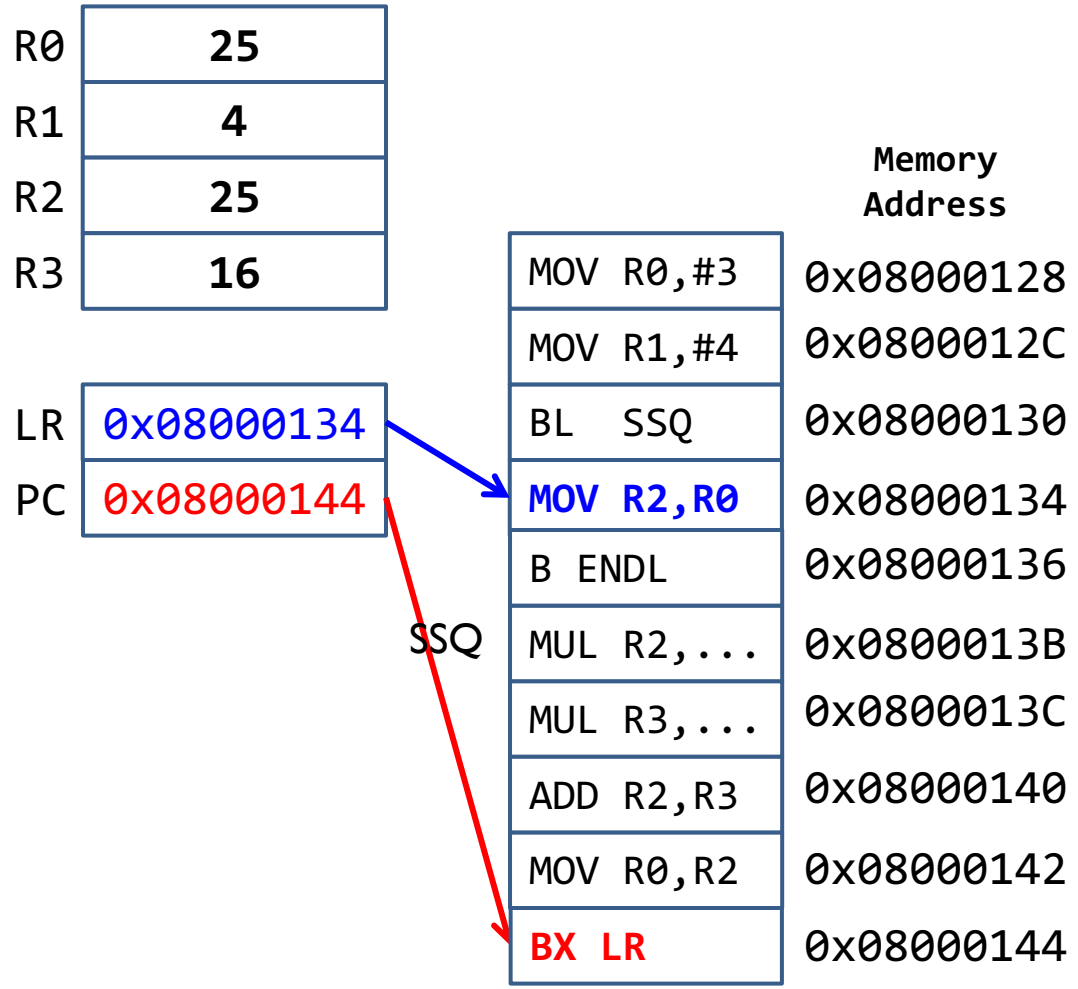
R0	25
R1	4
R2	25
R3	16

LR	0x08000134
PC	0x08000142



Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```

R0	25
R1	4
R2	25
R3	16

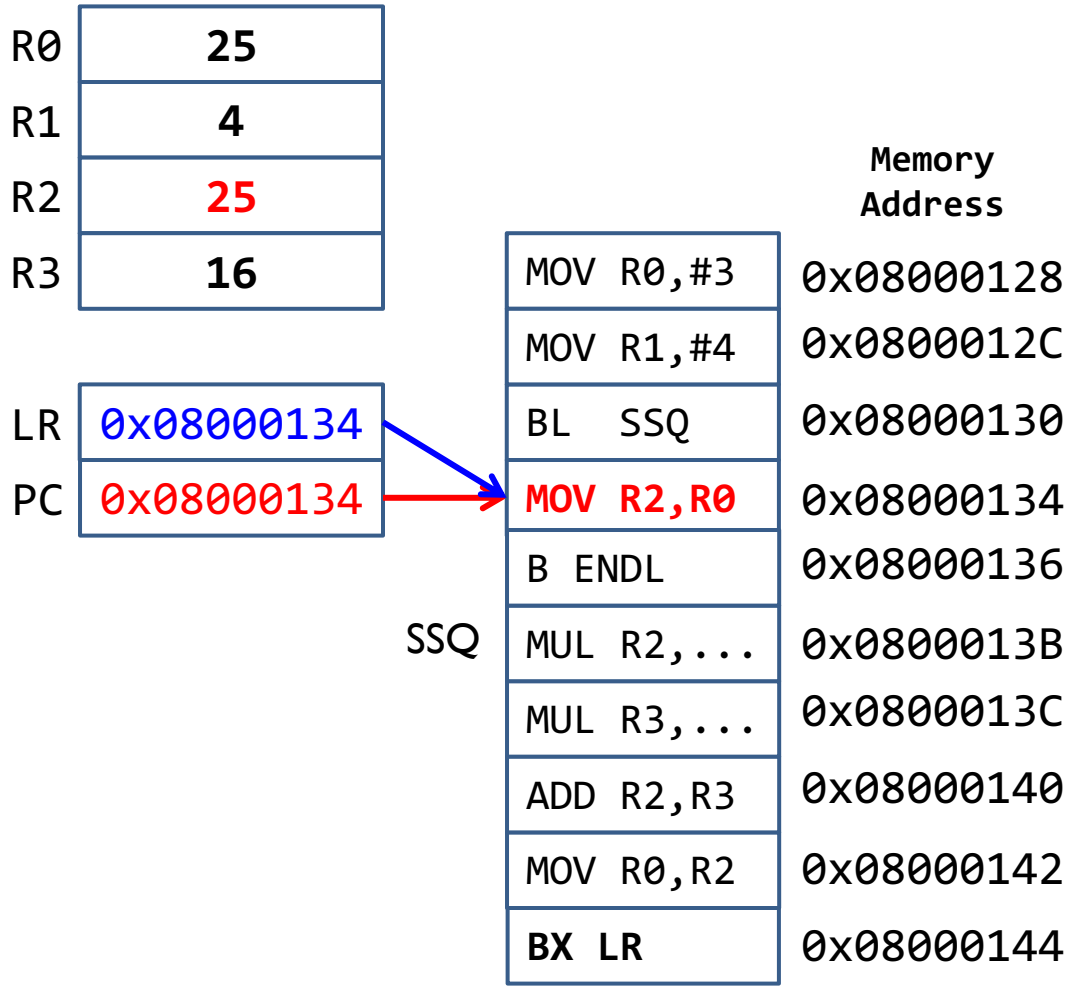
LR	0x08000134
PC	0x08000134

		Memory Address
	MOV R0,#3	0x08000128
	MOV R1,#4	0x0800012C
	BL SSQ	0x08000130
	MOV R2,R0	0x08000134
	B ENDL	0x08000136
SSQ	MUL R2,...	0x0800013B
	MUL R3,...	0x0800013C
	ADD R2,R3	0x08000140
	MOV R0,R2	0x08000142
	BX LR	0x08000144

Copy LR to PC when returning from a subroutine!

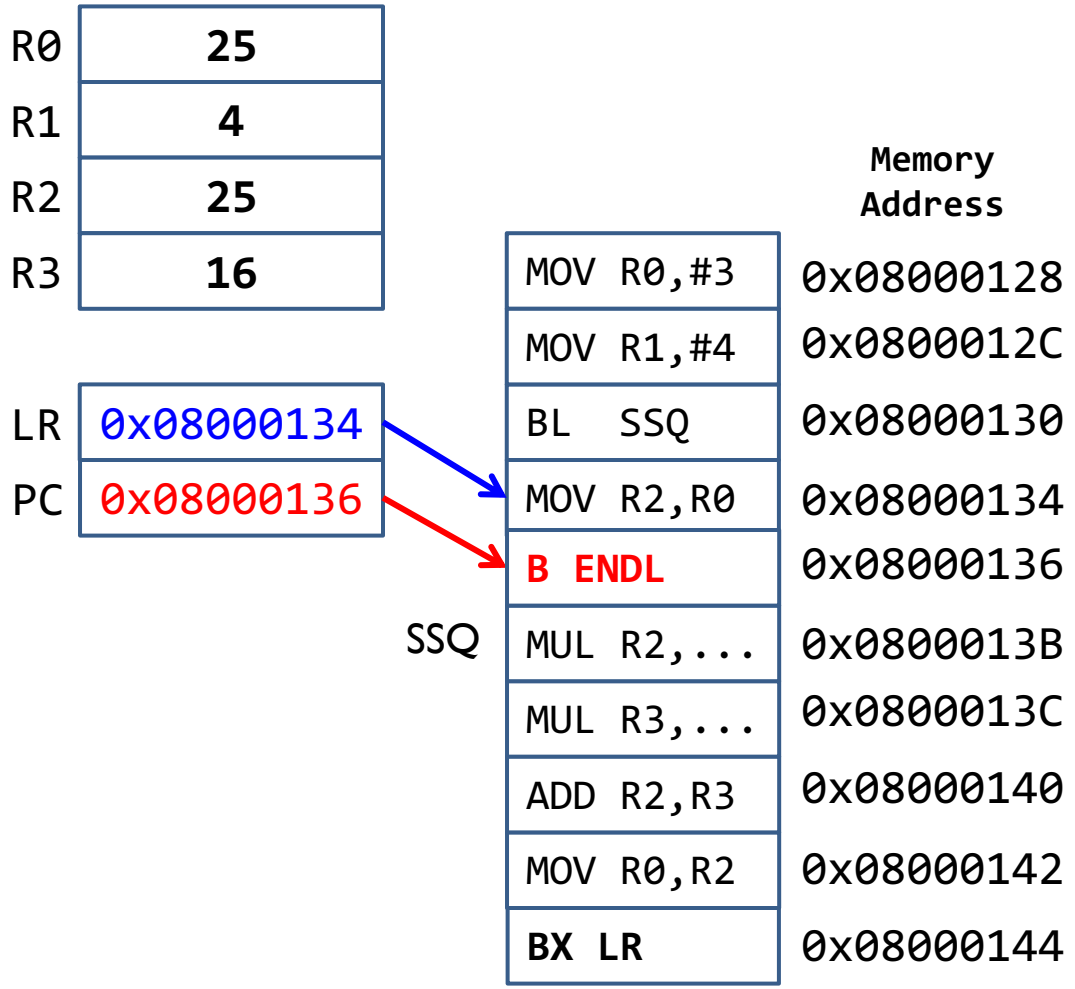
Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



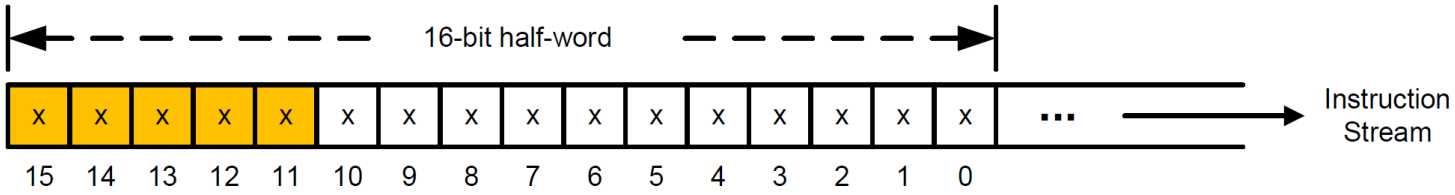
Example: SSQ(3, 4)

```
        MOV R0,#3
        MOV R1,#4
        BL  SSQ
        MOV R2,R0
        B ENDL
        ...
SSQ     PROC
        MUL R2,R0,R0
        MUL R3,R1,R1
        ADD R2,R2,R3
        MOV R0,R2
        BX  LR
        ENDP
ENDL    ...
```



Realities

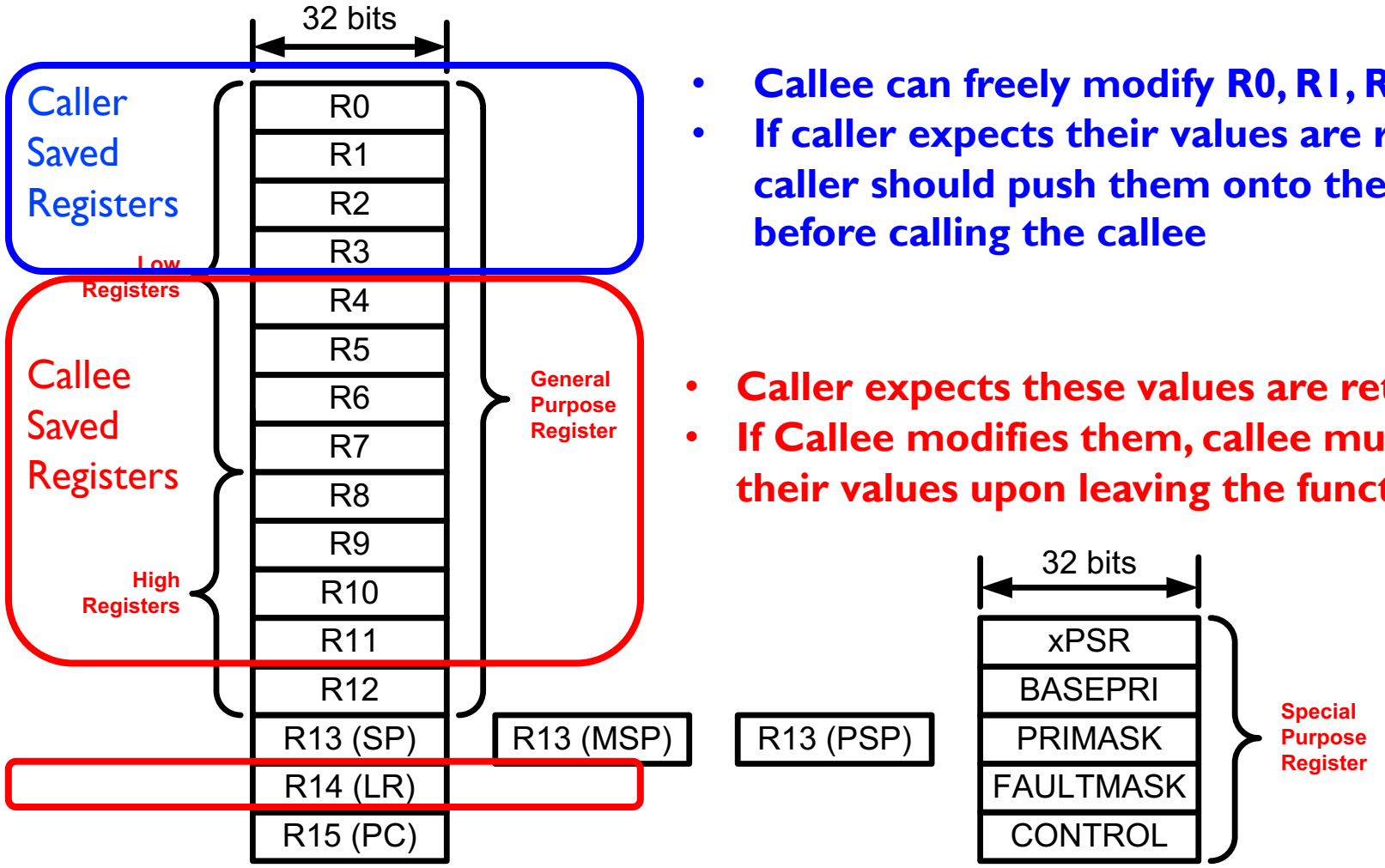
- PC is always incremented by **4**.
  - Each time, 4 bytes are fetched from the instruction memory
  - It is either two 16-bit instructions or one 32-bit instruction



If bit [15-11] = `11101`, `11110`, or `11111`, then, it is the first half-word of a 32-bit instruction. Otherwise, it is a 16-bit instruction.

- The least significant bit of LR is always **1** for ARM Cortex-M
  - This bit is used to control the processor mode:
    - `0` = ARM, 1 = THUMB
  - Cortex-M only supports THUMB.

Callee Saved Registers *vs* Caller Saved Registers



- **Callee can freely modify R0, R1, R2, and R3**
- **If caller expects their values are retained, caller should push them onto the stack before calling the callee**

- **Caller expects these values are retained .**
- **If Callee modifies them, callee must restore their values upon leaving the function.**

ARM Procedure Call Standard

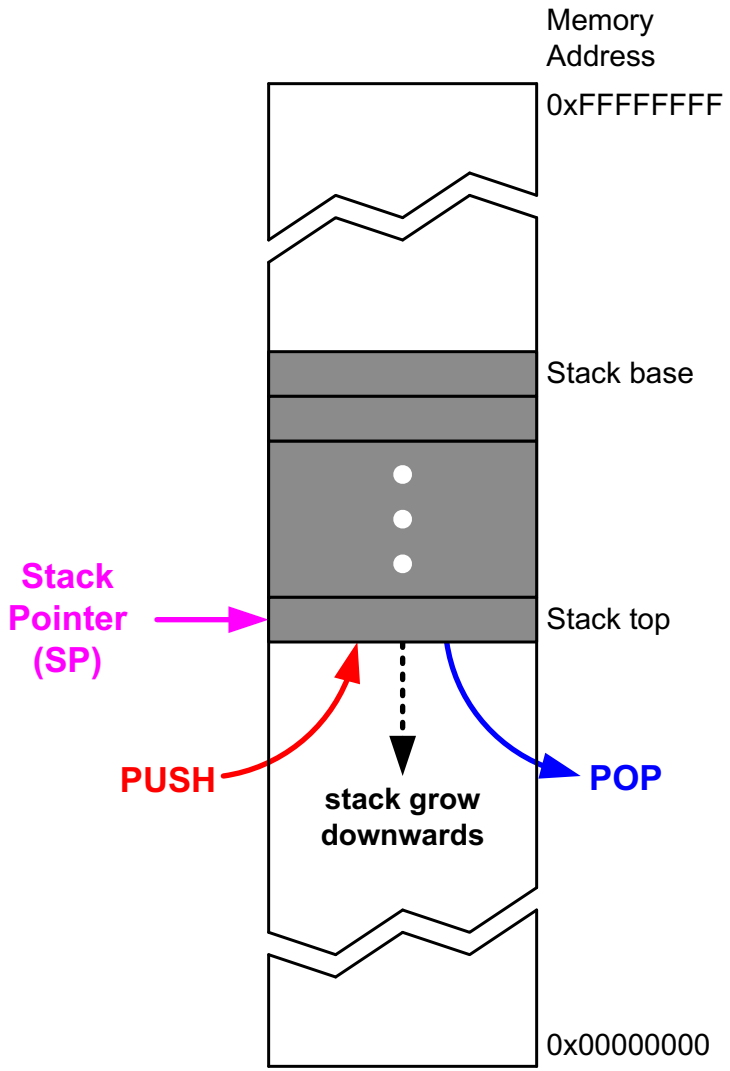
Register	Usage	Subroutine Preserved	Notes
r0	Argument 1 and return value	No	If return has 64 bits, then r0:r1 hold it. If argument 1 has 64 bits, r0:r1 hold it.
r1	Argument 2	No	
r2	Argument 3	No	If the return has 128 bits, r0-r3 hold it.
r3	Argument 4	No	If more than 4 arguments, use the stack
r4	General-purpose V1	Yes	Variable register 1 holds a local variable.
r5	General-purpose V2	Yes	Variable register 2 holds a local variable.
r6	General-purpose V3	Yes	Variable register 3 holds a local variable.
r7	General-purpose V4	Yes	Variable register 4 holds a local variable.
r8	General-purpose V5	Yes	Variable register 5 holds a local variable.
r9	Platform specific/V6	Yes/No	Usage is platform-dependent.
r10	General-purpose V7	Yes	Variable register 7 holds a local variable.
r11	General-purpose V8	Yes	Variable register 8 holds a local variable.
r12 (IP)	Intra-procedure-call register	No	It holds intermediate values between a procedure and the sub-procedure it calls.
r13 (SP)	Stack pointer	Yes	SP has to be the same after a subroutine has completed.
r14 (LR)	Link register	No	LR does not have to contain the same value after a subroutine has completed.
r15 (PC)	Program counter	N/A	Do not directly change PC

Cortex-M Stack

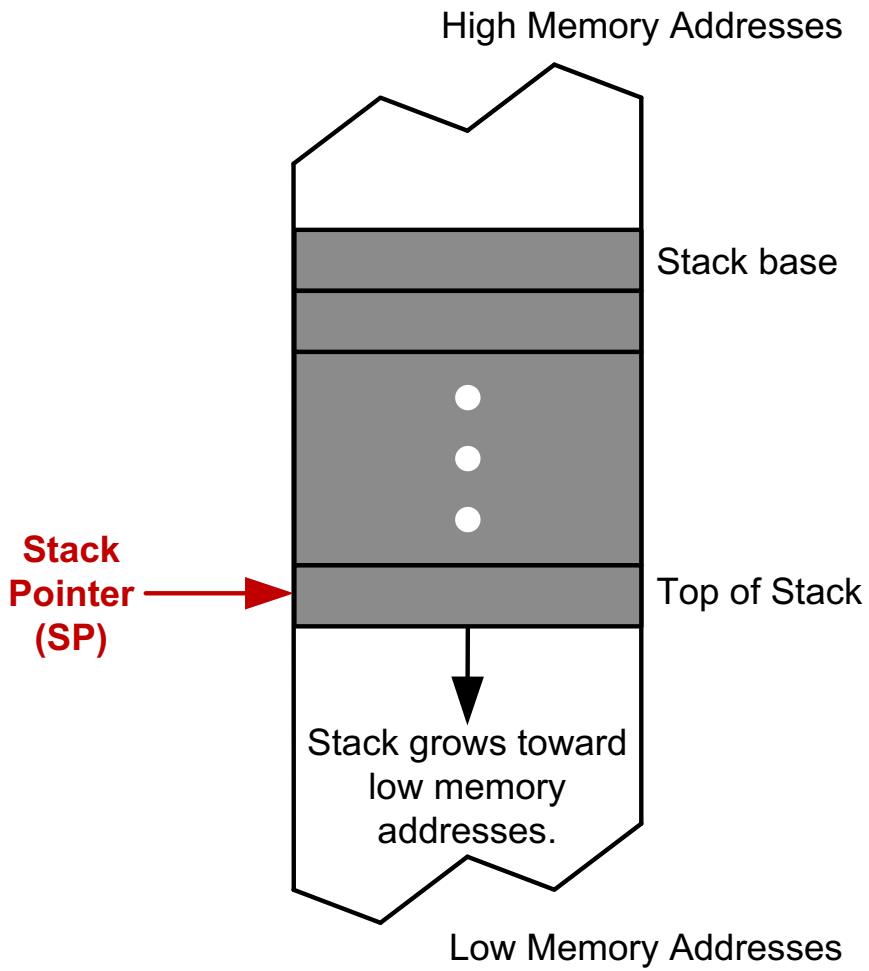
- Cortex-M uses ***full descending stack***
- Example:

```
PUSH/POP {r0,r6,r3}
```

- Stack pointer (SP, aka R13)
  - decremented on **PUSH**
    - **SP = SP – 4 * # of registers**
  - incremented on **POP**
    - **SP = SP + 4 * # of registers**
  - SP starts at **0x20000200** for STM32-Discovery by default (can be changed in startup.s)



Full Descending Stack



PUSH {register_list}
equivalent to:
STMDB SP!, {register_list}

DB: Decrement Before

POP {register_list}
equivalent to:
LDMIA SP!, {register_list}

IA: Increment After

Typical Usage of Stack

- Why need stack?
  - Saving the original contents of processor's registers at the beginning a subroutine (Contents are restored at the end of a subroutine)
  - Storing local variables in a subroutine
  - Passing extra arguments to a subroutine
  - Saving processor's registers upon an interrupt

Stack

PUSH {*Rd*}

- SP = SP-4 ⟶ descending stack
- (*SP) = *Rd* ⟶ full stack

Push multiple registers

```
PUSH {r6, r7, r8}
```

⟺ *They are equivalent.* ⟺

```
PUSH {r8, r7, r6}
```

```
PUSH {r8}
PUSH {r7}
PUSH {r6}
```

- The order in which registers listed in the register list does not matter.
- When pushing multiple registers, these registers are automatically sorted by name and the lowest-numbered register is stored to the lowest memory address, *i.e.* is stored last.

Stack

POP {*Rd*}

- *Rd* = (*SP) ⟶ full stack
- SP = SP + 4 ⟶ Stack shrinks

Pop multiple registers

They are equivalent.

```
POP {r6, r7, r8}   ⟺   POP {r8, r7, r6}   ⟺   POP {r6}
                                               POP {r7}
                                               POP {r8}
```

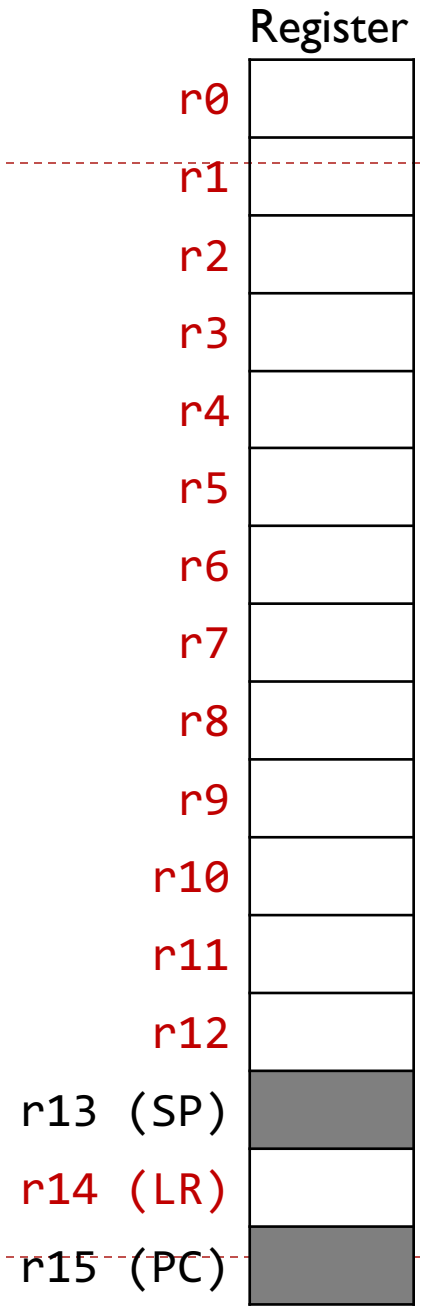
- The order in which registers listed in the register list does not matter.
- When popping multiple registers, these registers are automatically sorted by name and the lowest-numbered register is loaded from the lowest memory address, *i.e.* is loaded first.

Cortex-M Stack

- `PUSH {register list}`
- `POP  {register list}`
  - Eligible registers for PUSH: `r0 - r12, LR`
  - Eligible registers for POP: `r0 - r12, LR, PC`
  - Order specified in the list does not matter
    - Largest-numbered register is always pushed first and popped last
    - Lowest-numbered register stored at lowest memory address

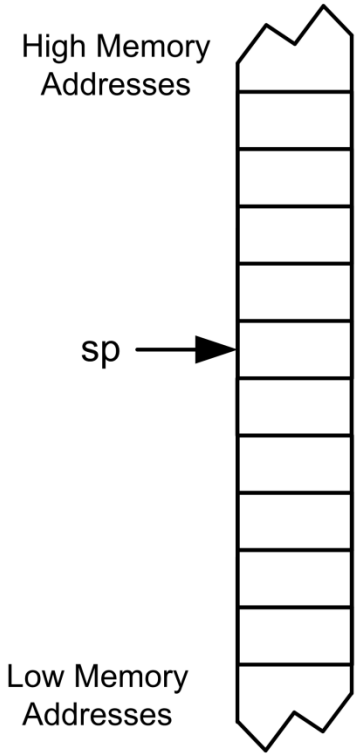
Instruction format for PUSH:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	0	1	0	0	1	0	1	1	0	1	(0)	M	(0)	register_list												



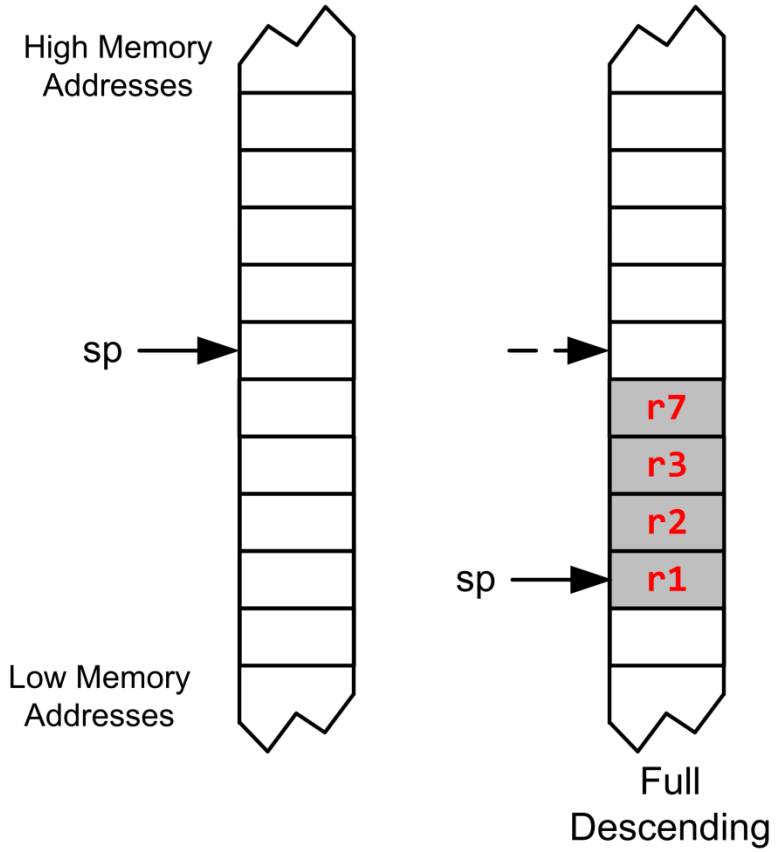
Full Descending Stack

```
PUSH {r3, r1, r7, r2}
```



Full Descending Stack

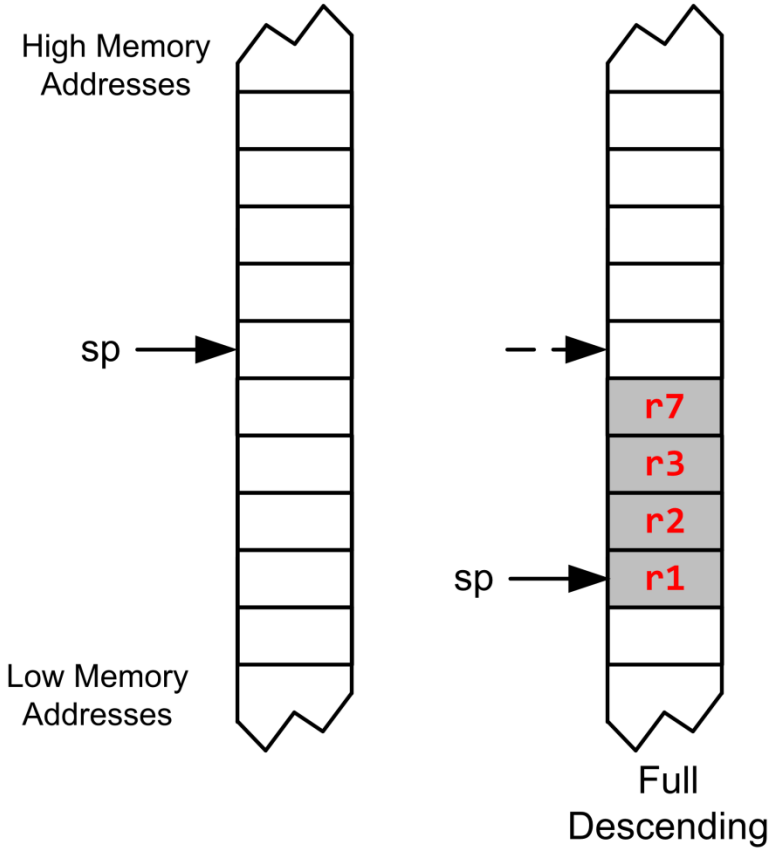
```
PUSH {r3, r1, r7, r2}
```



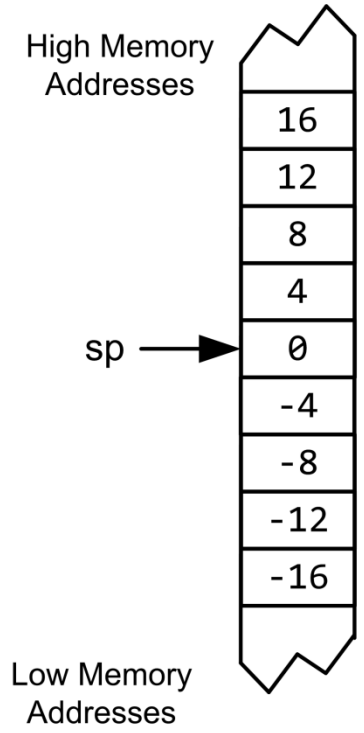
Largest-numbered register is pushed first.

Full Descending Stack

PUSH {r3, r1, r7, r2}



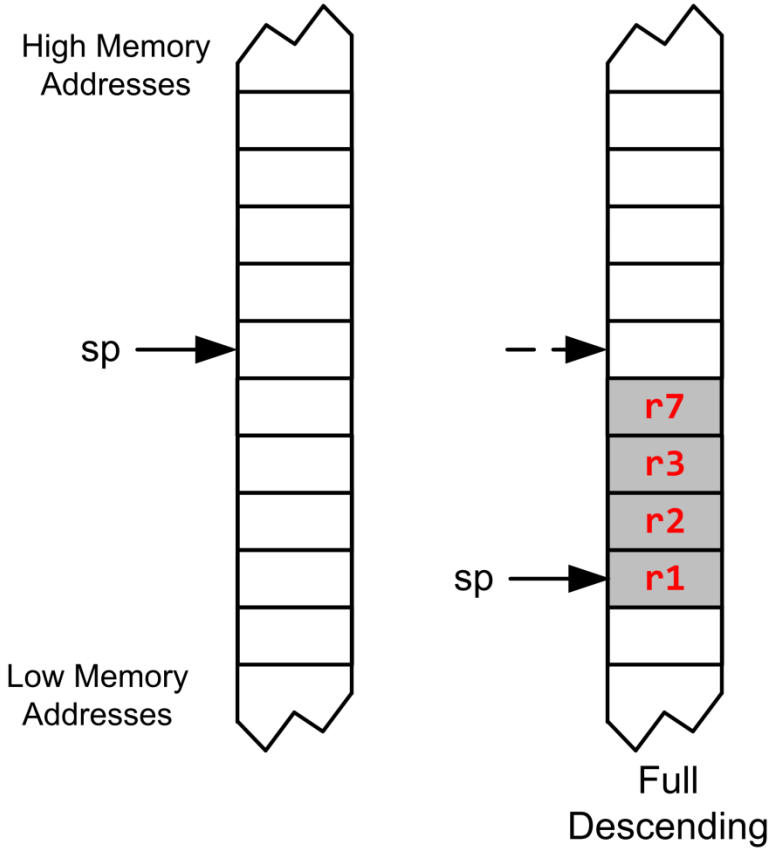
POP {r3, r1, r7, r2}



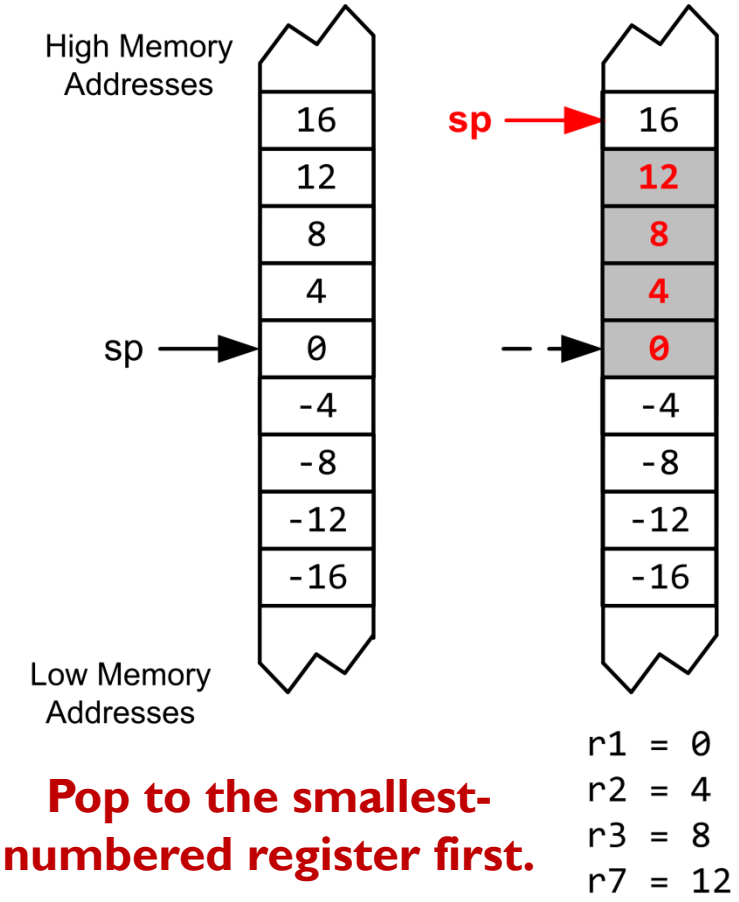
Full Descending Stack

Largest-numbered register is pushed first but popped last.

PUSH {r3, r1, r7, r2}



POP {r3, r1, r7, r2}

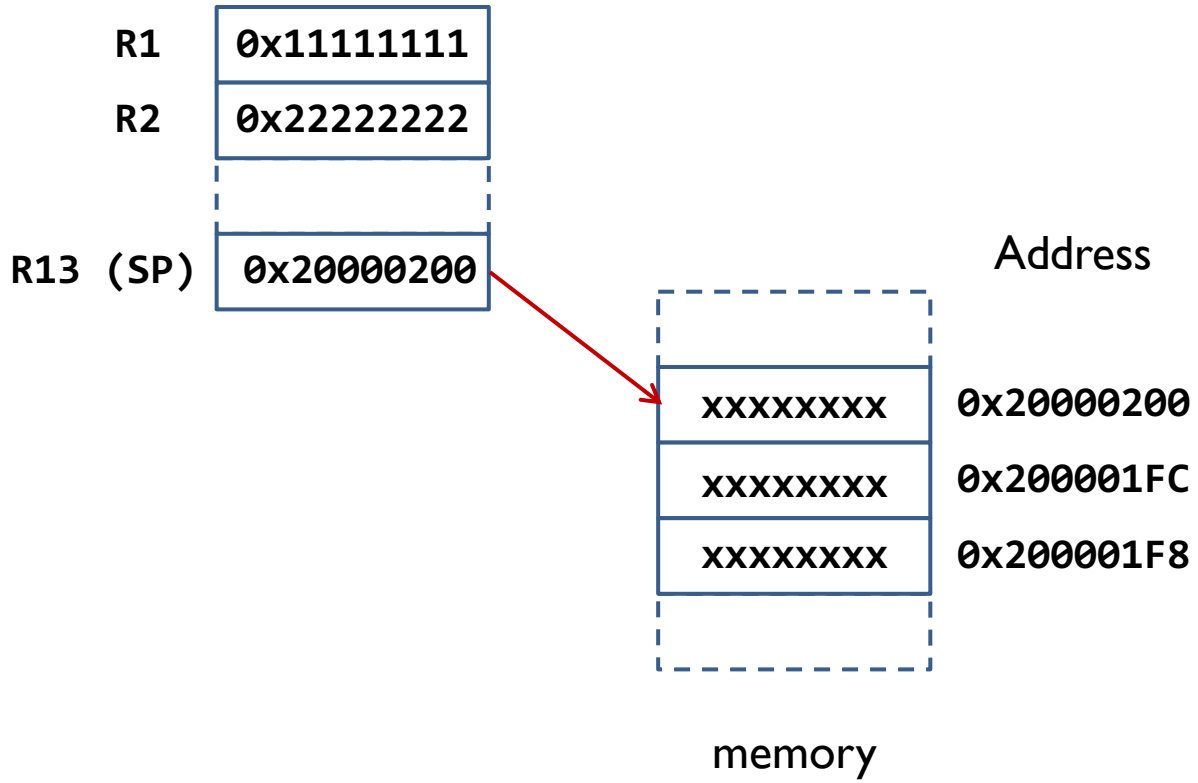


Pop to the smallest-numbered register first.

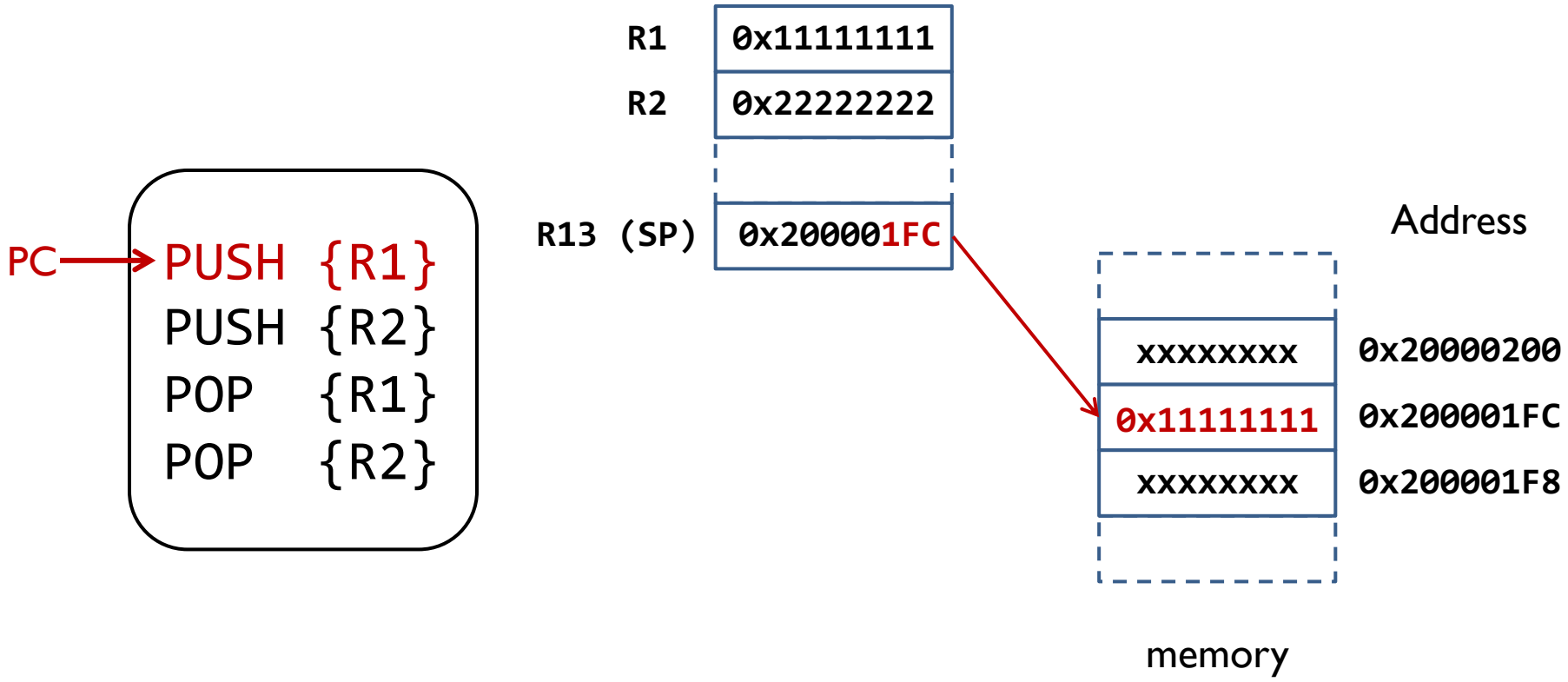
```
r1 = 0
r2 = 4
r3 = 8
r7 = 12
```

Example: swap R1 & R2

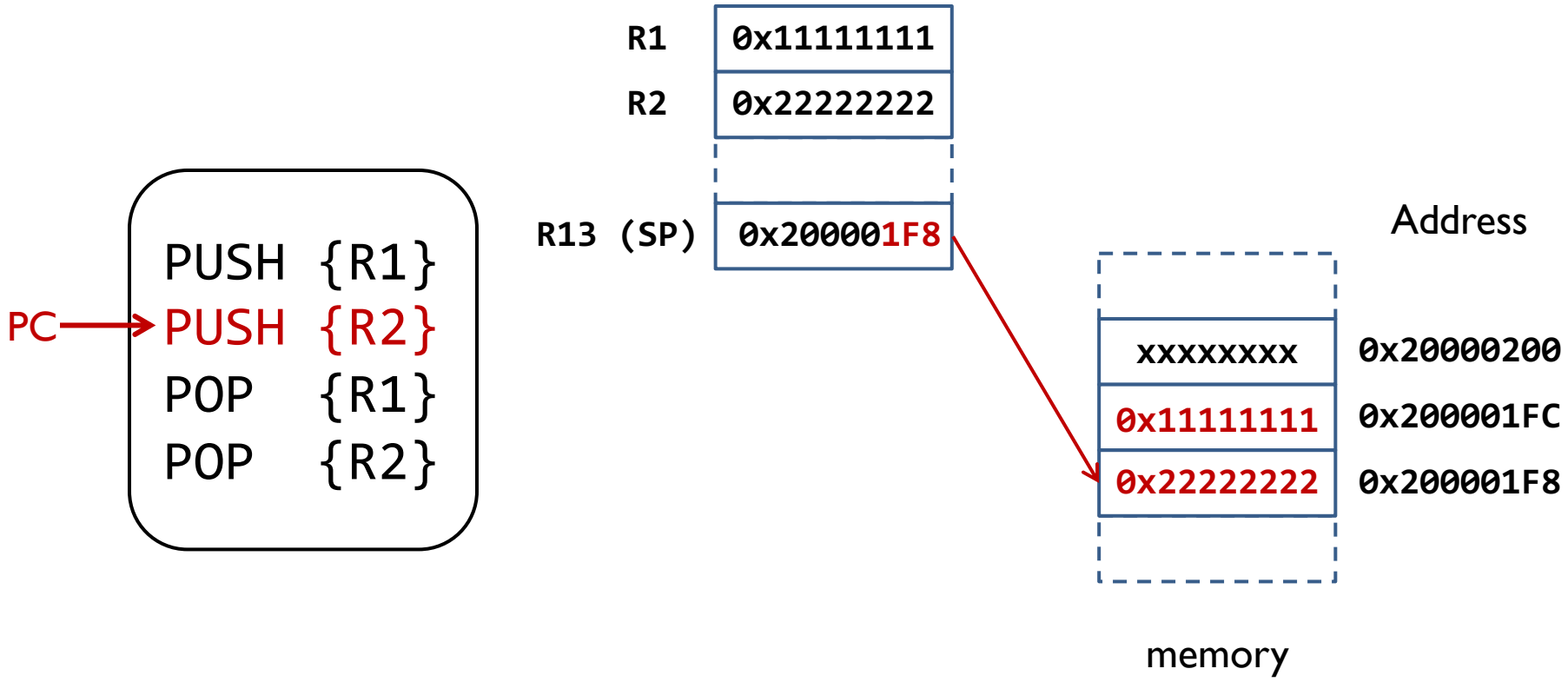
```
PUSH {R1}
PUSH {R2}
POP  {R1}
POP  {R2}
```



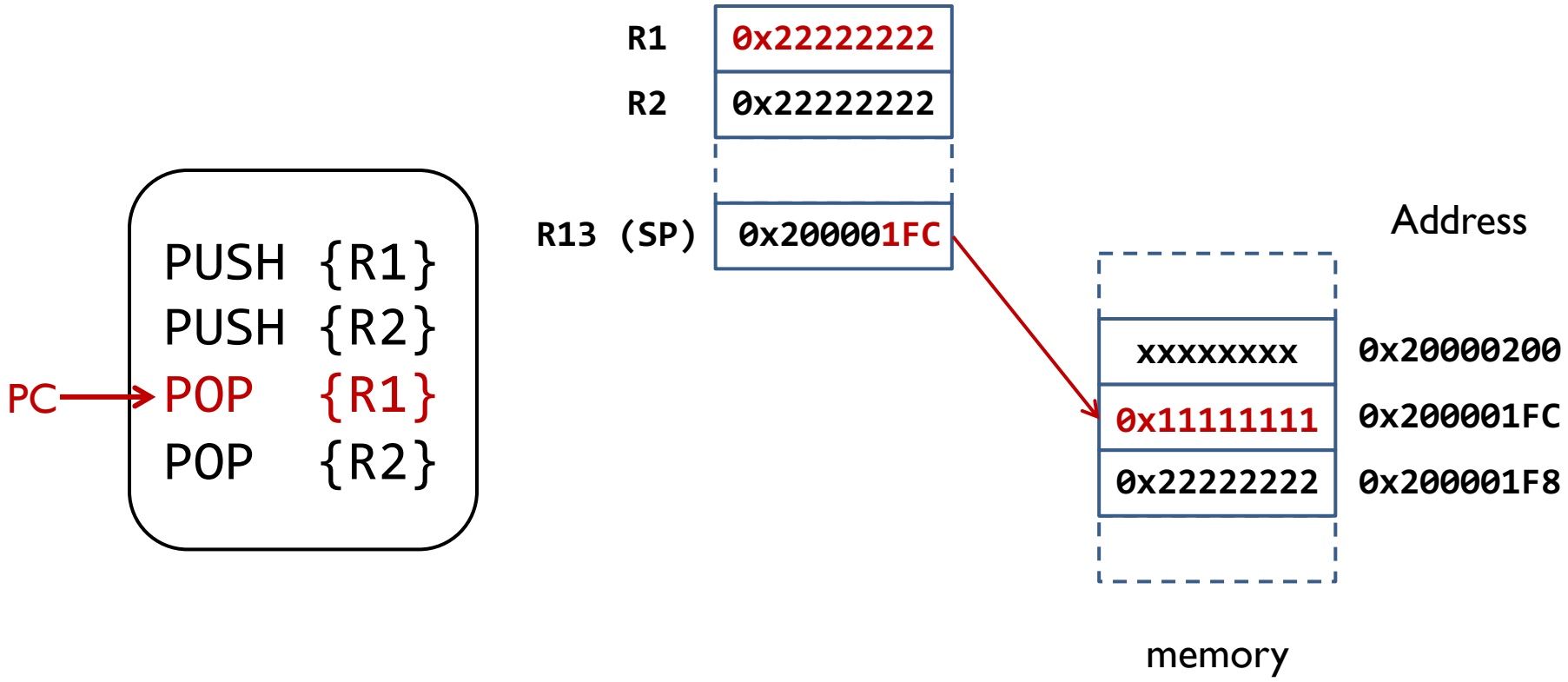
Example: swap R1 & R2



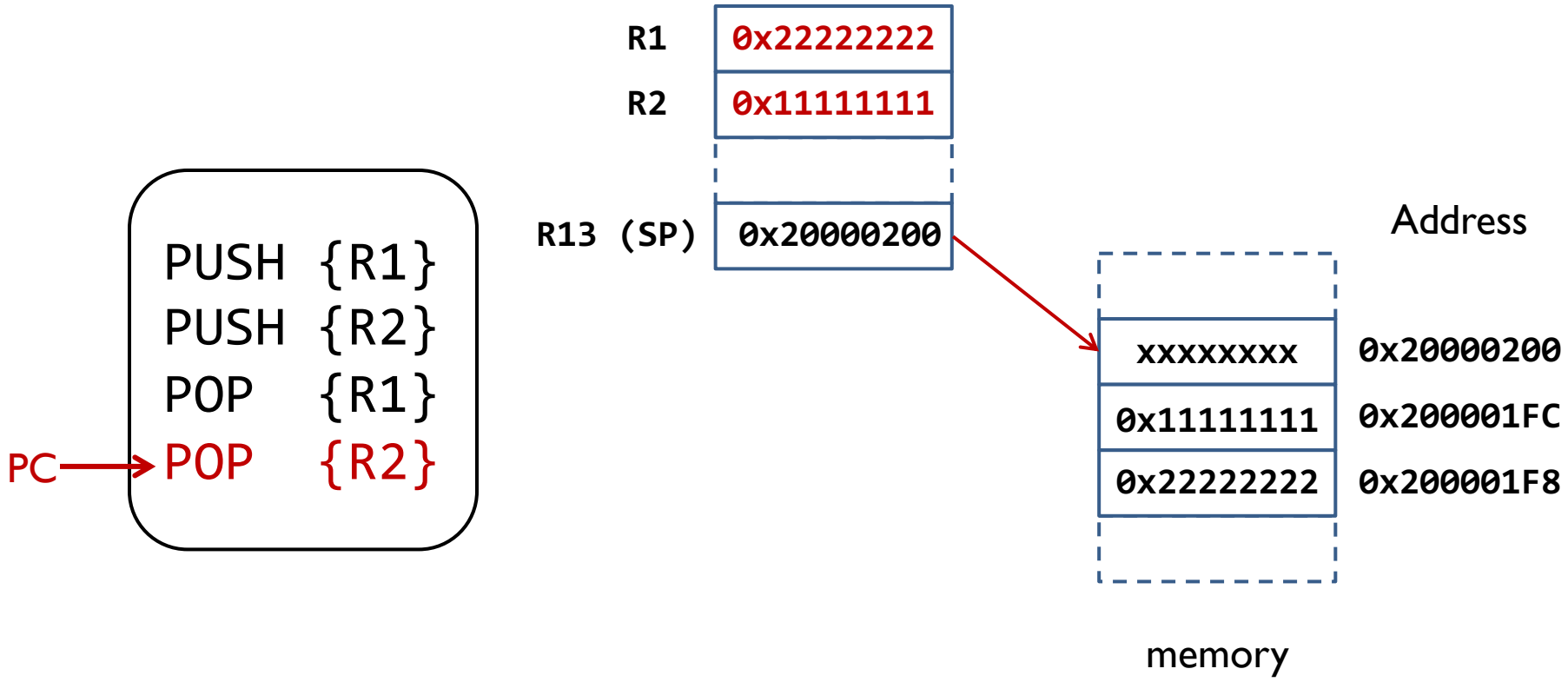
Example: swap R1 & R2



Example: swap R1 & R2



Example: swap R1 & R2



Call a Subroutine

Caller Program	Subroutine/Callee
MOV r4, #100 ... **BL foo** ... ADD r4, r4, #1 ; r4 = 101, not 11	foo PROC ... MOV r4, #10 ; foo changes r4 ... **BX LR** ENDP

Preserve Runtime Environment via Stack

Caller Program	Subroutine/Callee
MOV r4, #100 ... BL foo ... ADD r4, r4, #1 ; r4 = 101, not 11	foo PROC PUSH {r4} ; preserve r4 ... MOV r4, #10 ; foo changes r4 ... POP {r4} ; Recover r4 BX LR ENDP

Caller expects callee does not modify r4!

Callee should preserve r4!

Preserve Runtime Environment via Stack

Caller Program	Subroutine/Callee
Caller should save these registers if callers needs to re-use their original values: • R0 – R3 • R12 • CPSR	Callee should Preserve • R4 – R11 • R14 (LR) • R13 (SP)

Preserve Runtime Environment via Stack

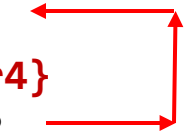
What is wrong in foo()?

Caller Program	Subroutine foo	Subroutine bar
MOV r4, #100 ... BL foo ... ADD r4, r4, #1	foo PROC PUSH {r4} ... MOV r4, #10 ... BL bar ... POP {r4} BX LR ENDP	bar PROC ... BX LR ENDP

Preserve Runtime Environment via Stack

What is wrong in foo()?

Caller Program	Subroutine foo	Subroutine bar
MOV r4, #100 ... BL foo ... ADD r4, r4, #1	foo PROC PUSH {r4} ... MOV r4, #10 ... BL bar ... POP {r4} BX LR ENDP	bar PROC ... BX LR ENDP

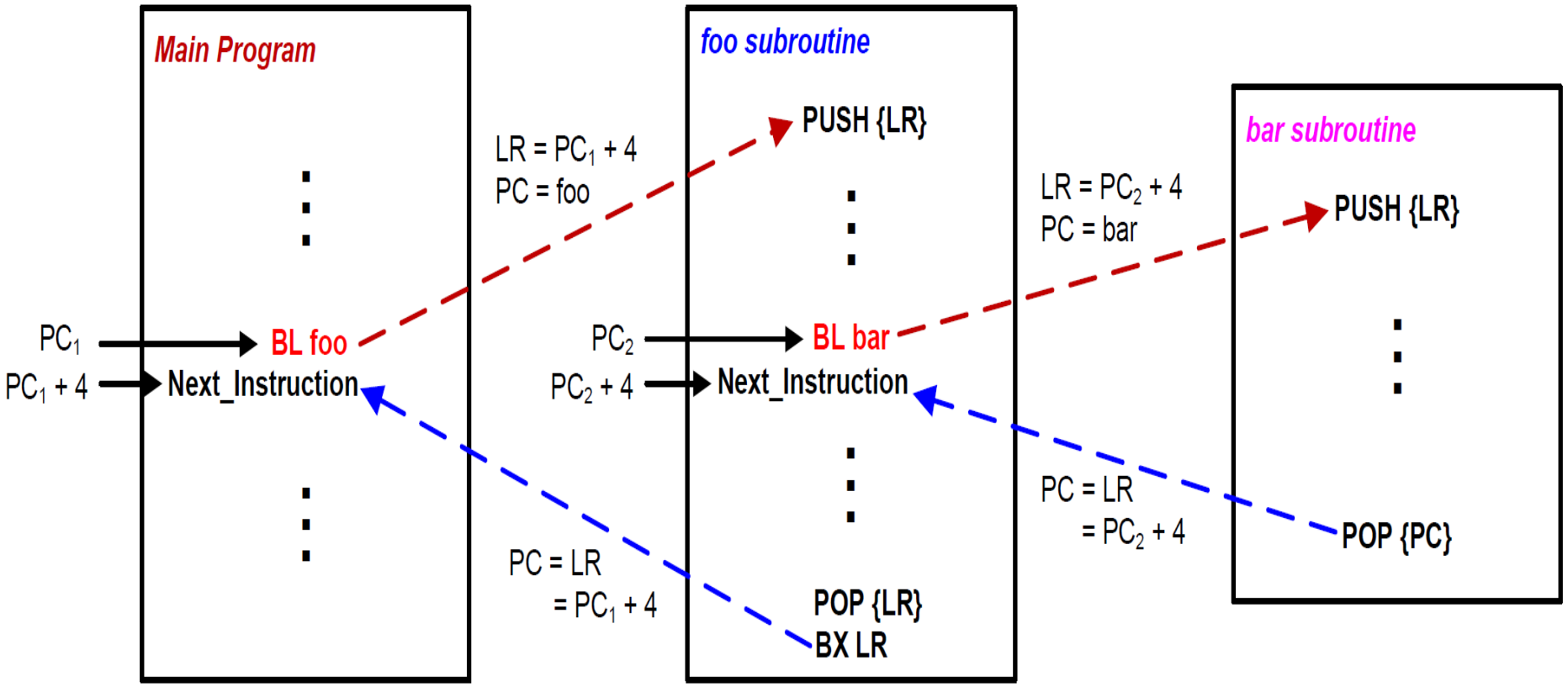


Preserve Runtime Environment via Stack

What is wrong in foo()? Solution #1

Caller Program	Subroutine foo	Subroutine bar
MOV r4, #100 ... BL foo ... ADD r4, r4, #1	foo PROC PUSH {r4, LR} ... MOV r4, #10 ... BL bar ... POP {r4, LR} BX LR ENDP	bar PROC ... BX LR ENDP

Stacks and Subroutines

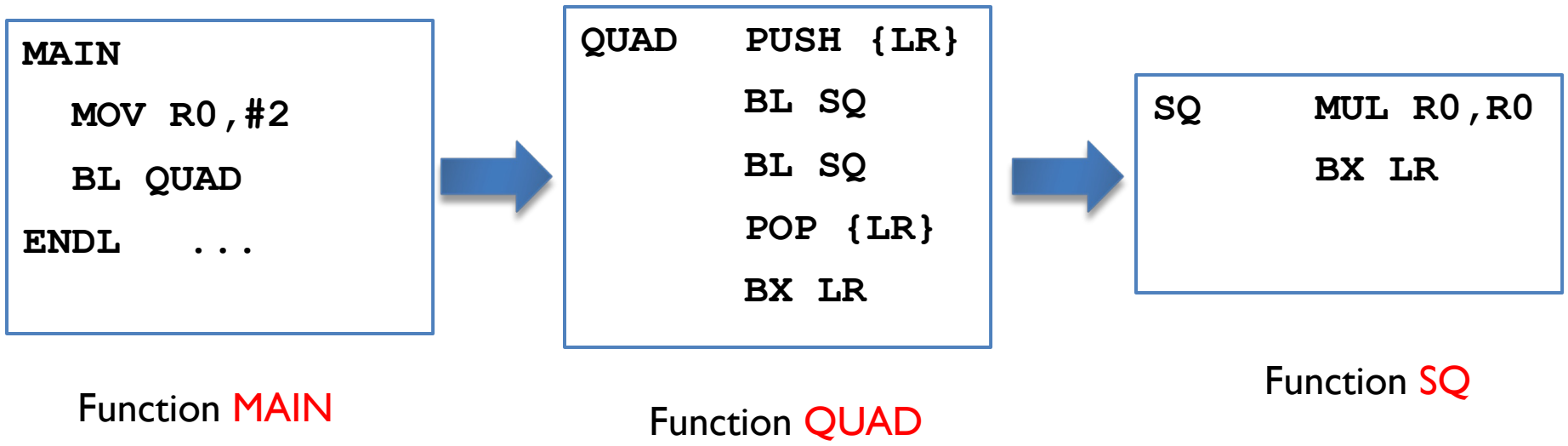


Preserve Runtime Environment via Stack

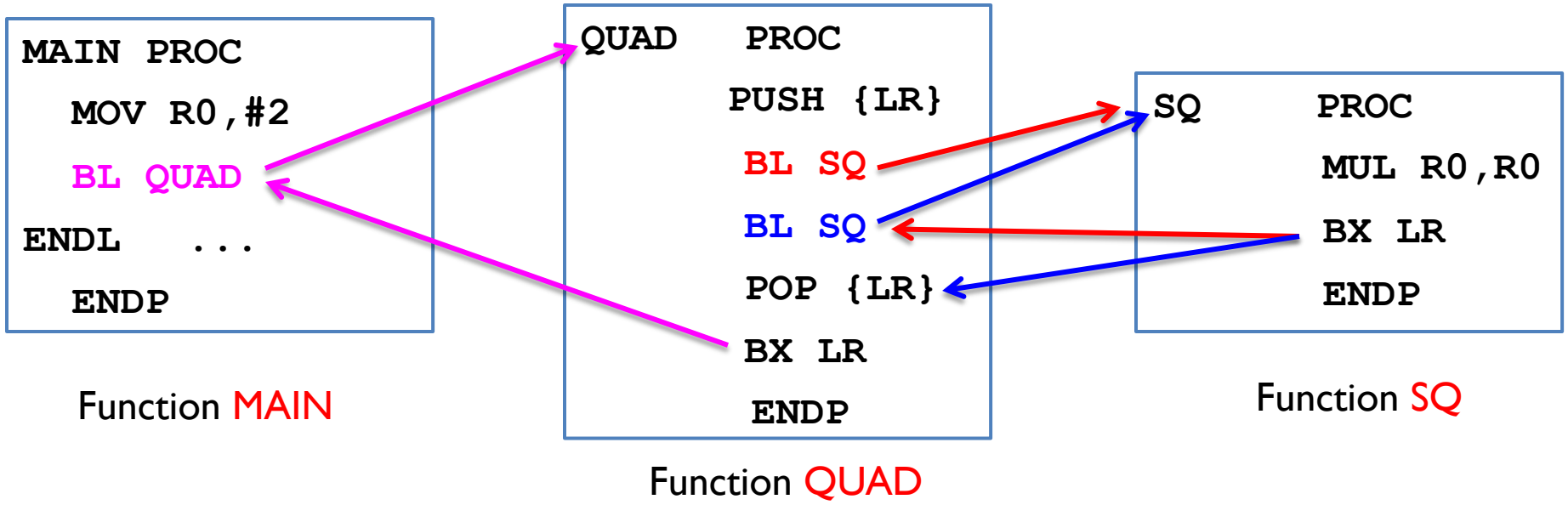
What is wrong in foo()? Solution #2

Caller Program	Subroutine foo	Subroutine bar
MOV r4, #100 ... BL foo ... ADD r4, r4, #1	foo PROC PUSH {r4, LR} ... MOV r4, #10 ... BL bar ... POP {r4, PC} ~~BX LR~~ ENDP	bar PROC ... BX LR ENDP

Subroutine Calling Another Subroutine



Subroutine Calling Another Subroutine

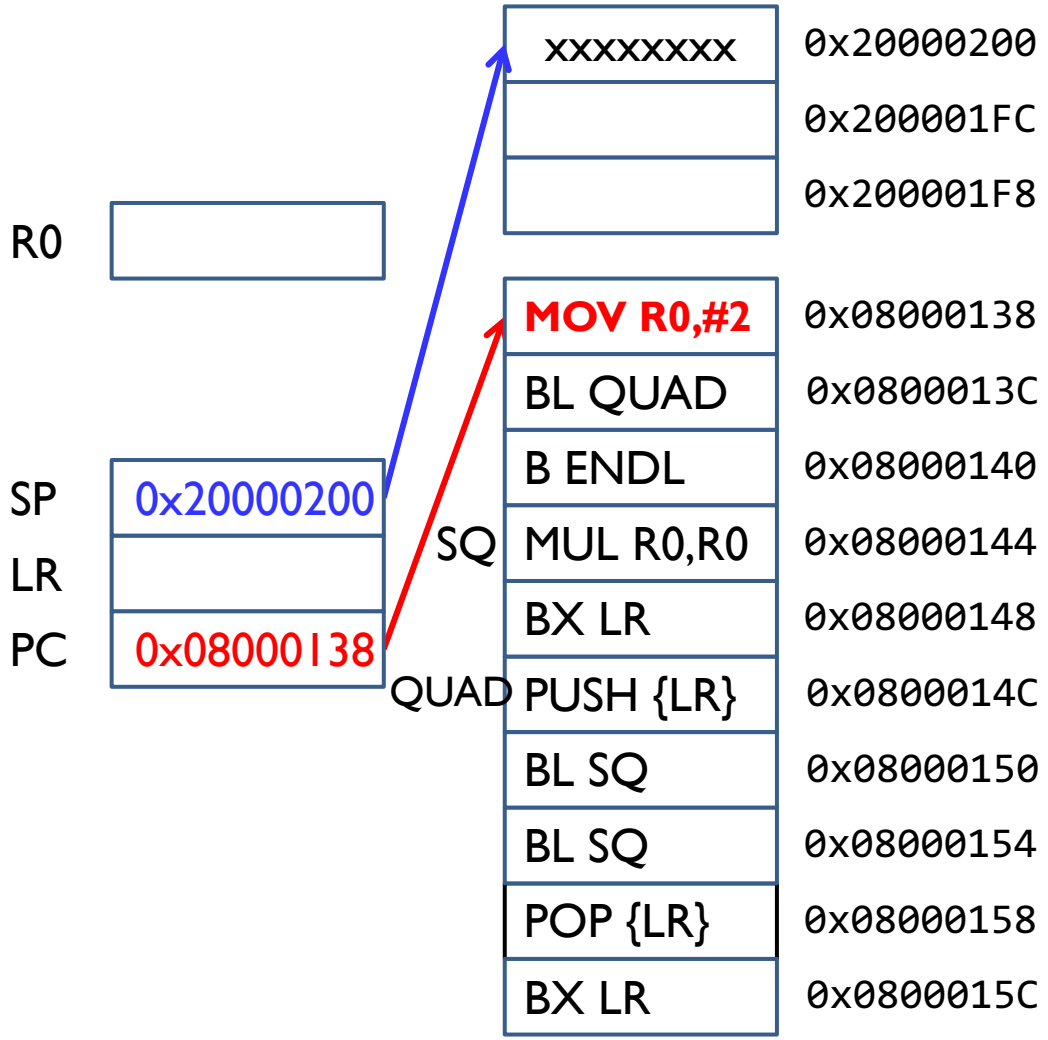


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

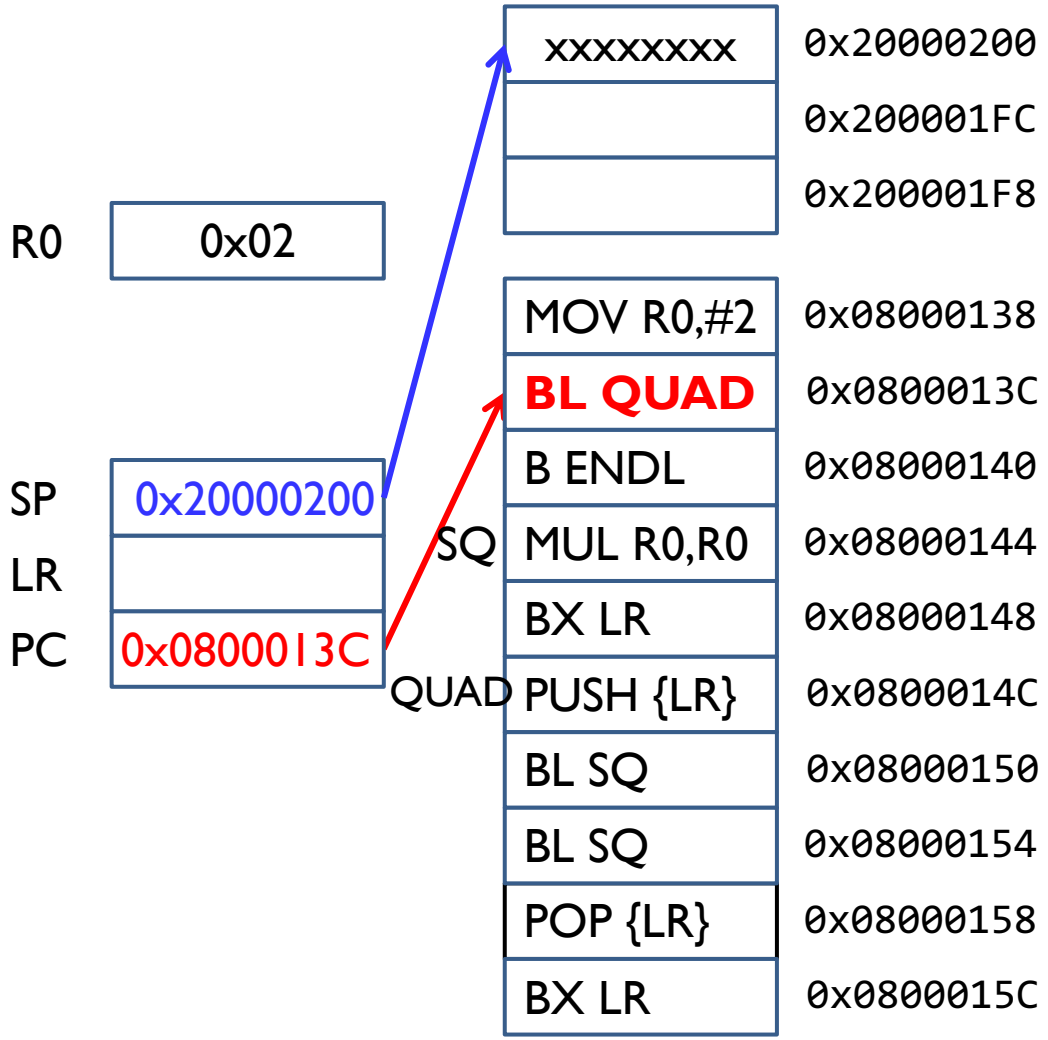


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

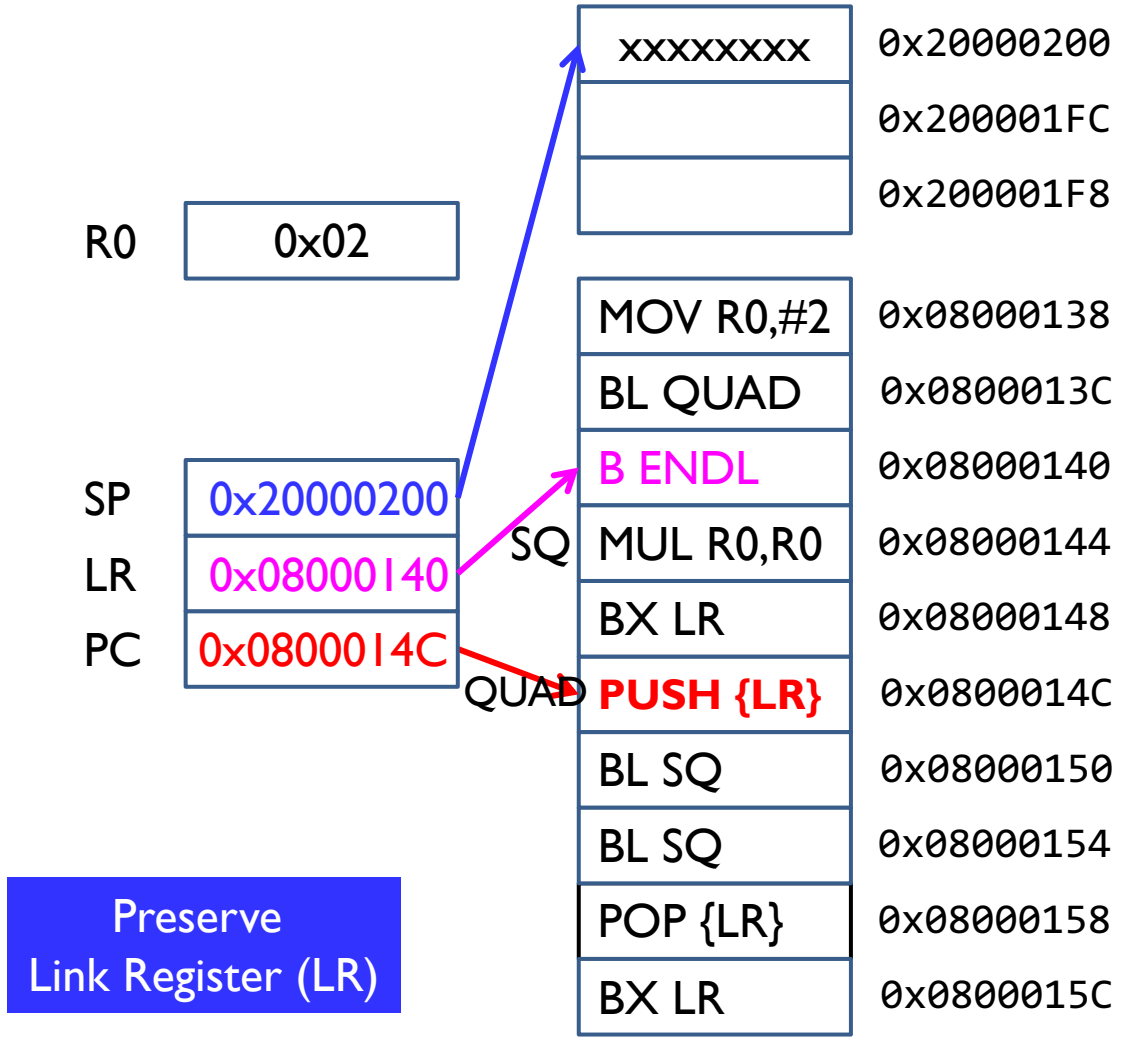


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

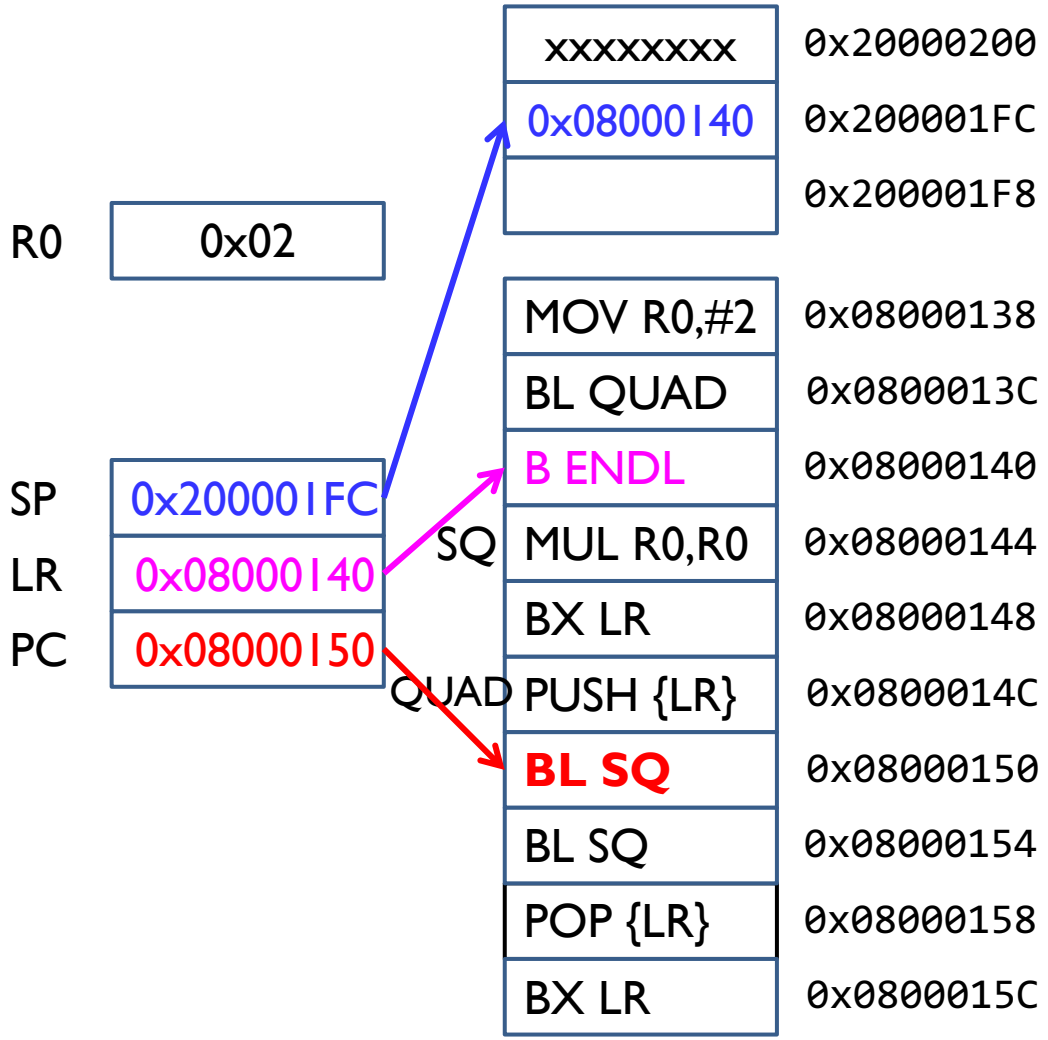


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

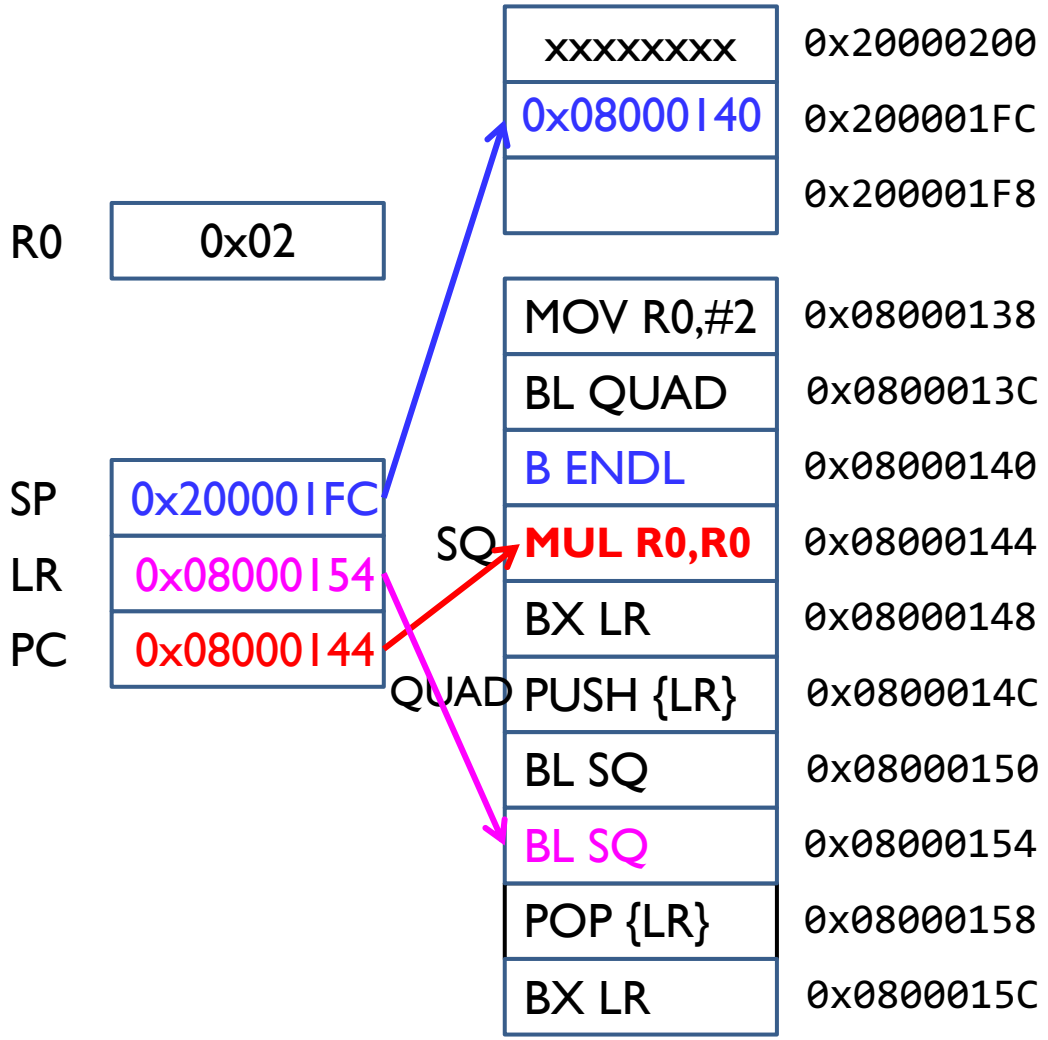


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

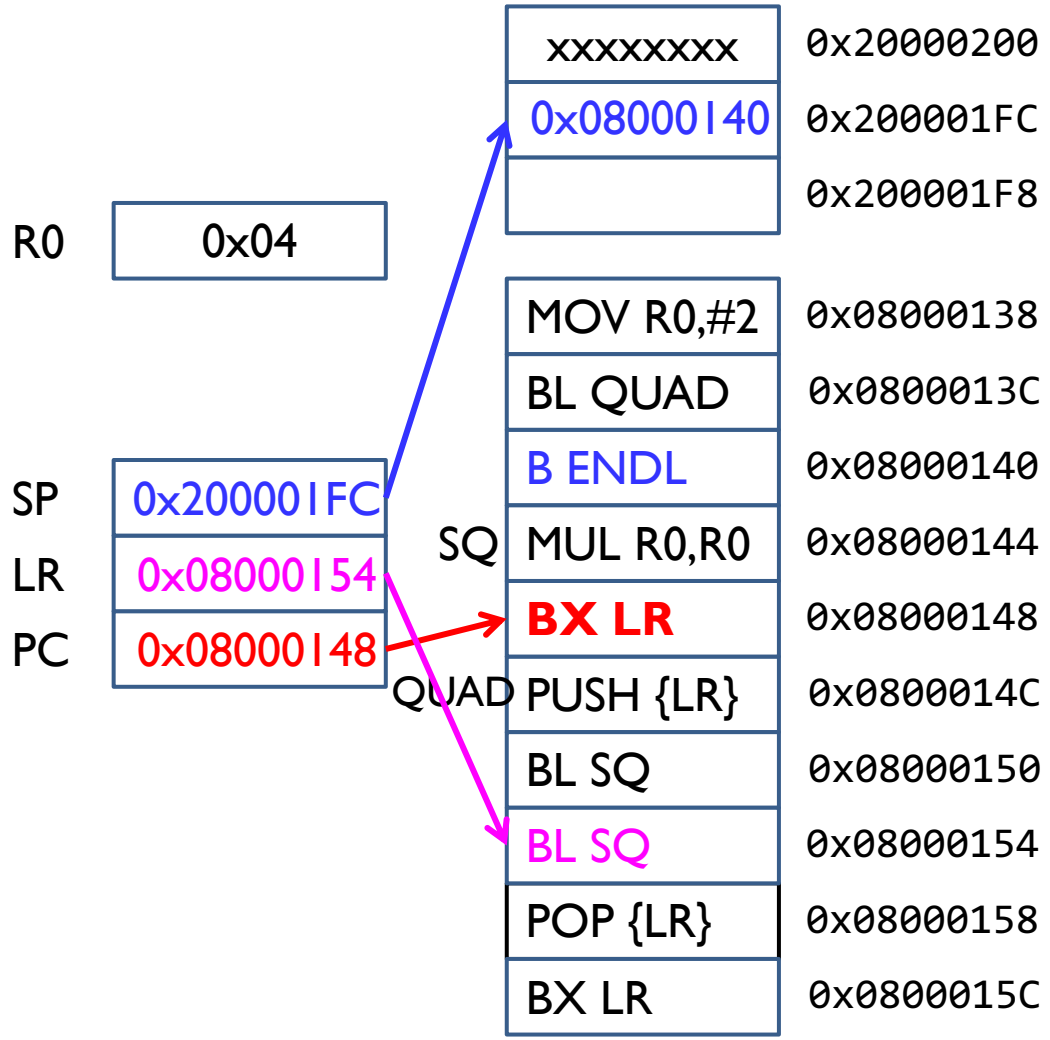


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

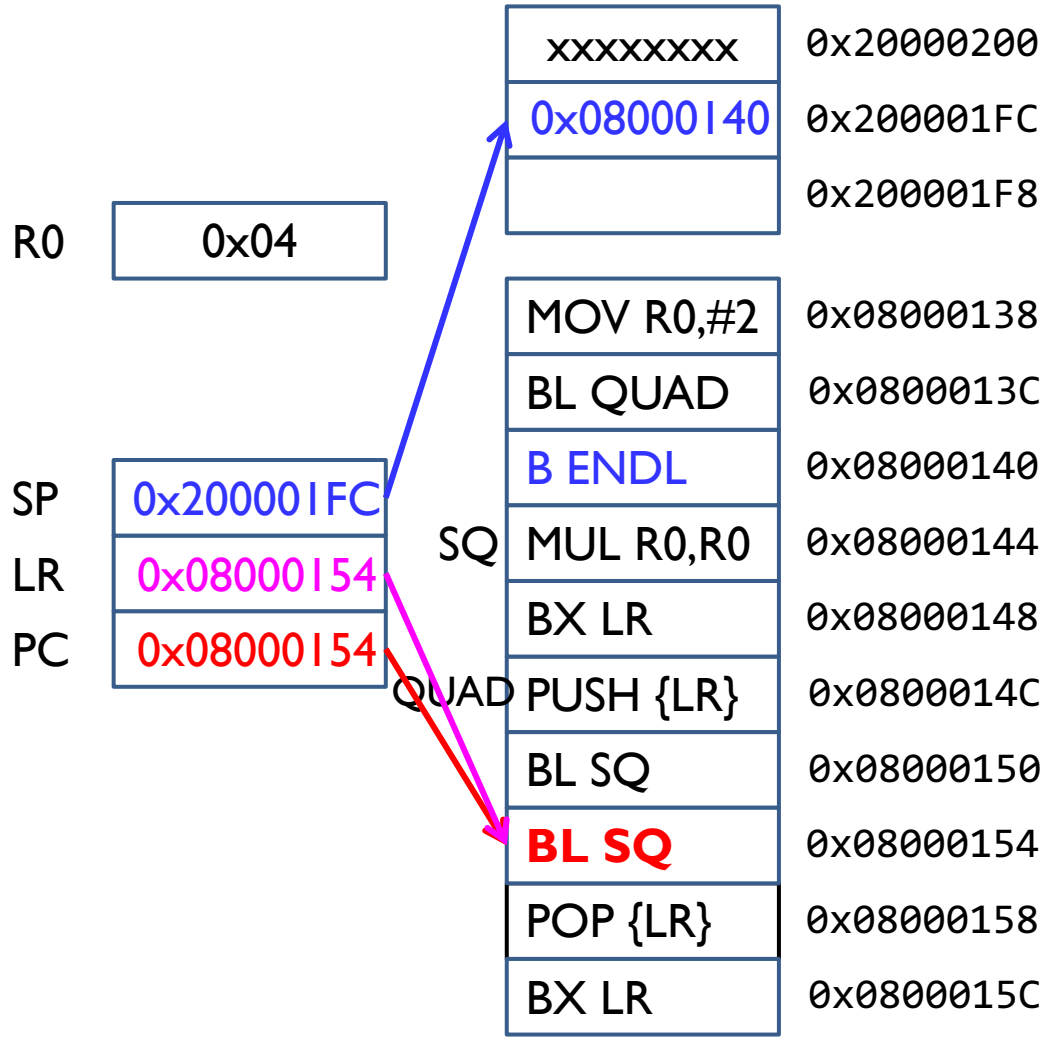


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

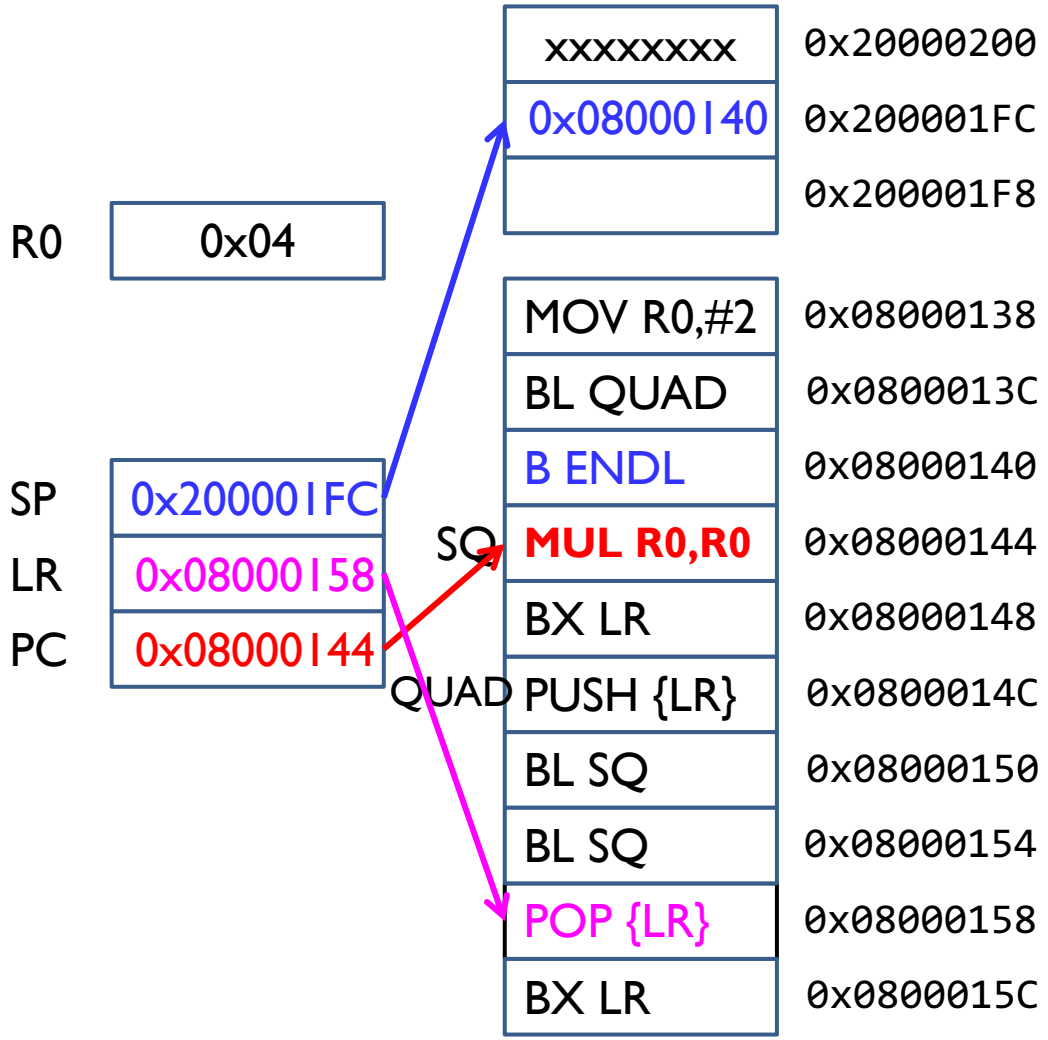


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

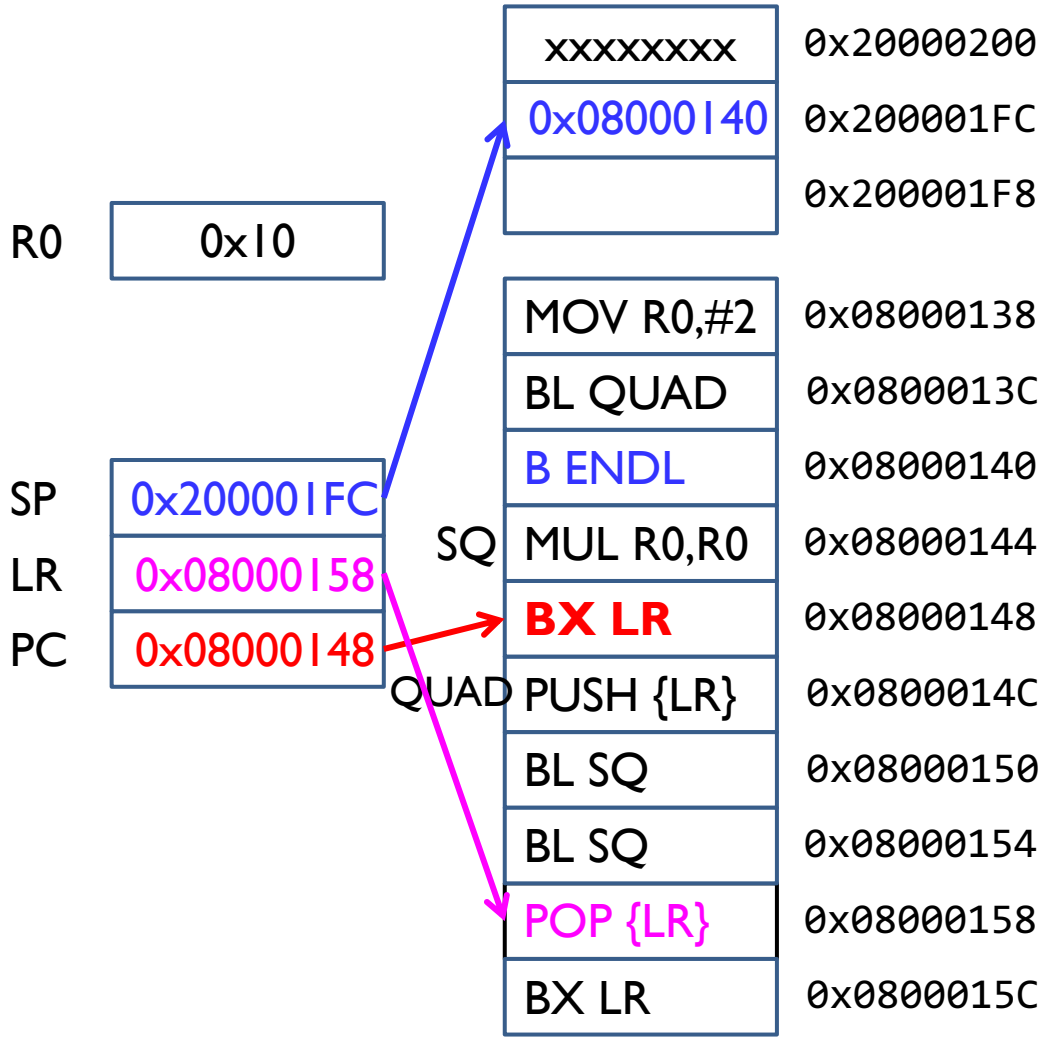


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

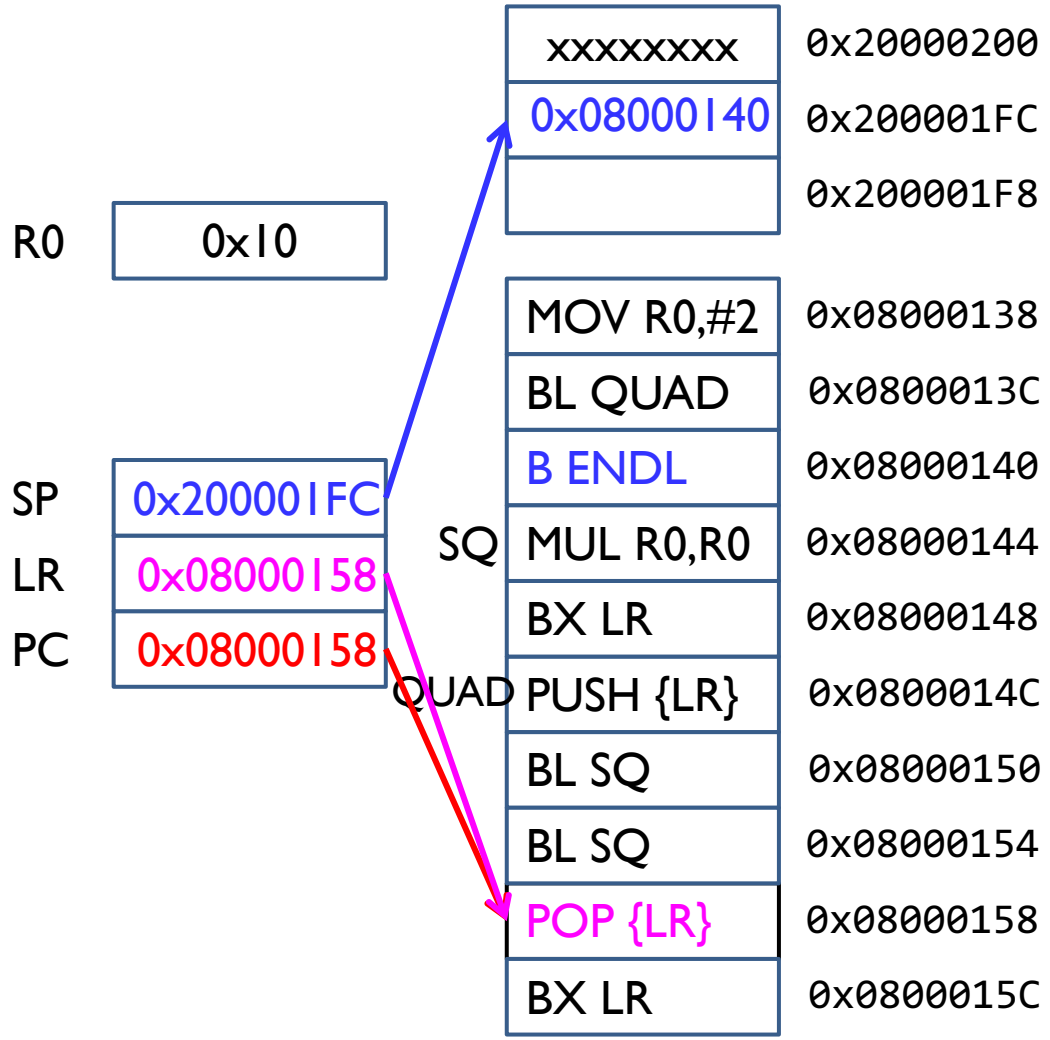


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

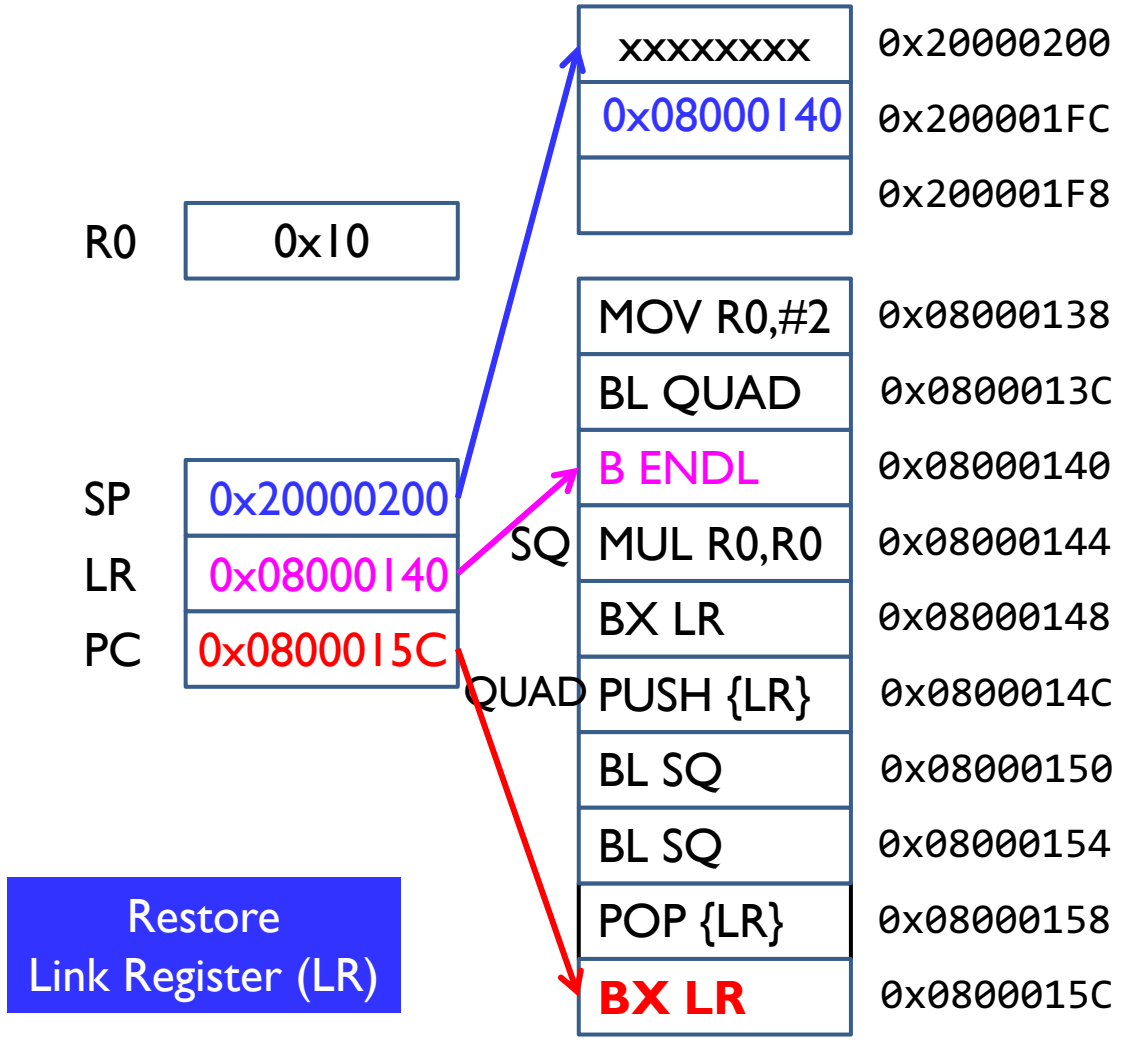


Example: R0 = R0^4

```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```

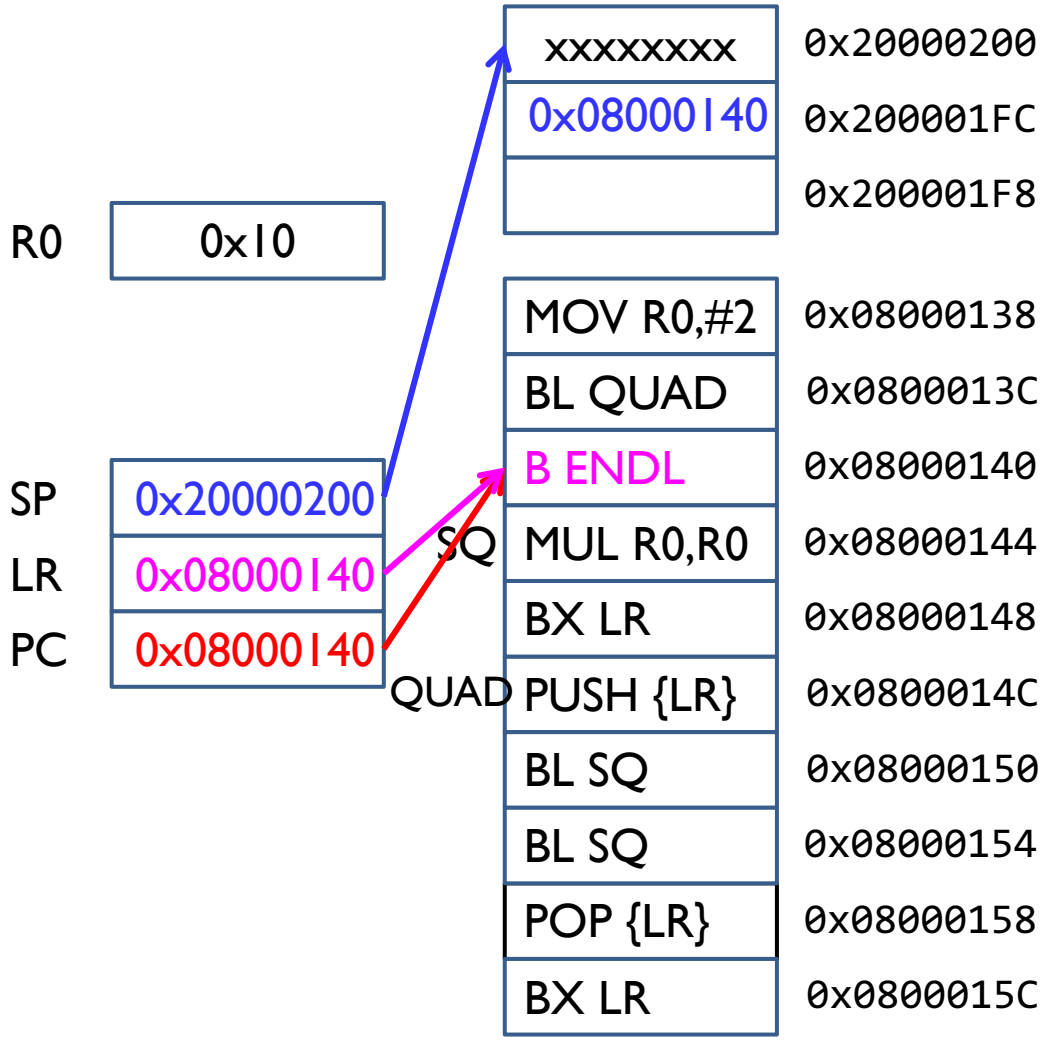


Example: R0 = R0^4

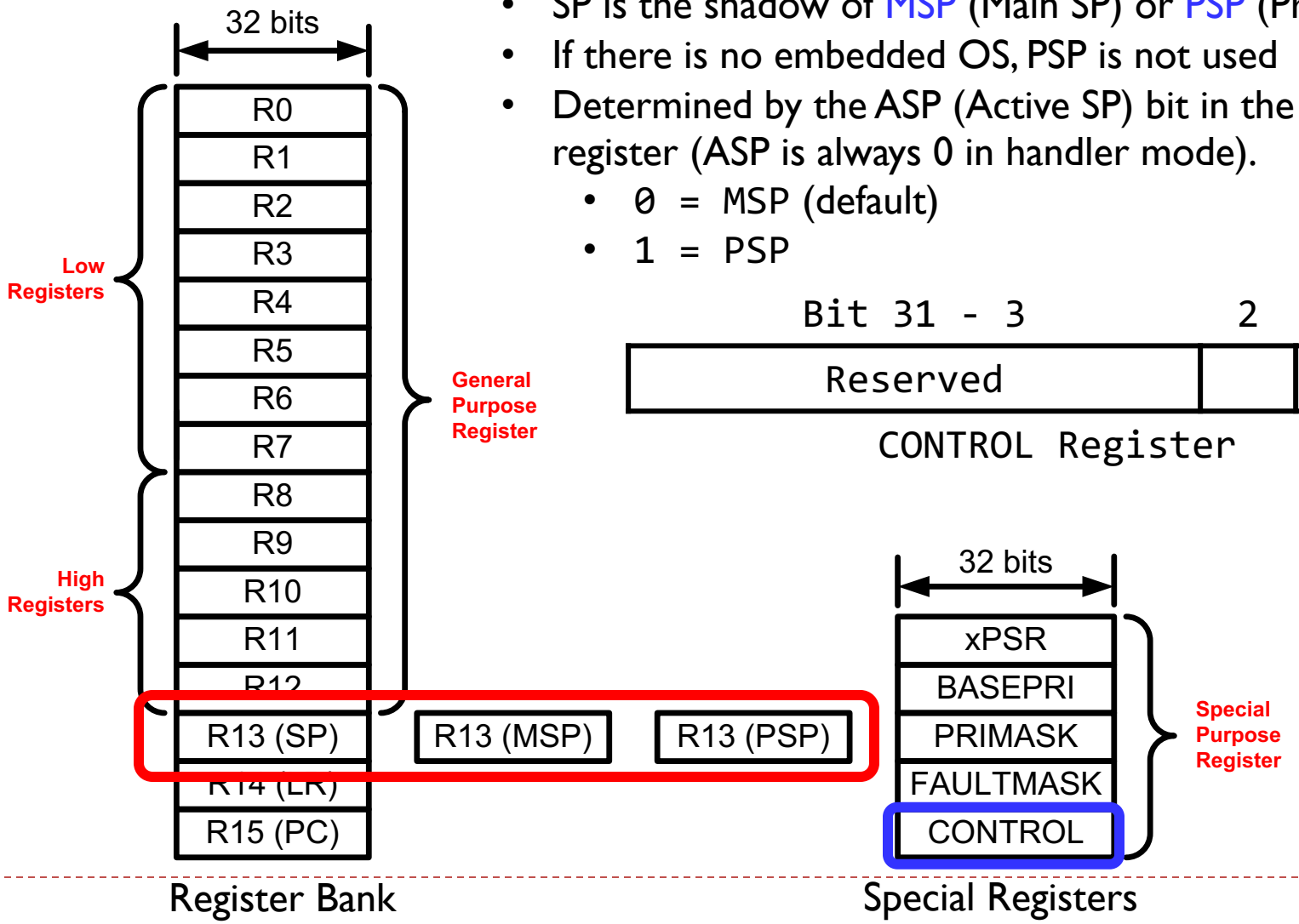
```
        MOV R0,#2
        BL QUAD
        B ENDL

SQ      MUL R0,R0
        BX LR

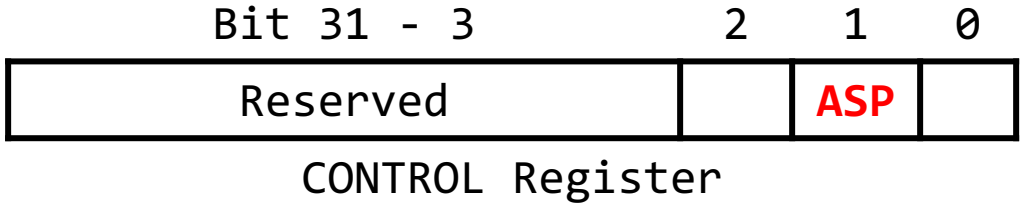
QUAD    PUSH {LR}
        BL SQ
        BL SQ
        POP {LR}
        BX LR
ENDL    ...
```



Stack Pointer (SP)



- SP is the shadow of MSP (Main SP) or PSP (Process SP)
- If there is no embedded OS, PSP is not used
- Determined by the ASP (Active SP) bit in the CONTROL register (ASP is always 0 in handler mode).
  - `0 = MSP (default)`
  - `1 = PSP`



Initializing the stack pointer (SP)

- Before using the stack, software has to define stack space and initialize the stack pointer (SP).
- The assembly file startup.s defines stack space and initialize SP.

Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,
            int32_t h, int32_t i, int32_t j, int32_t k);
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Caller

```
MOVS r0, #5
MOVS r1, #6
MOVS r2, #7
MOVS r3, #8
PUSH {r0, r1, r2, r3}
MOVS r0, #1
MOVS r1, #2
MOVS r2, #3
MOVS r3, #4
BL   sum
...
POP {r0, r1, r2, r3}
```

Callee

```
sum PROC
  EXPORT sum
  ADD r0, r0, r1    ; add a + b
  ADD r0, r0, r2    ; add c
  ADD r0, r0, r3    ; add d
  LDRD r1,r2, [sp]  ; r1=mem[sp],r2=mem[sp+4]
  ADD r0, r0, r1    ; add h
  ADD r0, r0, r2    ; add i
  LDRD r1,r2, [sp, #8] ; r1=mem[sp+8],r2=mem[sp+12]
  ADD r0, r0, r1    ; add j
  ADD r0, r0, r2    ; add k
  BX  LR
  ENDP
```

LDRD: Load Register Doubleword

Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,
            int32_t h, int32_t i, int32_t j, int32_t k);
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Caller

```
MOVS r0, #5
MOVS r1, #6
MOVS r2, #7
MOVS r3, #8
PUSH {r0, r1, r2, r3}
MOVS r0, #1
MOVS r1, #2
MOVS r2, #3
MOVS r3, #4
BL   sum
...
POP {r0, r1, r2, r3}
```

Callee

```
sum PROC
  EXPORT sum
  PUSH {r5, r6, lr}
  ADD r0, r0, r1    ; add a + b
  ADD r0, r0, r2    ; add c
  ADD r0, r0, r3    ; add d
  LDRD r5,r6, [sp, #12] ;r5=mem[sp+12],r6=mem[sp+16]
  ADD r0, r0, r5    ; add h
  ADD r0, r0, r6    ; add i
  LDRD r5,r6, [sp, #20] ;r5=mem[sp+20],r6=mem[sp+24]
  ADD r0, r0, r5    ; add j
  ADD r0, r0, r6    ; add k
  POP {r5, r6, pc}
  ENDP
```

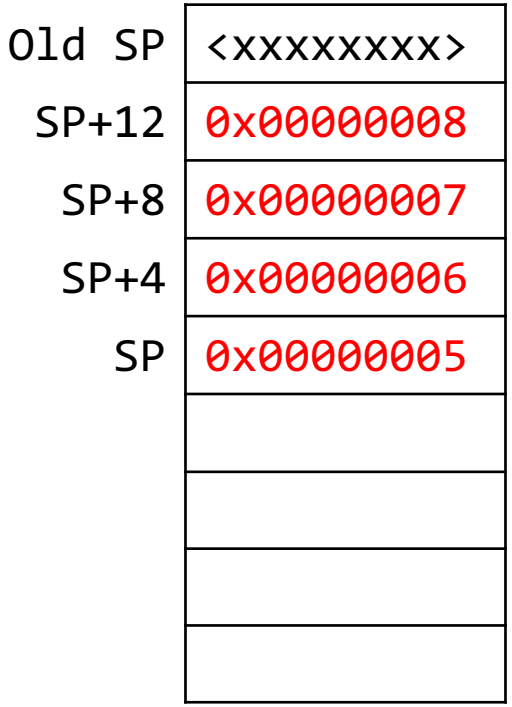
Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,
            int32_t h, int32_t i, int32_t j, int32_t k);
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Caller

```
MOVS r0, #5
MOVS r1, #6
MOVS r2, #7
MOVS r3, #8
PUSH {r0, r1, r2, r3}
MOVS r0, #1
MOVS r1, #2
MOVS r2, #3
MOVS r3, #4
BL   sum
...
POP {r0, r1, r2, r3}
```

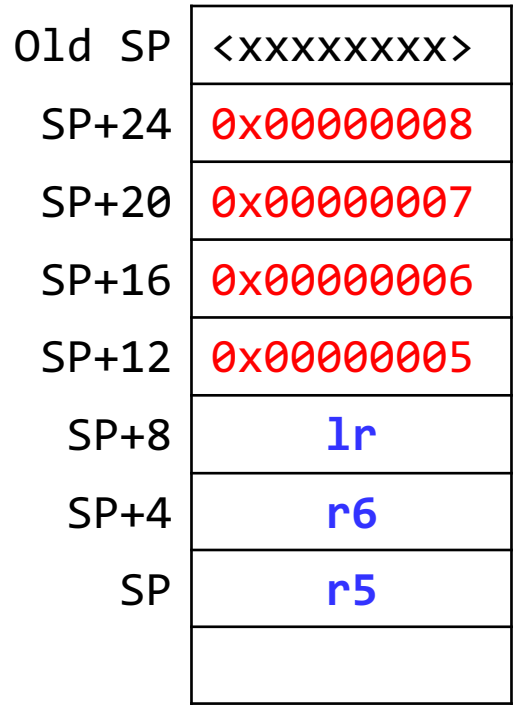
Stack



Passing Extra Arguments via Stack

```
int32_t sum(int32_t a, int32_t b, int32_t c, int32_t d,
            int32_t h, int32_t i, int32_t j, int32_t k);
s = sum(1, 2, 3, 4, 5, 6, 7, 8);
```

Stack



Callee

```
sum PROC
  EXPORT sum
  PUSH {r5, r6, lr}
  ADD r0, r0, r1    ; add a + b
  ADD r0, r0, r2    ; add c
  ADD r0, r0, r3    ; add d
  LDRD r5,r6, [sp, #12] ; r5=mem[sp+12],r6=mem[sp+16]
  ADD r0, r0, r5    ; add h
  ADD r0, r0, r6    ; add i
  LDRD r5,r6, [sp, #20] ; r5=mem[sp+20],r6=mem[sp+24]
  ADD r0, r0, r5    ; add j
  ADD r0, r0, r6    ; add k
  POP {r5, r6, pc}
  ENDP
```

Recursive Functions

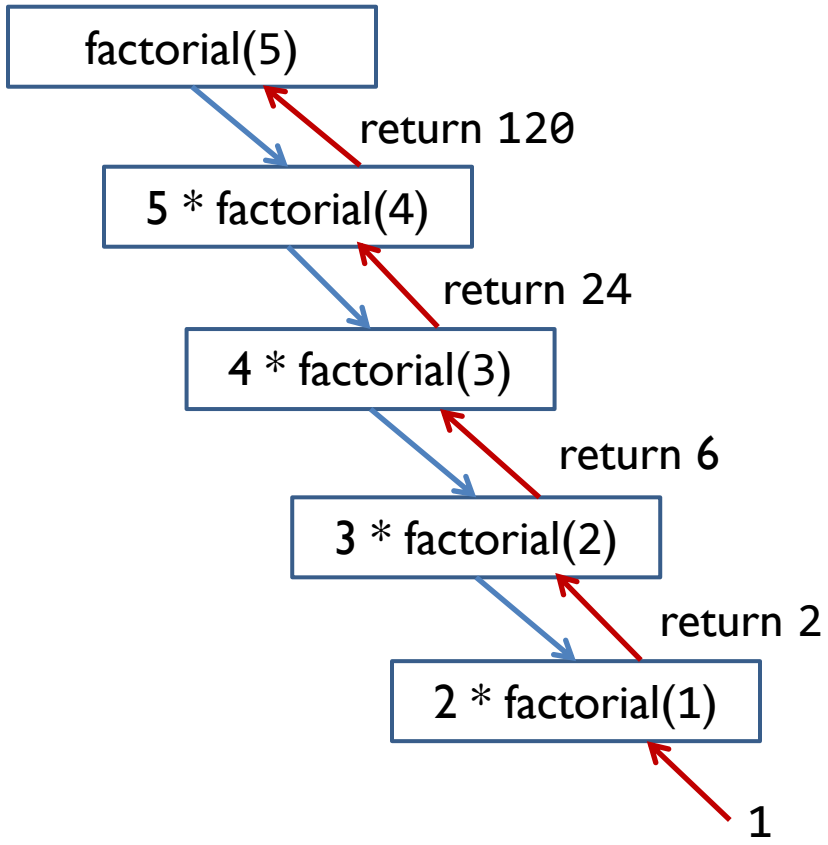
- A recursive function is one that solves its task by calling itself on smaller pieces of data.
- An effective tactic is to
  - divide a problem into sub-problems of the same type as the original,
  - solve those sub-problems, and
  - combine the results

Classic Example: Recursive Factorial in C

```
int factorial(int n);

int main(void){
  factorial(5);
  return 0;
}

int factorial(int n) {
  int f;
  if(n==1)
    return 1;
  else
    f = n * factorial(n-1);
  return f;
}
```



Recursive Functions

- `PUSH LR` (& working registers) onto stack before nested call
- `POP LR` (& working registers) off stack after nested return

Recursive Factorial in Assembly

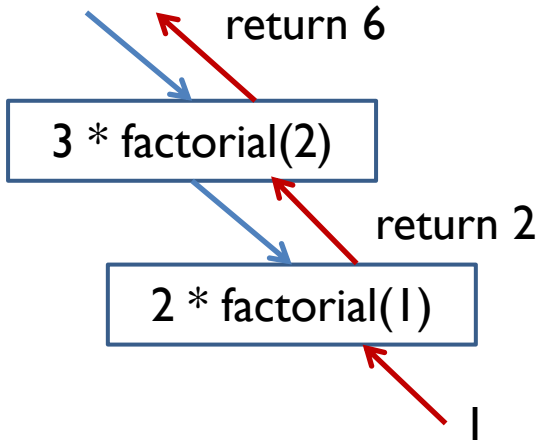
```
                AREA main, CODE, READONLY
                EXPORT  __main

                ENTRY

__main PROC
                MOV r0, #0x03
0x08000130      BL  factorial
0x08000134 stop B   stop
                ENDP

factorial
0x08000136      PUSH {r4, lr}
0x08000138      MOV  r4, r0
0x0800013A      CMP  r4, #0x01
0x0800013C      BNE  NZ
0x0800013E      MOVS r0, #0x01
0x08000140 loop POP  {r4, pc}
0x08000142 NZ   SUBS r0, r4, #1
0x08000144      BL   factorial
0x08000148      MUL  r0, r4, r0
0x0800014C      B    loop

                END
```



Recursive Factorial in Assembly

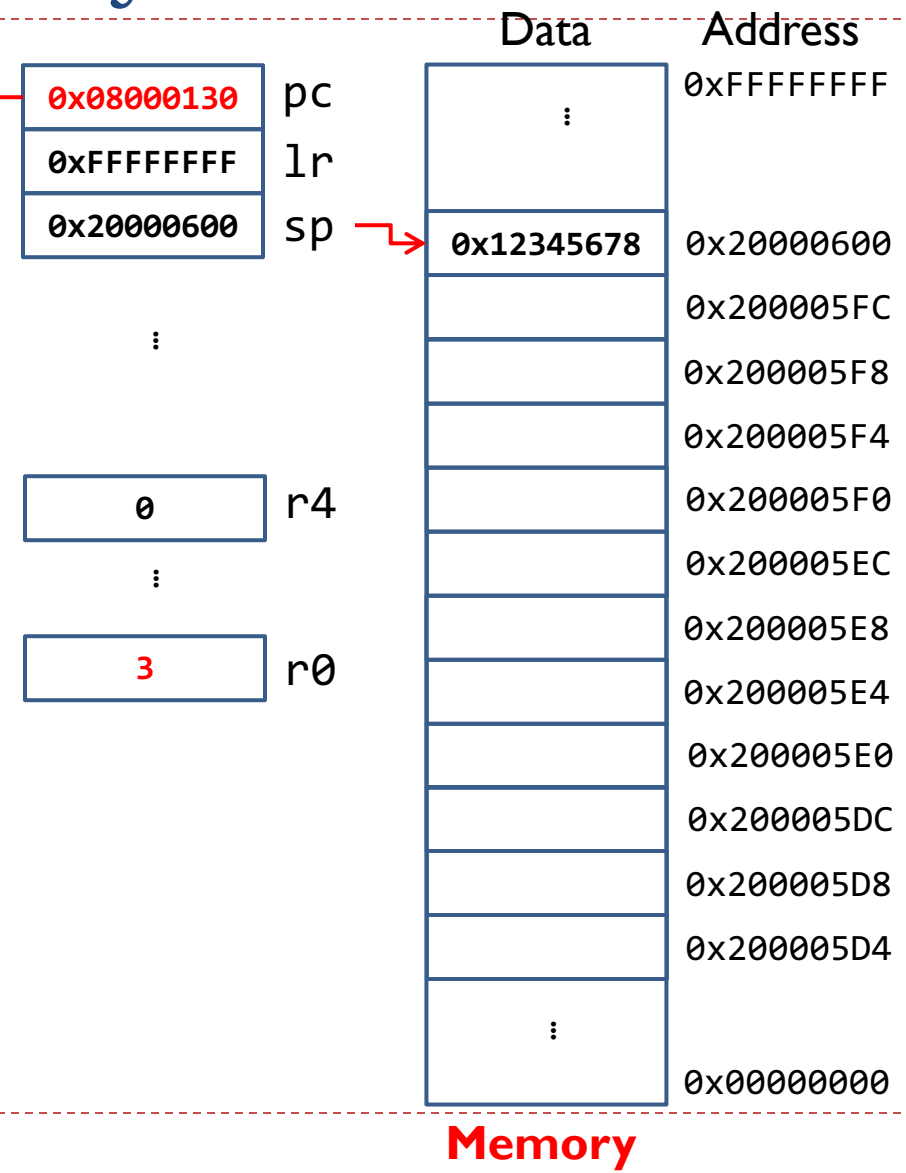
```
                AREA main, CODE, READONLY
                EXPORT  __main

                ENTRY

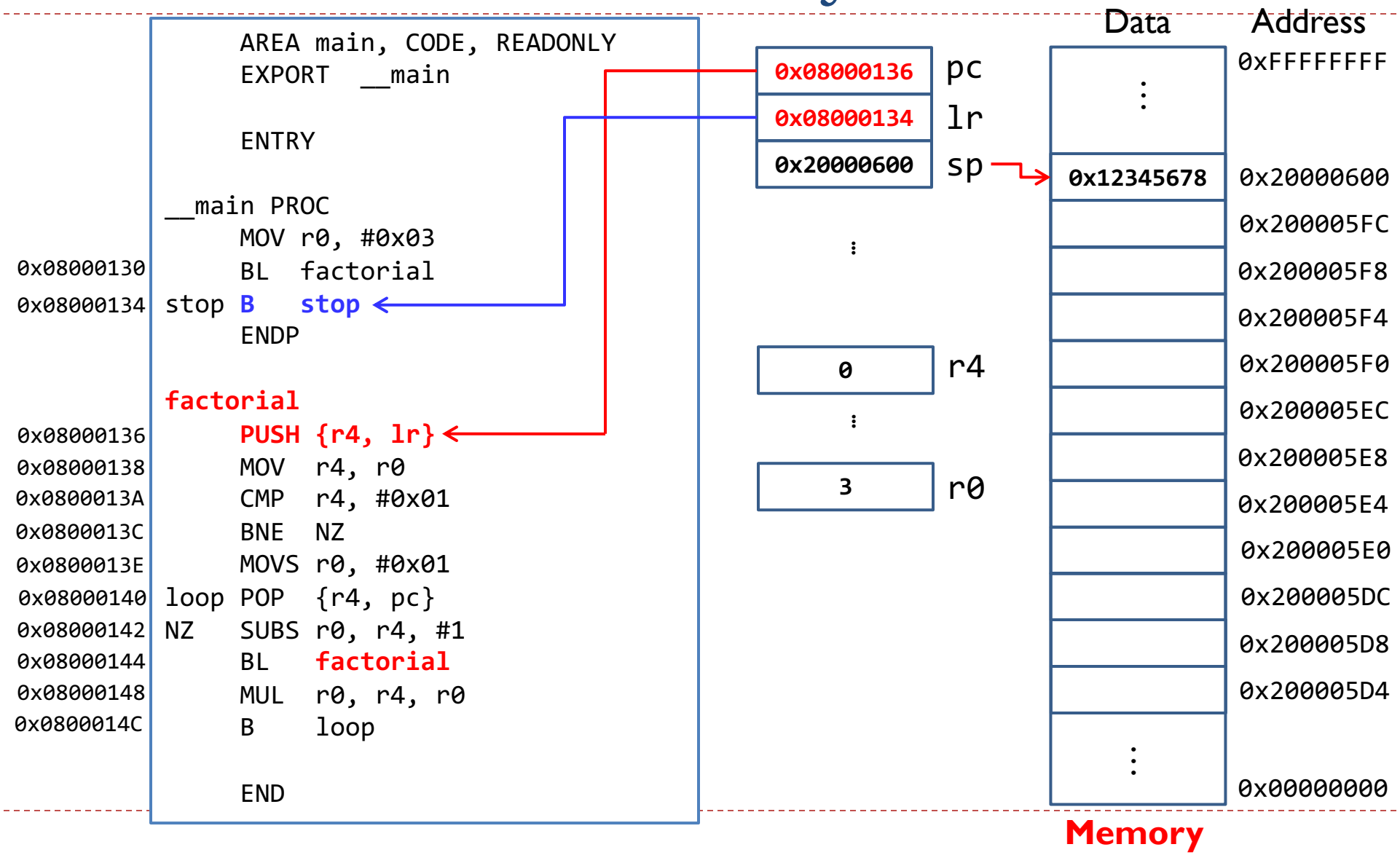
__main PROC
                MOV r0, #0x03
0x08000130      BL  factorial
0x08000134 stop B   stop
                ENDP

factorial
0x08000136      PUSH {r4, lr}
0x08000138      MOV  r4, r0
0x0800013A      CMP  r4, #0x01
0x0800013C      BNE  NZ
0x0800013E      MOVS r0, #0x01
0x08000140 loop POP  {r4, pc}
0x08000142 NZ   SUBS r0, r4, #1
0x08000144      BL   factorial
0x08000148      MUL  r0, r4, r0
0x0800014C      B    loop

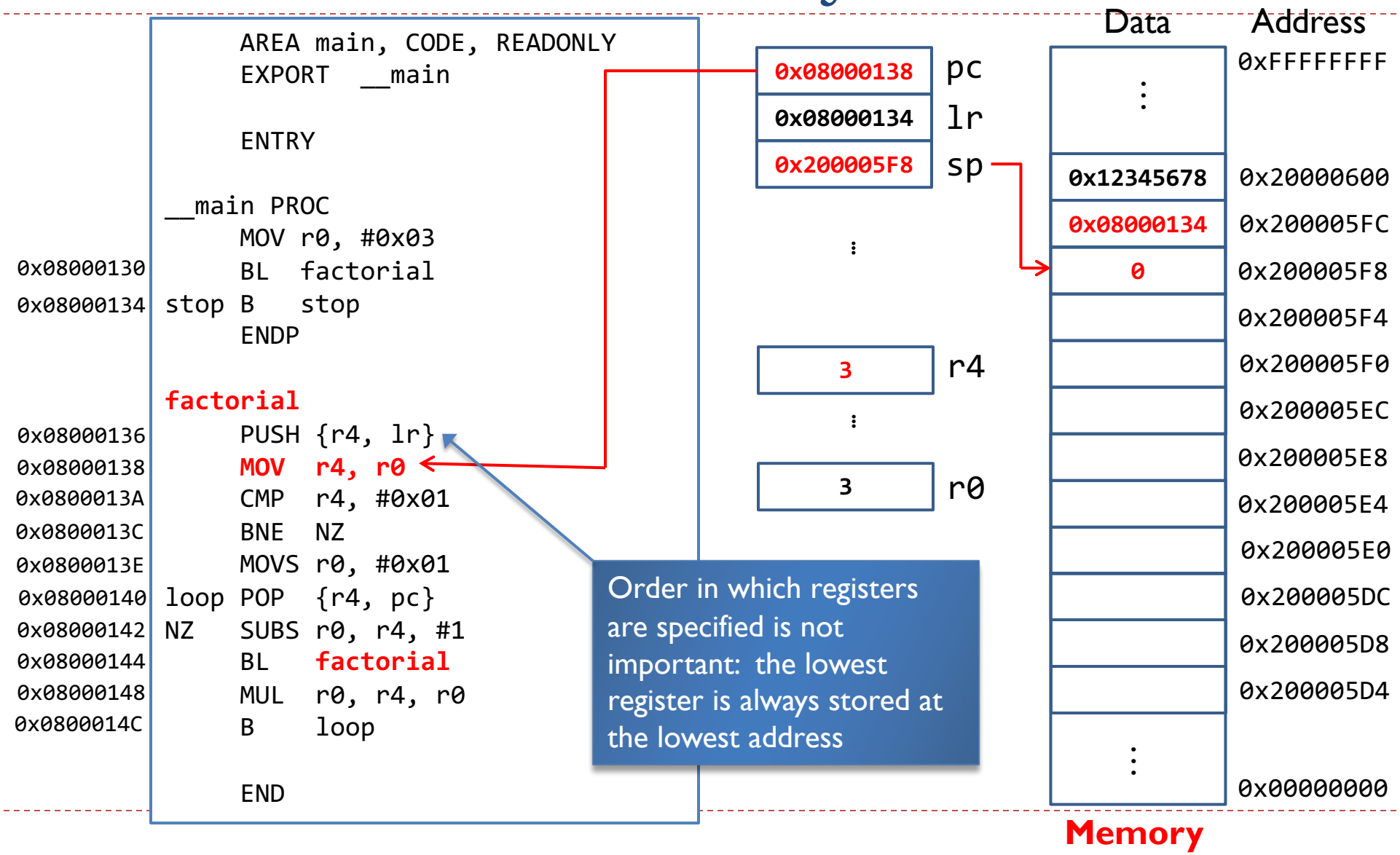
                END
```



Recursive Factorial in Assembly



Recursive Factorial in Assembly



Recursive Factorial in Assembly

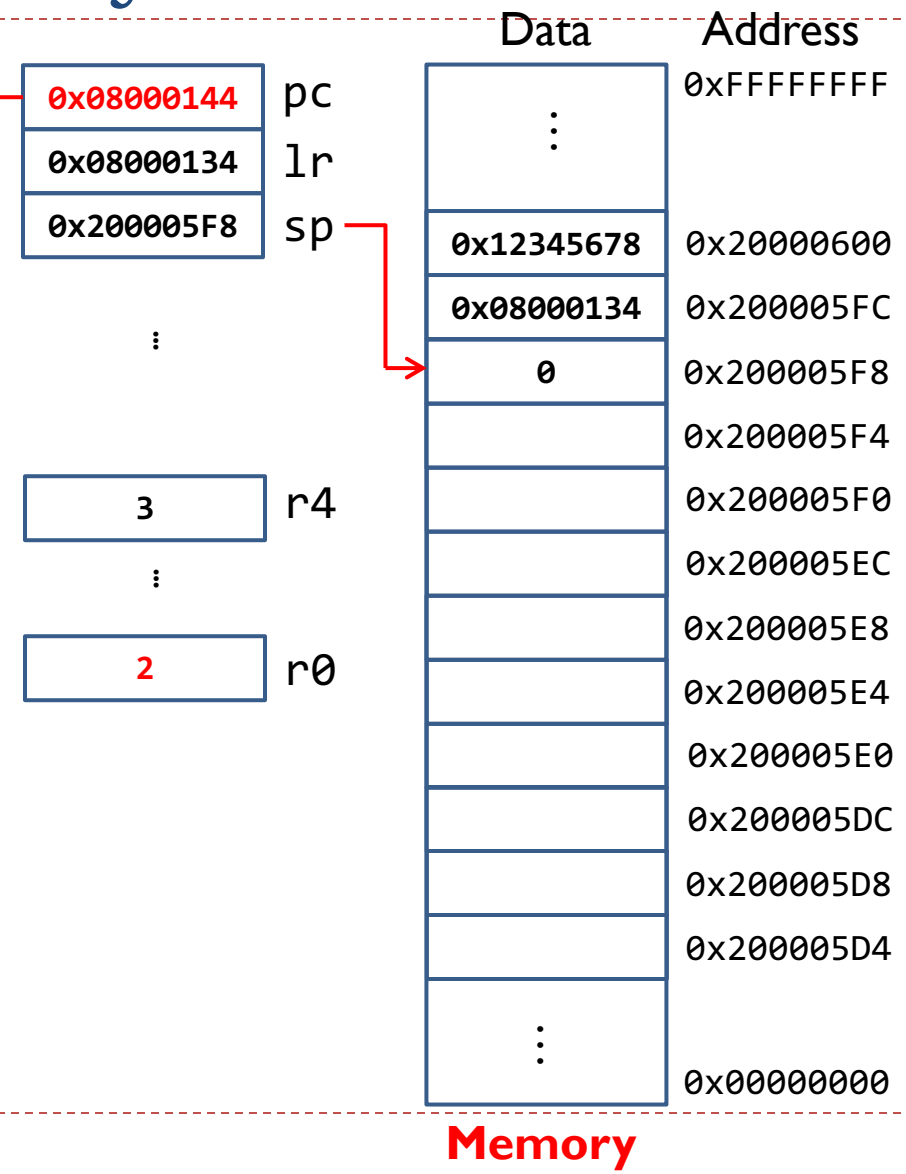
```
                AREA main, CODE, READONLY
                EXPORT  __main

                ENTRY

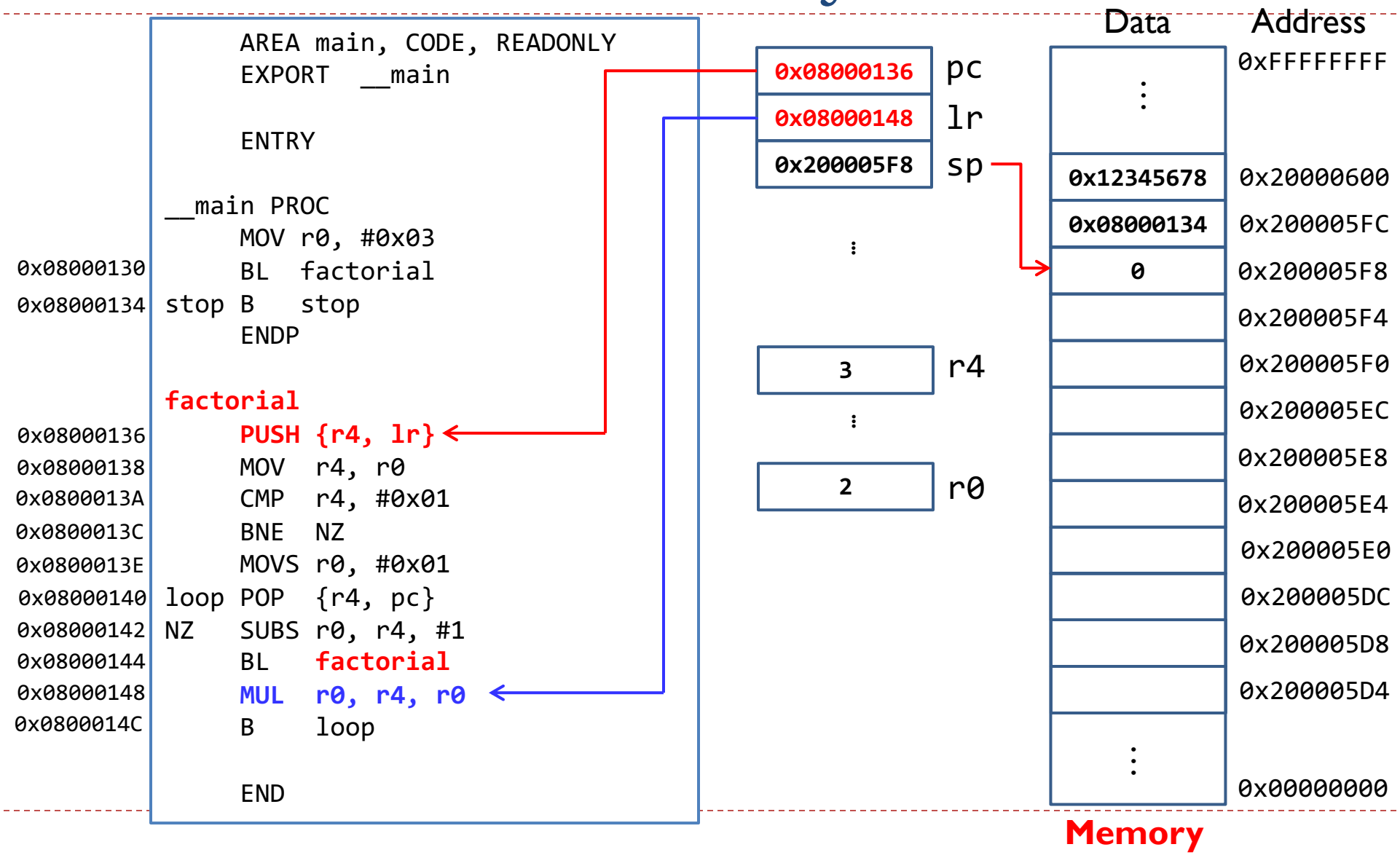
__main PROC
                MOV r0, #0x03
0x08000130      BL  factorial
0x08000134 stop B   stop
                ENDP

factorial
0x08000136      PUSH {r4, lr}
0x08000138      MOV  r4, r0
0x0800013A      CMP  r4, #0x01
0x0800013C      BNE  NZ
0x0800013E      MOVS r0, #0x01
0x08000140 loop POP  {r4, pc}
0x08000142 NZ   SUBS r0, r4, #1
0x08000144      BL   factorial
0x08000148      MUL  r0, r4, r0
0x0800014C      B    loop

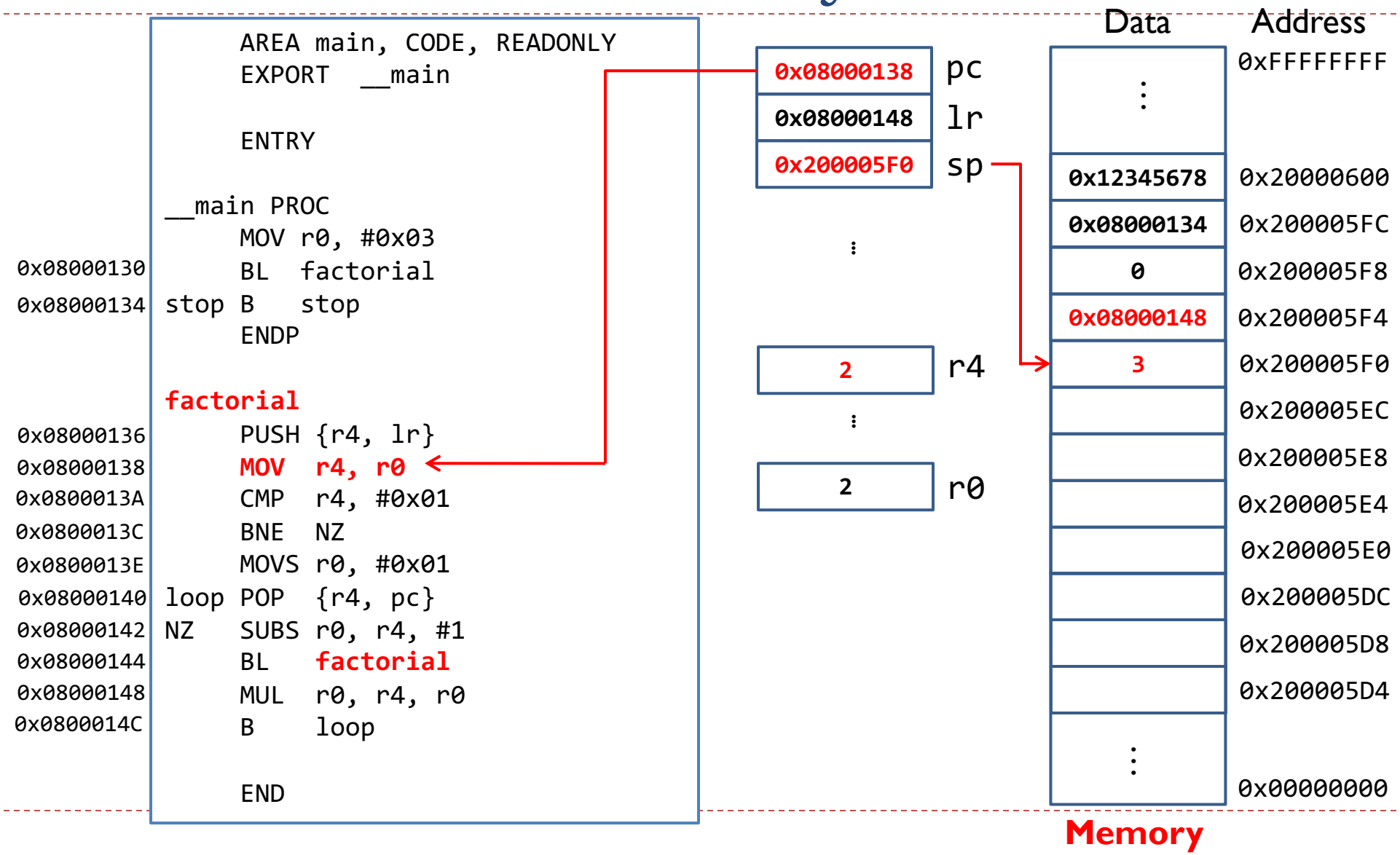
                END
```



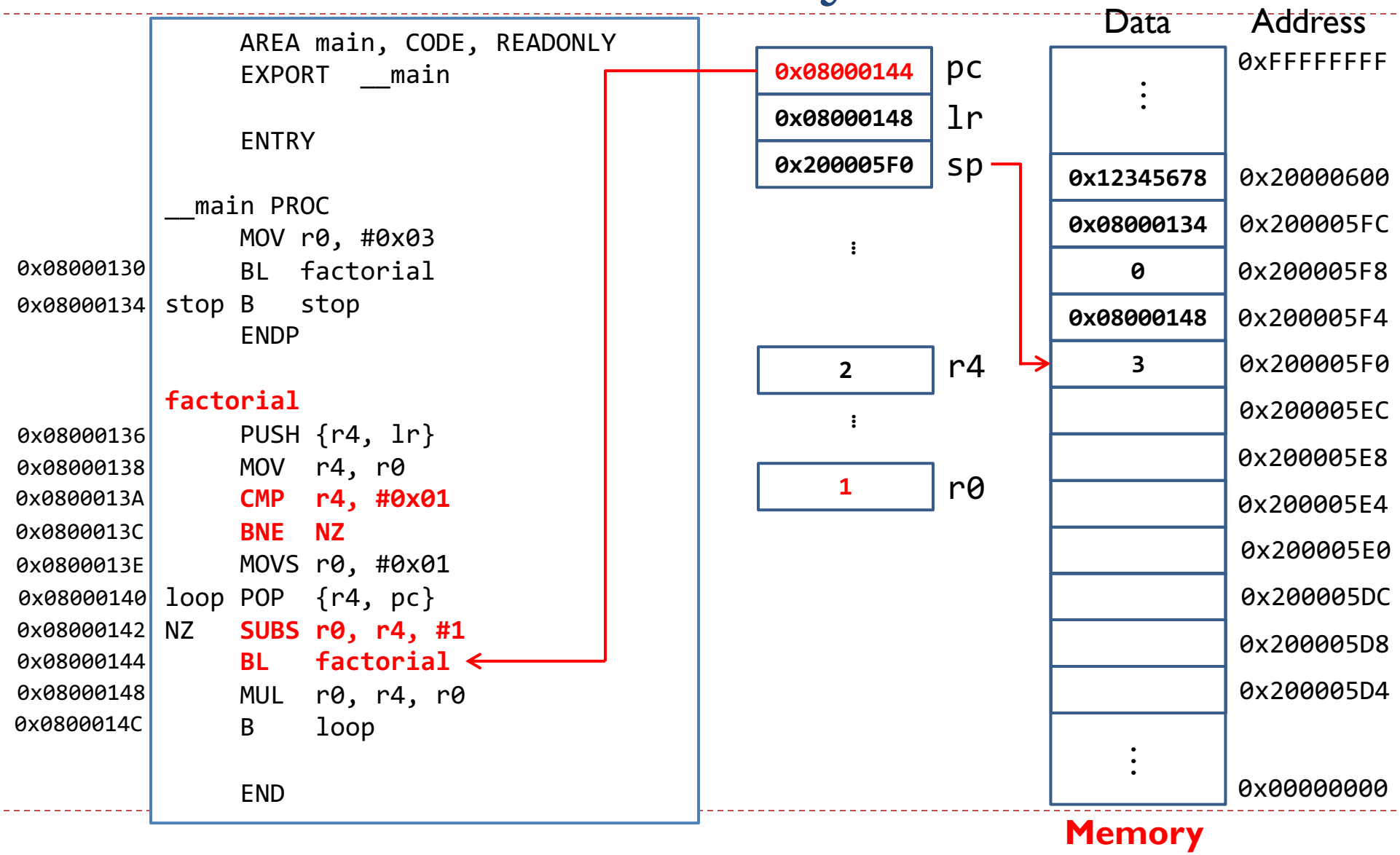
Recursive Factorial in Assembly



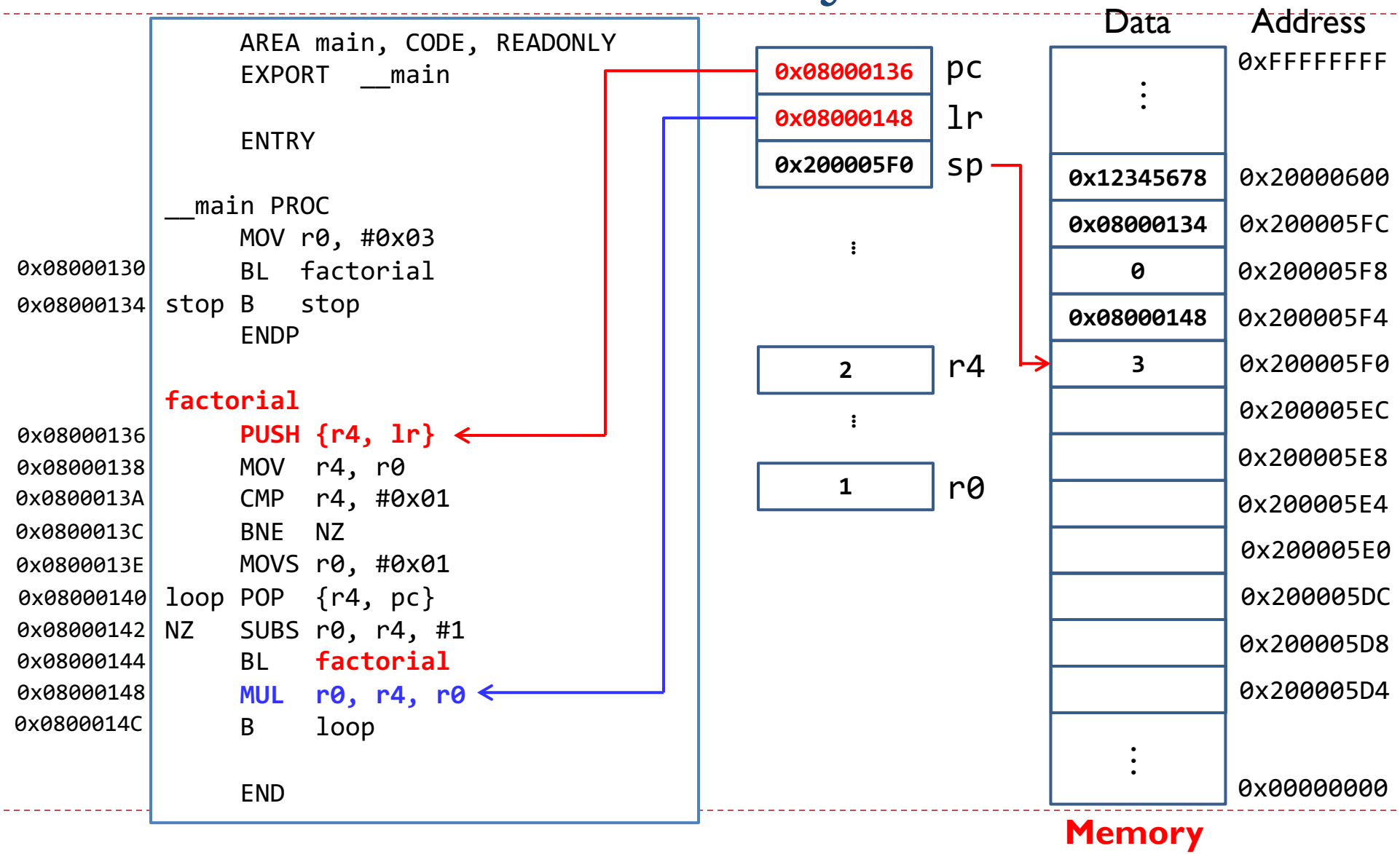
Recursive Factorial in Assembly



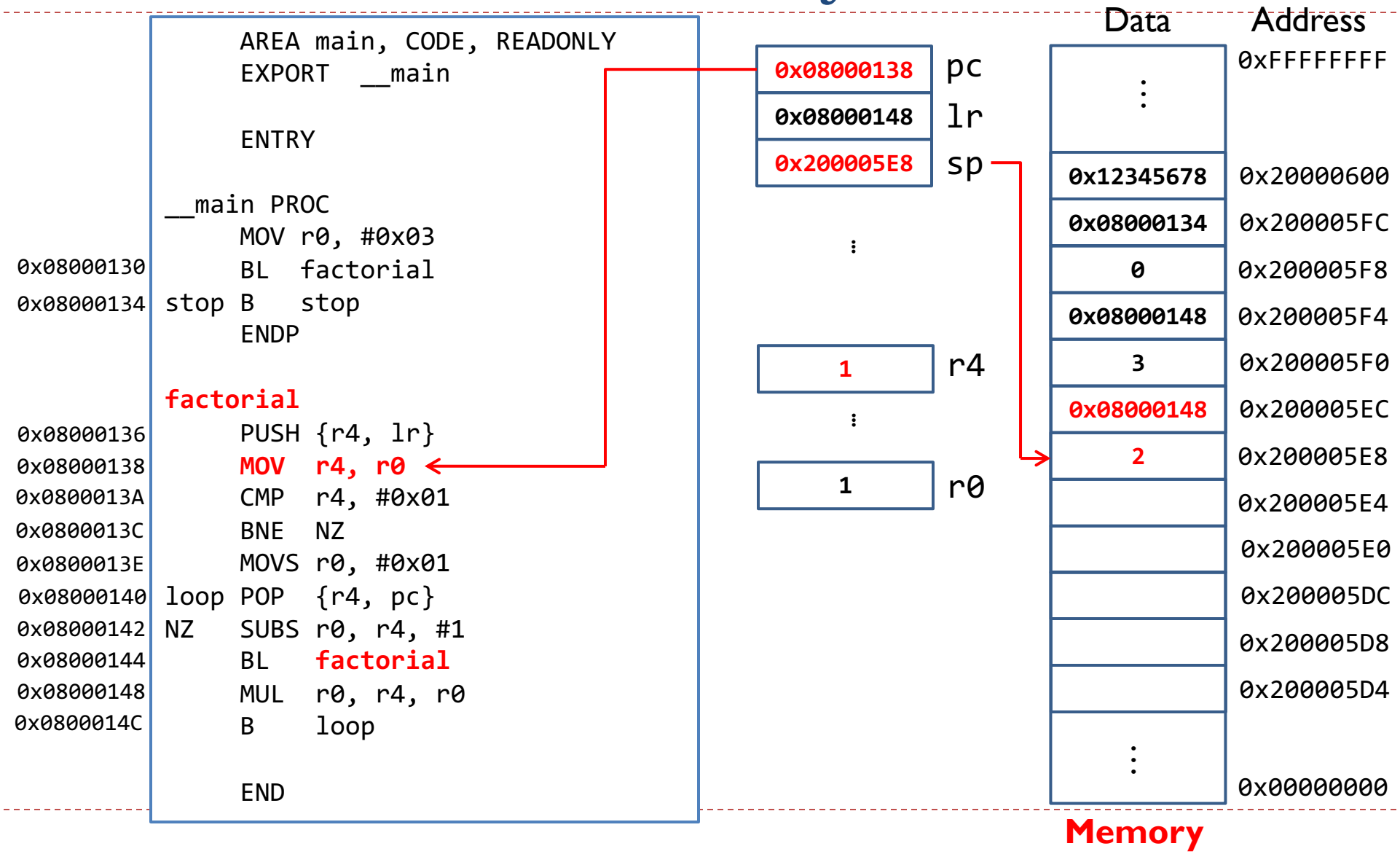
Recursive Factorial in Assembly



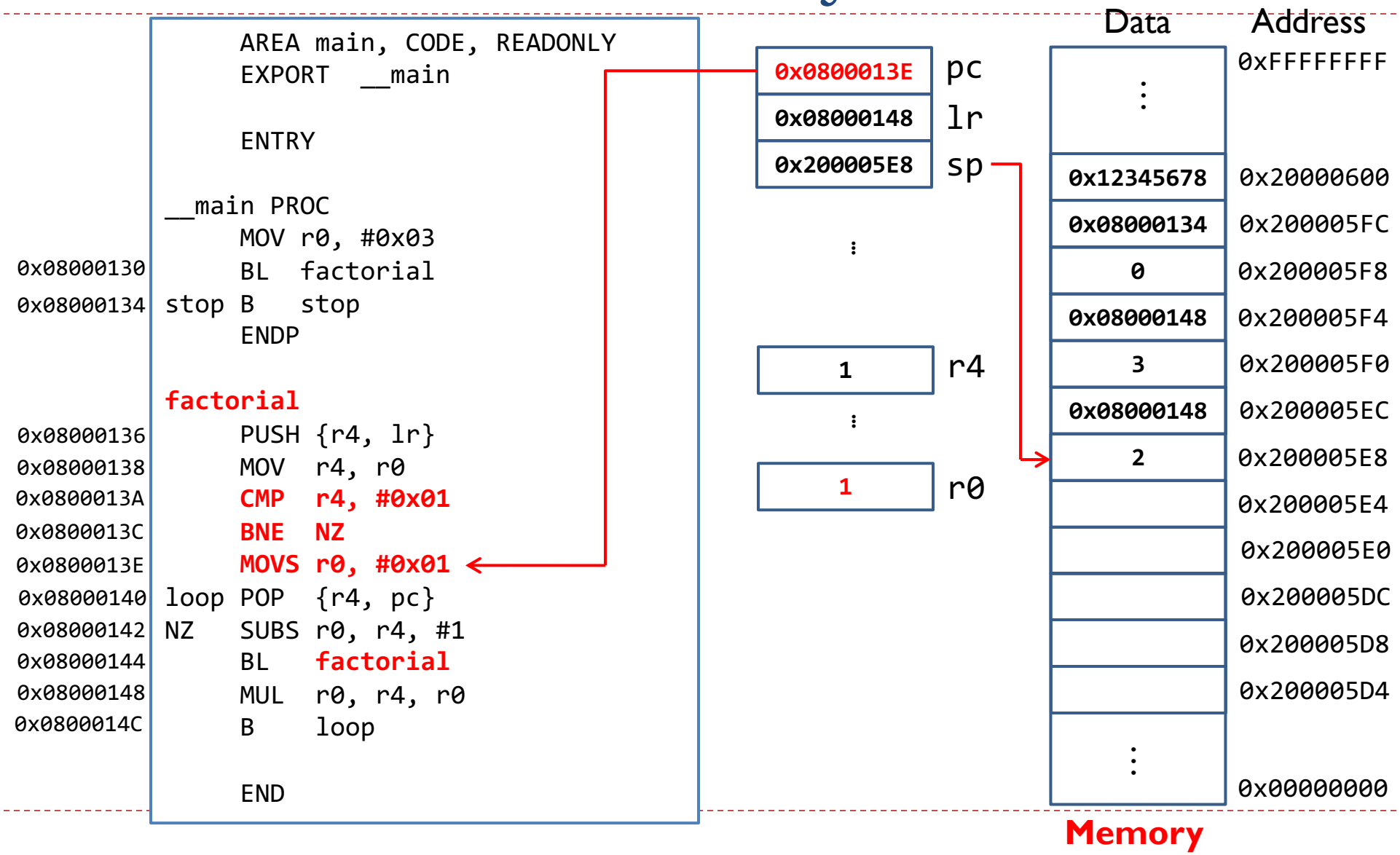
Recursive Factorial in Assembly



Recursive Factorial in Assembly



Recursive Factorial in Assembly



Recursive Factorial in Assembly

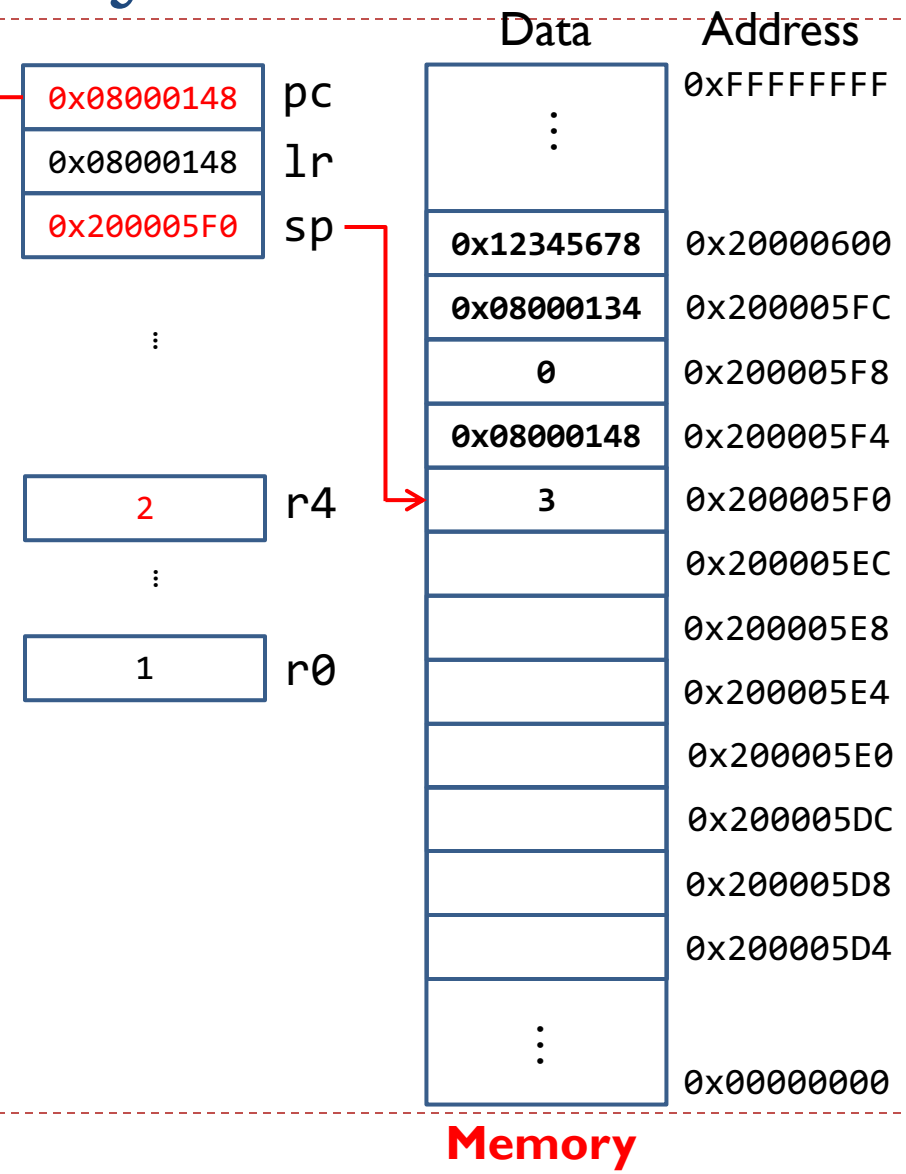
```
                AREA main, CODE, READONLY
                EXPORT  __main

                ENTRY

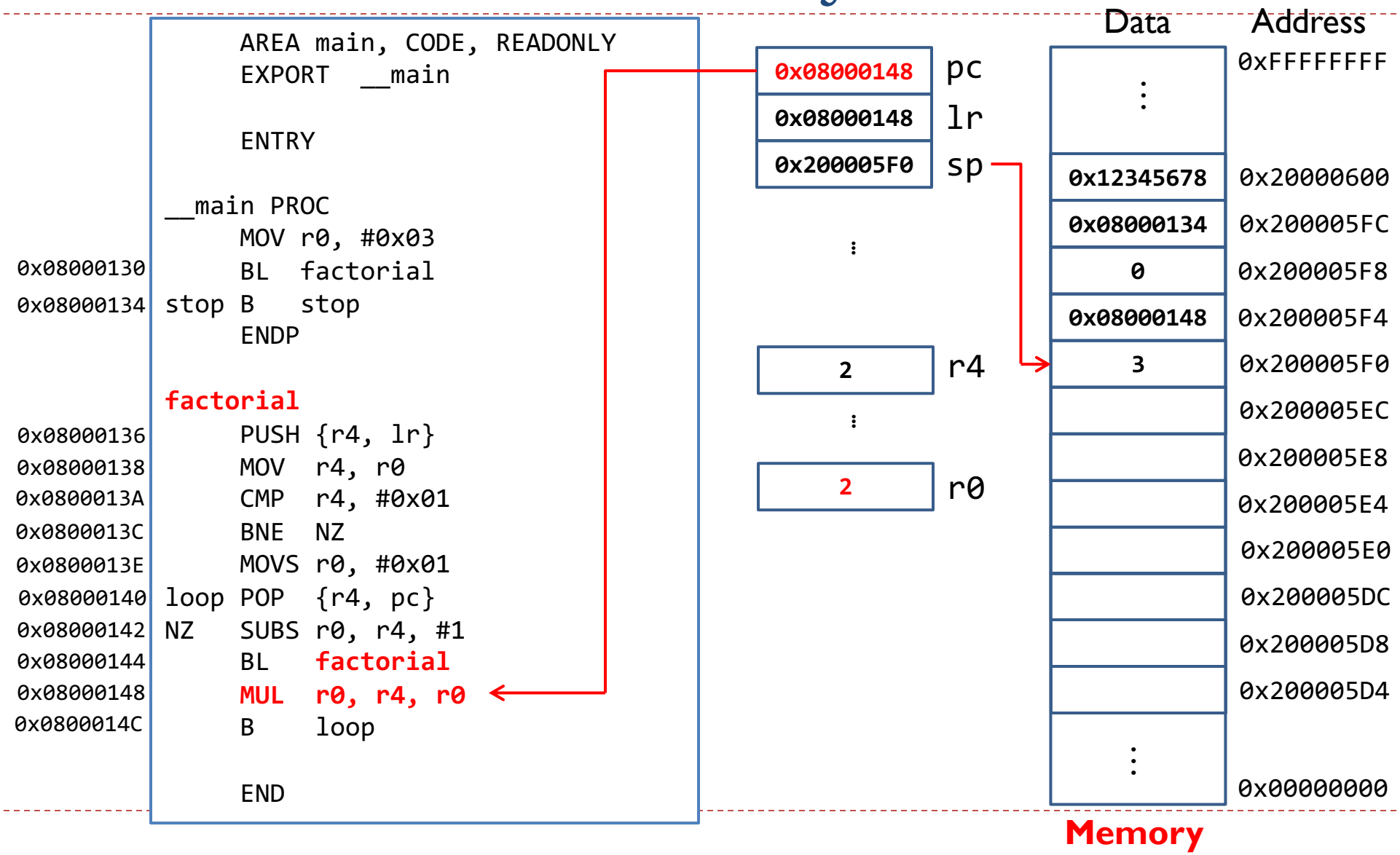
__main PROC
                MOV r0, #0x03
0x08000130      BL  factorial
0x08000134 stop B   stop
                ENDP

factorial
0x08000136      PUSH {r4, lr}
0x08000138      MOV  r4, r0
0x0800013A      CMP  r4, #0x01
0x0800013C      BNE  NZ
0x0800013E      MOVS r0, #0x01
0x08000140 loop POP  {r4, pc}
0x08000142 NZ   SUBS r0, r4, #1
0x08000144      BL   factorial
0x08000148      MUL  r0, r4, r0
0x0800014C      B    loop

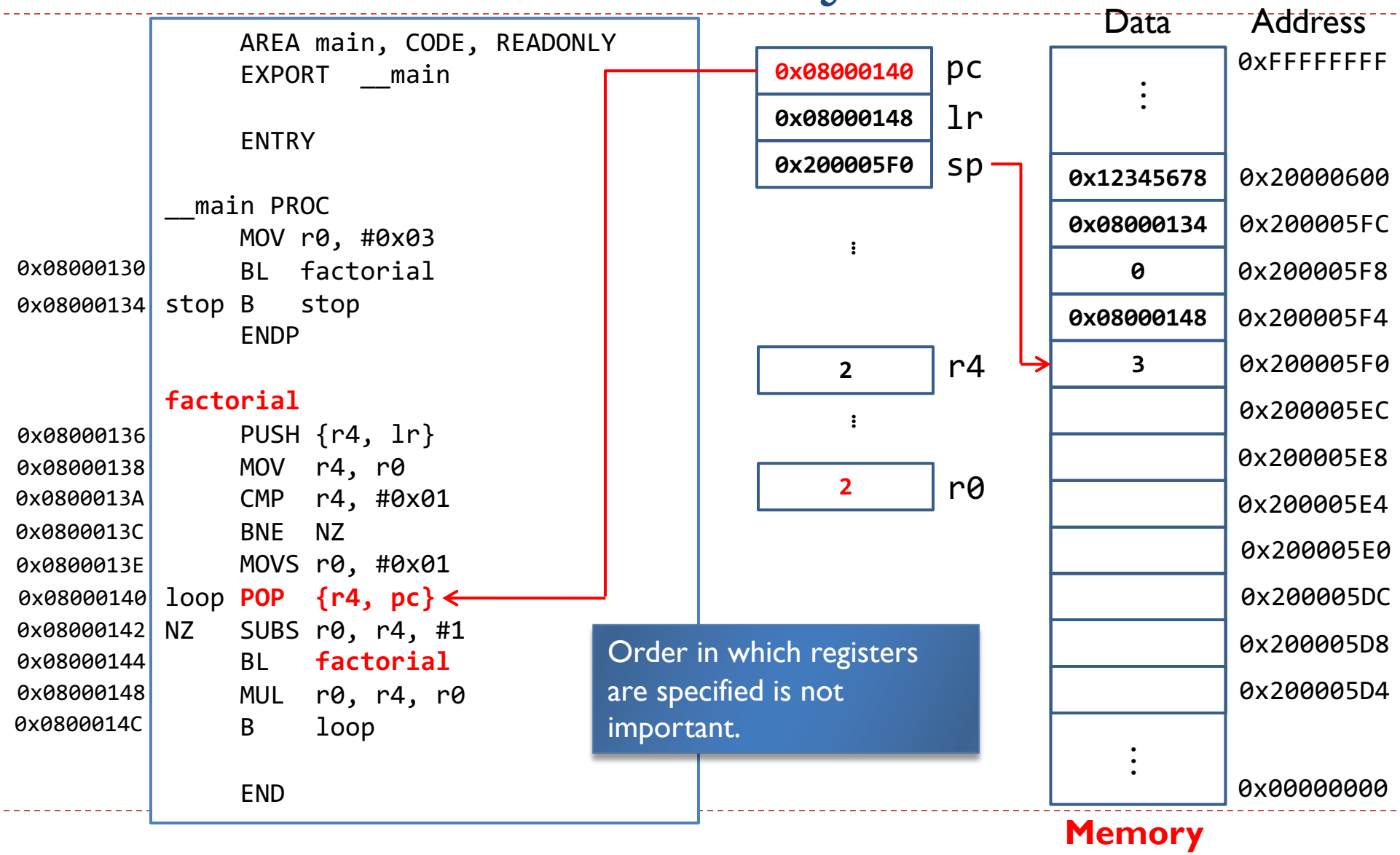
                END
```



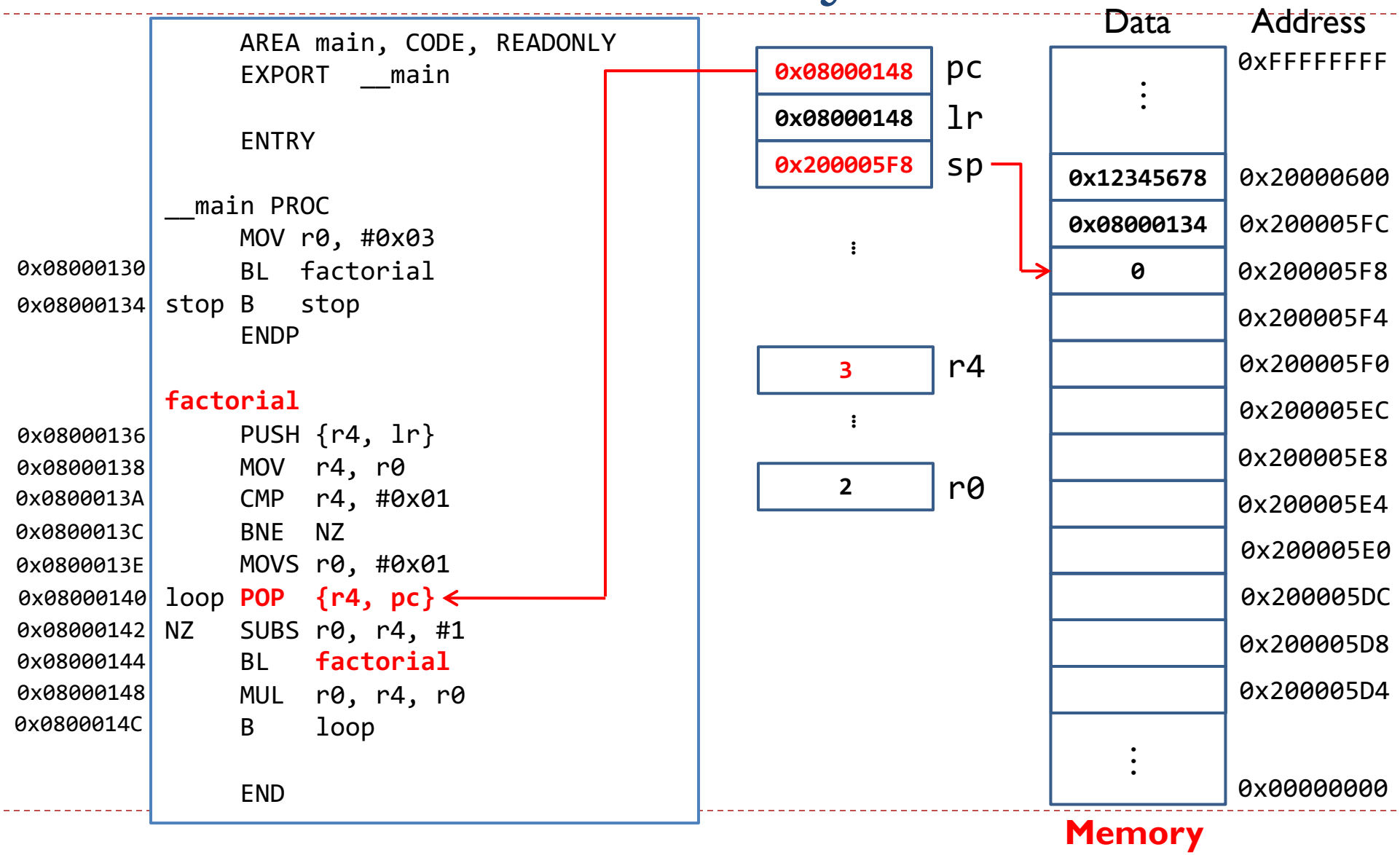
Recursive Factorial in Assembly



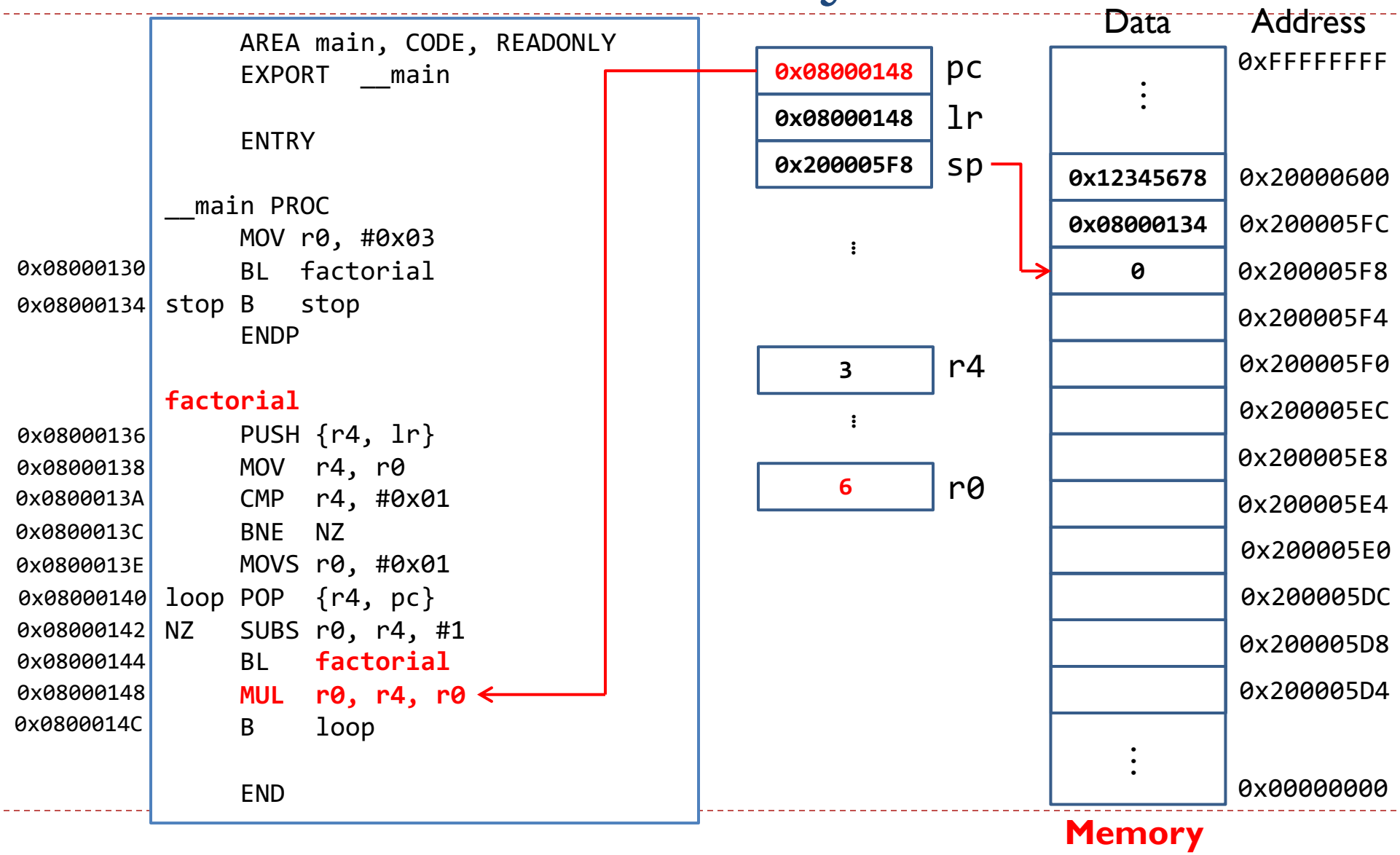
Recursive Factorial in Assembly



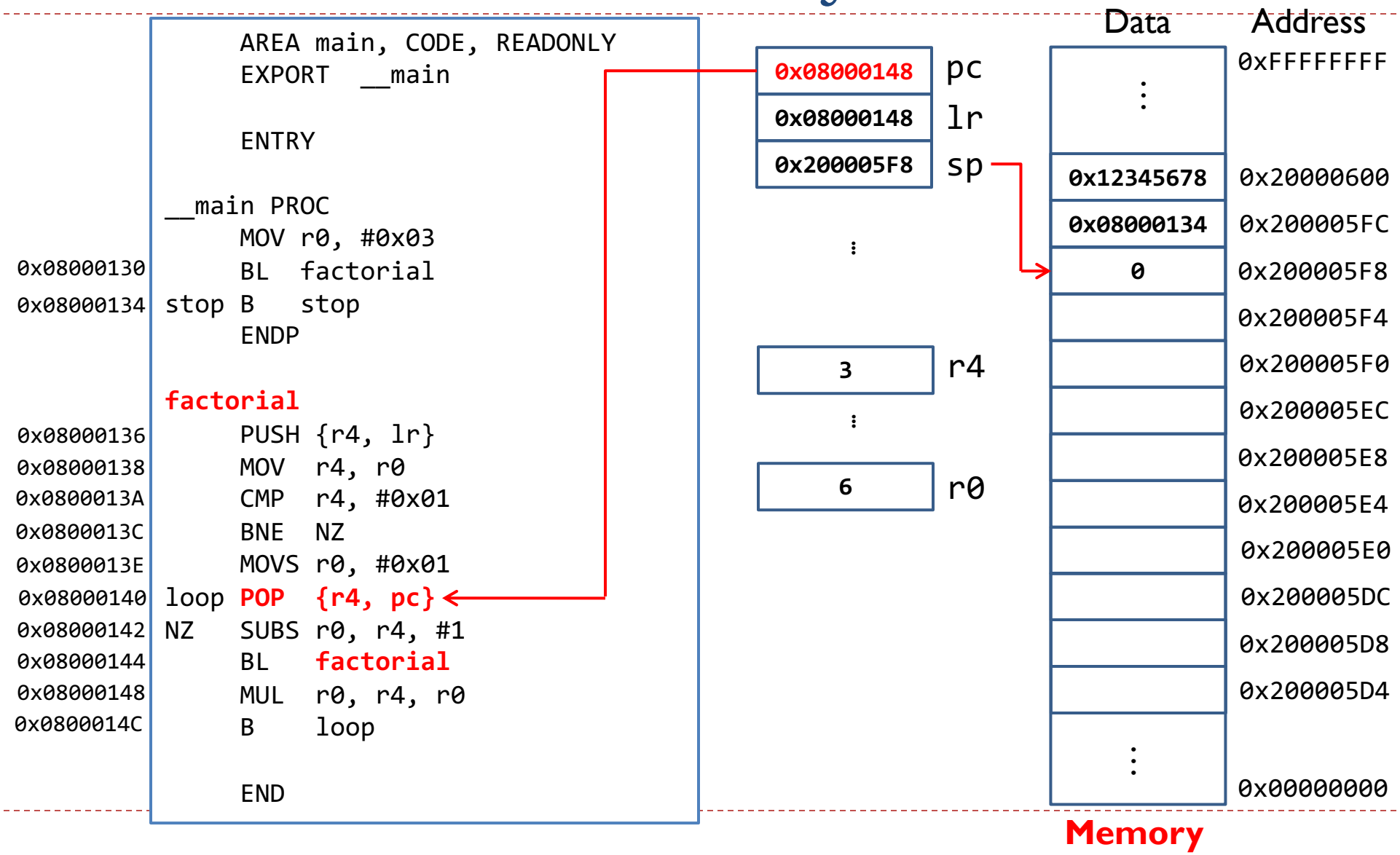
Recursive Factorial in Assembly



Recursive Factorial in Assembly



Recursive Factorial in Assembly



Recursive Factorial in Assembly

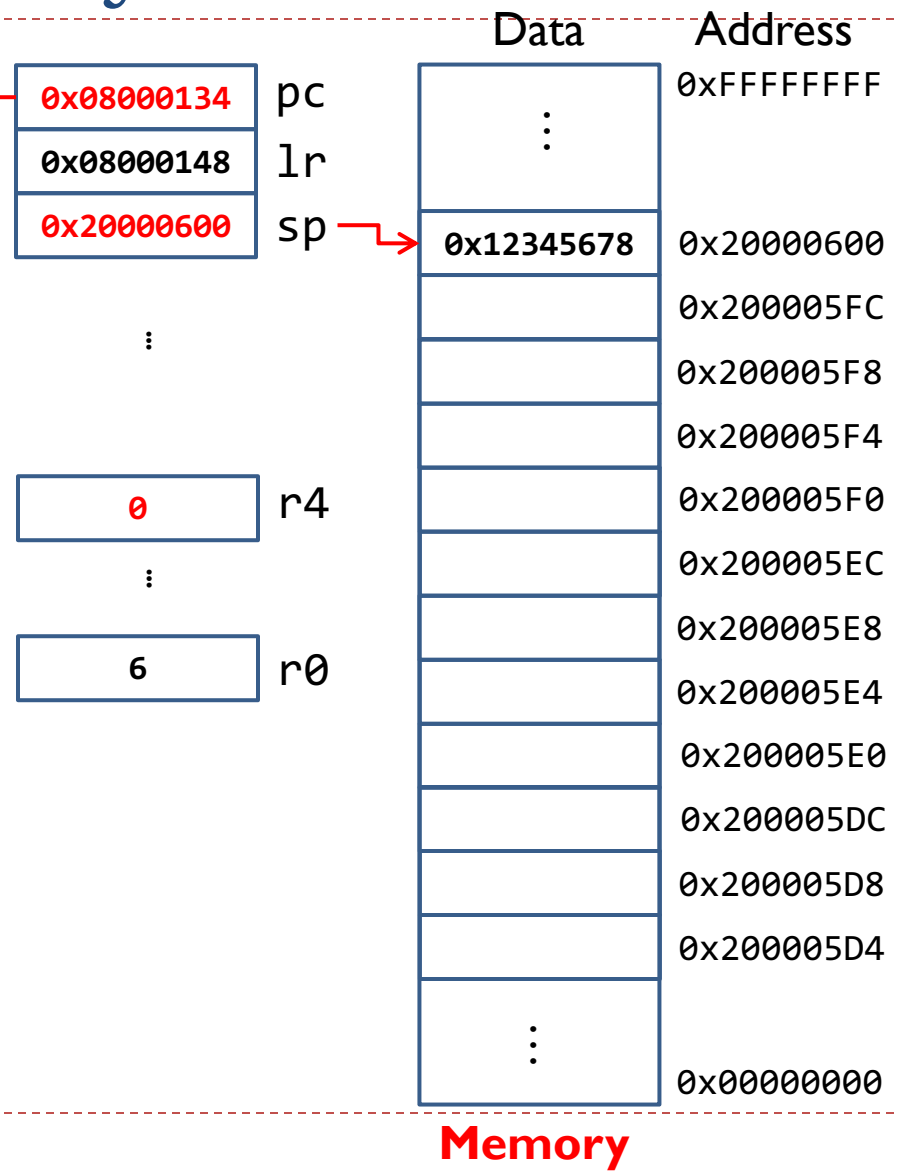
```
            AREA main, CODE, READONLY
            EXPORT  __main

            ENTRY

__main PROC
            MOV r0, #0x03
0x08000130  BL  factorial
0x08000134  stop B   stop
            ENDP

factorial
0x08000136  PUSH {r4, lr}
0x08000138  MOV  r4, r0
0x0800013A  CMP  r4, #0x01
0x0800013C  BNE  NZ
0x0800013E  MOVS r0, #0x01
0x08000140  loop POP  {r4, pc}
0x08000142  NZ   SUBS r0, r4, #1
0x08000144  BL   factorial
0x08000148  MUL  r0, r4, r0
0x0800014C  B    loop

            END
```



Recursive Factorial in Assembly

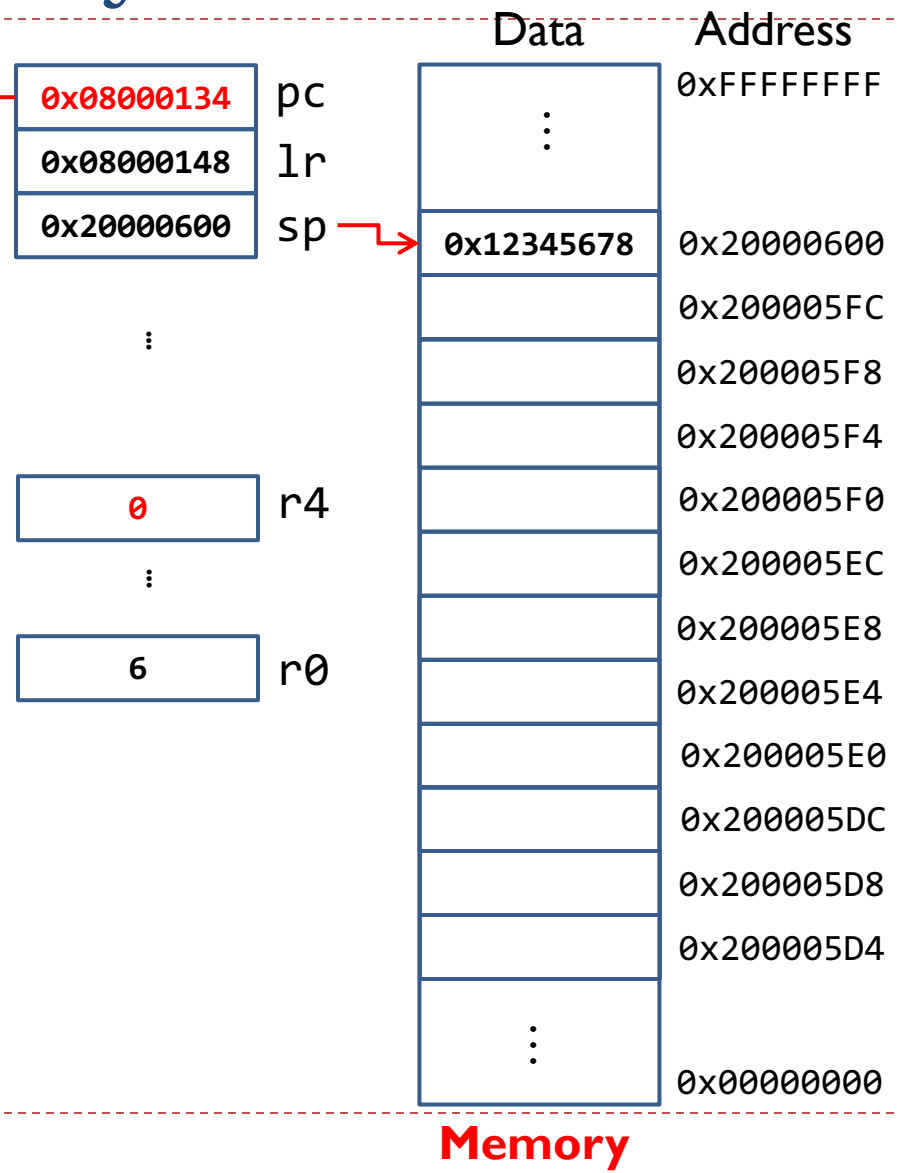
```
                AREA main, CODE, READONLY
                EXPORT  __main

                ENTRY

__main PROC
                MOV r0, #0x03
0x08000130      BL  factorial
0x08000134 stop B   stop
                ENDP

factorial
0x08000136      PUSH {r4, lr}
0x08000138      MOV  r4, r0
0x0800013A      CMP  r4, #0x01
0x0800013C      BNE  NZ
0x0800013E      MOVS r0, #0x01
0x08000140 loop POP  {r4, pc}
0x08000142 NZ   SUBS r0, r4, #1
0x08000144      BL   factorial
0x08000148      MUL  r0, r4, r0
0x0800014C      B    loop

                END
```



Summary

- How to call a subroutine?
  - Branch with link: **`BL subroutine`**
- How to return the control back to the caller?
  - Branch and exchange: **`BX LR`**
- How to pass arguments into a subroutine?
  - Each 8-, 16- or 32-bit variables is passed via `r0`, `r1`, `r2`, `r3`
  - Extra parameters are passed via stack
- How to return a value in a subroutine?
  - Value is returned in `r0`
- What registers should be preserved?
  - Caller-saved registers *vs* callee-saved registers
- How to preserve the running environment for the caller?
  - Via stack